



UNIVERSITÀ DI PISA

Corso di Laurea Magistrale in Informatica
Umanistica

**Sistemi conversazionali aziendali basati
su Large Language Models: confronto
tra RAG nella piattaforma Synapsis ML
e fine-tuning con LoRA**

Relatore:

Prof. Alessandro Bondielli

Candidato:

Emily Di Prizio

Correlatore:

Gianmaria Meriano

ANNO ACCADEMICO 2025/2026

*“Non sono soltanto una cosa, ma anche un modo di essere,
uno dei tanti modi, e conoscere le vie che ho seguito
e quelle che non ho preso mi aiuterà a capire
che cosa sto diventando.”*

DANIEL KEYES

Abstract

L'integrazione di Large Language Models in contesti applicativi aziendali pone una sfida fondamentale, come adattare un modello generale a un dominio specifico garantendo accuratezza, pertinenza e aggiornabilità delle risposte. Il presente lavoro affronta questa sfida confrontando due strategie di adattamento, l'adattamento parametrico tramite Low-Rank Adaptation (LoRA) e l'integrazione di conoscenza esterna tramite una pipeline Retrieval-Augmented Generation (RAG), progettata e sviluppata all'interno di Synapsis ML — piattaforma per l'intelligenza artificiale sviluppata nell'ecosistema Mative — come contributo principale del presente lavoro. Il confronto è condotto attraverso un esperimento controllato applicato al dominio del supporto clienti per un'azienda specializzata nella vendita di macchine da caffè, mantenendo invariati modello base, prompt e parametri di generazione. La valutazione combina metriche automatiche quantitative — tra cui BLEU, ROUGE e BERTScore — con un sistema di giudizio qualitativo basato su LLM-as-a-Judge, articolato su sette dimensioni e tre modalità di prompting distinte. I risultati mostrano che la pipeline RAG produce risposte più accurate, complete e aderenti al dominio tecnico rispetto al modello LoRA, I risultati confermano che un'architettura RAG ben progettata rappresenta una soluzione efficace e accessibile per l'integrazione di capacità conversazionali in sistemi software industriali, superiore all'adattamento parametrico in termini di accuratezza e aggiornabilità della conoscenza.

Indice

1	Introduzione	1
1.1	Motivazioni e obiettivi del lavoro	2
2	Background e stato dell'arte	5
2.1	Large Language Models	5
2.1.1	Overview ed evoluzione dei modelli linguistici	6
2.1.2	Stato e applicazioni dei recenti LLM	13
2.2	Strategie di adattamento dei modelli pre-addestrati	15
2.2.1	Fine-Tuning	15
2.2.2	Parameter-Efficient Fine-Tuning	16
2.2.2.1	Low-Rank Adaptation (LoRA)	17
2.2.3	Adattamento senza aggiornamento dei pesi	17
2.2.3.1	Prompt Engineering	18
2.2.3.2	In-Context Learning	18
2.2.3.3	Strategie di decodifica	19
2.3	Retrieval-Augmented Generation	20
2.3.1	Pipeline di Indicizzazione	21
2.3.1.1	Vector Store	22
2.3.2	Pipeline di Generazione	22
2.3.3	Paradigmi architetturali del RAG	25
2.3.4	RAG Multimodale	27
2.3.5	Framework di sviluppo per sistemi RAG	28
2.3.5.1	Langchain	29
2.4	Metriche di valutazione	30
2.4.1	Metriche per LLM	30
2.4.2	Metriche per RAG	31
2.4.3	Framework di monitoraggio	32
2.4.3.1	MLflow	32
2.5	Lavori correlati	33
3	Architettura del sistema e piattaforma Synapsis ML	35
3.1	Introduzione all'ecosistema	36
3.1.1	Synapsis IoT	37
3.1.2	Mative Cloud	39

3.1.3	Synapsis Analysis	41
3.2	Synapsis ML	43
3.2.1	Architettura della piattaforma	43
3.2.2	Livello di persistenza e modellazione dei dati	45
3.2.2.1	Organizzazione delle tabelle	45
3.2.2.2	Registry dei modelli	46
3.2.3	Gestione dei contesti operativi (Scope)	46
3.2.4	Integrazione con MLflow	47
3.2.5	Esecuzione dei modelli e gestione dei provider	48
3.3	Synapsis ML: componente LLM e architettura RAG	49
3.3.1	Routing e orchestrazione delle capability	50
3.3.1.1	Routing della richiesta	52
3.3.1.2	Selezione ed esecuzione delle capability	53
3.3.1.3	Aggregazione della risposta	54
3.3.1.4	Flusso complessivo	55
3.3.2	Implementazione della pipeline RAG	55
3.3.2.1	Indicizzazione e retrieval documentale	56
3.3.2.2	Costruzione della chain di generazione	59
3.3.3	Gestione delle sorgenti informative	62
3.3.3.1	Documenti testuali	62
3.3.3.2	Immagini e contenuti multimodali	66
3.3.3.3	Basi di dati relazionali	67
3.3.3.4	Cronologia conversazionale	69
3.3.4	Scelte progettuali e vincoli implementativi	70
4	Setup sperimentale e presentazione di un caso d'uso	73
4.1	Dominio di applicazione	73
4.2	Scelta del modello	74
4.2.1	Utilizzo del modello in ambiente di addestramento (LoRA)	75
4.2.2	Utilizzo del modello in fase di inferenza (RAG in Synapsis ML)	75
4.3	Adattamento tramite LoRA	76
4.3.1	Dataset di addestramento	76
4.3.2	Training dell'adapter LoRA	77
4.3.2.1	Configurazione del training	78
4.3.2.2	Valutazione del training	79
4.4	Fase di inferenza	81
4.4.1	Prompt e parametri di generazione	81
4.4.1.1	Definizione del prompt	82

4.4.1.2	Parametri di generazione	84
4.4.2	Modalità di inferenza	85
4.4.2.1	Costruzione del test set	85
4.4.2.2	Procedura di inferenza	86
4.5	Valutazione	87
4.5.1	Valutazione automatica	88
4.5.1.1	Metriche di valutazione	88
4.5.1.2	Modello adattato tramite LoRA	89
4.5.1.3	Pipeline RAG	92
4.5.2	LLM as a judge	96
4.5.2.1	Definizione del sistema di valutazione	97
4.5.2.2	Analisi dei punteggi	101
5	Discussione dei risultati	108
5.1	Sintesi dei risultati	108
5.2	Interpretazione delle differenze tra RAG e LoRA	109
5.3	Limiti del sistema	115
6	Conclusioni	117
6.1	Sviluppi futuri	118

Elenco delle figure

2.1	Architettura dei modelli Word2Vec: CBOW e Skip-Gram (Mikolov et al. 2013)	9
2.2	Architettura Transformer (Vaswani et al. 2017)	11
2.3	Rappresentazione cronologica dei principali rilasci di Large Language Models. I riquadri blu indicano modelli esclusivamente pre-addestrati, mentre quelli arancioni identificano modelli sottoposti a <i>instruction tuning</i> . I modelli collocati nella parte superiore del diagramma sono distribuiti come open-source, mentre quelli nella parte inferiore sono closed-source (Naveed et al. 2023).	14
2.4	Fasi di pre-addestramento e fine-tuning per BERT (Devlin et al. 2019). . .	16
2.5	Diversi approcci di Parameter-Efficient Fine-Tuning (Naveed et al. 2023). .	17
2.6	Esempio di architettura RAG per il question answering open-domain (Minaee et al. 2024).	20
2.7	Fasi del caricamento dei dati nella pipeline di indicizzazione di un sistema RAG (Kimothi 2025).	21
2.8	Processo di generazione in un sistema RAG (Kimothi 2025).	23
2.9	Confronto tra le principali configurazioni architetturali: Naive RAG, Advanced RAG e Modular RAG (Y. Gao et al. 2023).	25
2.10	Tre approcci per la pipeline di indicizzazione nel RAG multimodale (Kimothi 2025).	28
3.1	Architettura generale dell’ecosistema Mative. Il diagramma illustra l’integrazione tra i dispositivi IoT, l’infrastruttura cloud e le piattaforme Synapsis dedicate alla gestione dei dati, all’analisi e all’intelligenza artificiale.	36
3.2	Architettura della piattaforma Synapsis IoT	38
3.3	Interfaccia della piattaforma Mative Cloud per il monitoraggio e la gestione di dispositivi e flotte tramite dashboard interattive	40
3.4	Architettura della piattaforma Synapsis ML	44
3.5	Workflow di esecuzione dei modelli in Synapsis ML	48
3.6	Interfaccia della componente LLM di Synapsis ML. La schermata consente di configurare il comportamento del sistema selezionando modello e provider, definendo i parametri di generazione e integrando diverse sorgenti informative, quali documenti, immagini e basi di dati, che vengono utilizzate durante il processo di generazione della risposta.	49

3.7	Schema semplificato dell'architettura della componente LLM. L'orchestratore coordina il flusso di esecuzione operando su un contesto condiviso e delegando le operazioni ai moduli specializzati.	50
4.1	Andamento della training loss e validation loss durante il training LoRA .	79
4.2	Metriche di valutazione calcolate durante il training LoRA	80
4.3	Metriche medie ottenute dal modello LoRA sul test set	89
4.4	Relazione tra BERTScore F1 e Token F1 per le risposte generate dal modello LoRA	90
4.5	Metriche medie ottenute dalla pipeline RAG sul test set	92
4.6	Relazione tra BERTScore F1 e Token F1 per le risposte generate dalla pipeline RAG	94
4.7	Profilo qualitativo medio per dimensione nelle tre modalità di prompting. Il poligono blu (RAG) risulta più esteso di quello arancione (LoRA) in quasi tutte le direzioni, con l'unica eccezione della dimensione <i>conciseness</i> . 103	
4.8	Δ Score (LoRA–RAG) per dimensione e modalità di prompting. Valori negativi (in rosso) indicano superiorità della pipeline RAG; l'unico valore positivo sistematico (in verde) riguarda la dimensione <i>conciseness</i>	104
4.9	Distribuzione dei punteggi <i>overall_score</i> assegnati a RAG e LoRA per ciascuna modalità di prompting. La pipeline RAG mostra una distribuzione ampia e spostata verso i valori alti della scala; il modello LoRA presenta una concentrazione marcata sul valore 1.	104
4.10	Distribuzione del delta <i>overall_score</i> (LoRA–RAG) per modalità di prompting. Le barre blu indicano istanze in cui RAG è risultato migliore, le barre arancioni istanze in cui LoRA è risultato migliore. La linea rossa indica il valore medio del delta; la linea tratteggiata nera indica lo zero.	105
4.11	Esempio di istanza in cui il modello LoRA ottiene punteggi superiori alla pipeline RAG.	106
4.12	Esempio di istanza in cui il modello RAG ottiene punteggi superiori alla pipeline LoRA.	106

1. Introduzione

Negli ultimi anni, i Large Language Models (LLM) hanno trasformato profondamente il panorama del Natural Language Processing, dimostrando capacità avanzate nella comprensione e generazione del linguaggio naturale. Modelli basati su architetture Transformer hanno reso possibile affrontare con successo una vasta gamma di task, dall'assistenza conversazionale alla generazione automatica di contenuti, fino al supporto decisionale in contesti applicativi complessi. Tuttavia, queste capacità emergono principalmente in contesti general-purpose, mentre la loro applicazione in domini specifici introduce nuove sfide legate alla correttezza fattuale, alla pertinenza delle risposte e alla necessità di integrare conoscenza aggiornata e contestualizzata.

In ambito industriale, tali criticità risultano particolarmente rilevanti. I sistemi basati su LLM devono infatti operare su dati proprietari, interagire con basi di conoscenza eterogenee e garantire affidabilità in scenari in cui errori o imprecisioni possono avere impatti concreti. In questo contesto, il problema dell'adattamento dei modelli general-purpose a domini applicativi specifici assume un ruolo centrale, richiedendo soluzioni che bilancino qualità delle risposte, costi computazionali e facilità di integrazione nei sistemi esistenti.

Le strategie di adattamento disponibili si collocano su uno spettro che va dalla modifica diretta dei parametri del modello, come nel fine-tuning e nelle sue varianti efficienti, all'integrazione di conoscenza esterna in fase di inferenza, come nella Retrieval-Augmented Generation (RAG). Ciascun approccio presenta vantaggi e compromessi, e la scelta tra di essi dipende dal contesto applicativo, dalle risorse disponibili e dai requisiti di qualità e aggiornabilità del sistema.

Il presente lavoro si inserisce in questo scenario, con l'obiettivo di analizzare e confrontare due strategie di adattamento degli LLM applicate a un caso d'uso concreto: lo sviluppo di un chatbot aziendale per il supporto clienti nel dominio delle macchine da caffè. Le due strategie considerate sono l'adattamento parametrico tramite Low-Rank Adaptation (LoRA), tecnica di fine-tuning efficiente in termini di parametri, e l'integrazione di conoscenza esterna tramite una pipeline Retrieval-Augmented Generation implementata all'interno della piattaforma Synapsis ML, un sistema software progettato per l'integrazione di capacità di intelligenza artificiale in contesti industriali.

L'obiettivo del lavoro non è limitato alla valutazione delle prestazioni assolute dei due approcci, ma si estende alla comprensione delle loro caratteristiche distintive, dei pattern di errore e dei compromessi che ciascuno introduce nel contesto applicativo considerato. La valutazione viene condotta attraverso una combinazione di metriche automatiche quantitative e un sistema di valutazione qualitativa basato su LLM-as-a-Judge, al fine di

ottenere una prospettiva completa e multi-livello sulla qualità delle risposte generate.

Il lavoro di tesi è nato nell'ambito delle attività di ricerca e sviluppo svolte presso Mative S.r.l., azienda attiva nel settore delle soluzioni IoT e delle piattaforme software per applicazioni industriali. L'azienda sviluppa soluzioni orientate alla gestione di dispositivi connessi, al monitoraggio di infrastrutture e all'analisi dei dati provenienti da sistemi distribuiti, operando in ambiti quali Smart Industry, Smart Mobility e Smart Farming.

Nel corso delle attività aziendali è emersa la necessità di integrare funzionalità basate su Large Language Models all'interno dei sistemi software sviluppati da Mative, con particolare interesse verso strumenti conversazionali in grado di supportare l'accesso a documentazione tecnica, dati operativi e conoscenza specialistica di dominio. La crescente disponibilità di informazioni eterogenee e la necessità di renderle facilmente interrogabili hanno motivato l'avvio di attività di sperimentazione legate all'utilizzo di modelli linguistici e pipeline Retrieval-Augmented Generation in scenari applicativi reali.

Parallelamente a queste esigenze, Mative ha avviato lo sviluppo della piattaforma Synapsis ML, progettata per facilitare l'integrazione di modelli di intelligenza artificiale e componenti di machine learning all'interno dell'infrastruttura software aziendale. Il progetto di tesi si inserisce all'interno di questo percorso di sviluppo e sperimentazione, con l'obiettivo di analizzare le problematiche legate all'adattamento degli LLM a domini specifici e valutare differenti strategie per la realizzazione di sistemi conversazionali basati su conoscenza aziendale.

1.1 Motivazioni e obiettivi del lavoro

Nonostante la crescente diffusione di piattaforme dedicate allo sviluppo di applicazioni basate su Large Language Models, l'implementazione di sistemi conversazionali basati su Retrieval-Augmented Generation in contesti applicativi reali presenta ancora diverse criticità. In particolare, l'integrazione di dati provenienti da fonti eterogenee, la definizione dei prompt e la regolazione dei parametri di generazione richiedono spesso un processo iterativo di sperimentazione e adattamento.

Un ulteriore aspetto riguarda la gestione dei meccanismi di recupero delle informazioni e la loro integrazione con i modelli generativi, che può influenzare in modo significativo la qualità e la coerenza delle risposte prodotte dal sistema.

Inoltre, molte delle applicazioni descritte nella letteratura e nella documentazione tecnica si concentrano prevalentemente sugli aspetti architetturali o sulle capacità dei modelli linguistici, mentre risulta meno approfondita la descrizione delle problematiche pratiche legate alla configurazione, alla valutazione e all'ottimizzazione di sistemi basati su Retrieval-Augmented Generation in scenari applicativi concreti.

Alla luce di queste considerazioni, il lavoro presentato in questa tesi si propone di analizzare il processo di progettazione, configurazione e ottimizzazione di sistemi conversazionali basati su RAG applicati a dati specifici di dominio. L'obiettivo è esaminare le principali scelte progettuali e le difficoltà operative che emergono durante l'integrazione tra modelli linguistici, meccanismi di retrieval e fonti di dati eterogenee, fornendo un'analisi del processo di sviluppo e delle strategie adottate per migliorare le prestazioni del sistema in un contesto applicativo reale.

Il presente lavoro è organizzato come segue:

Il **Capitolo 2** fornisce il quadro teorico e tecnologico di riferimento. Viene ripercorsa l'evoluzione dei modelli linguistici, dalle prime formulazioni statistiche basate su n-grammi fino all'architettura Transformer e ai moderni LLM. Vengono quindi descritte le principali strategie di adattamento dei modelli pre-addestrati, con particolare attenzione alla tecnica LoRA e ai metodi di adattamento senza aggiornamento dei pesi, quali il prompt engineering e l'in-context learning. Il capitolo introduce inoltre l'architettura Retrieval-Augmented Generation nelle sue diverse varianti, dai sistemi Naive RAG alle architetture modulari più avanzate, e presenta le metriche e i framework di valutazione rilevanti per l'analisi di sistemi basati su LLM. La sezione conclusiva del capitolo posiziona il presente lavoro rispetto allo stato dell'arte, motivando le scelte metodologiche adottate.

Il **Capitolo 3** descrive l'architettura del sistema oggetto di studio, la piattaforma Synapsis ML. Viene innanzitutto introdotto l'ecosistema tecnologico di riferimento, composto da Synapsis IoT per la gestione dei dispositivi connessi, Mative Cloud per il monitoraggio e la gestione delle flotte, e Synapsis Analysis per l'analisi dei dati. Il capitolo si concentra poi sull'architettura di Synapsis ML, descrivendone il livello di persistenza, il registry dei modelli, la gestione dei contesti operativi e l'integrazione con MLflow. La parte centrale del capitolo è dedicata alla componente LLM della piattaforma, viene illustrato il meccanismo di orchestrazione dinamica basato su capability, e viene descritta l'implementazione della pipeline RAG, comprensiva della gestione di documenti testuali, contenuti multimodali, basi di dati relazionali e cronologia conversazionale. Il capitolo si chiude con un'analisi delle scelte progettuali e dei vincoli implementativi che hanno caratterizzato lo sviluppo della piattaforma.

Il **Capitolo 4** presenta il setup sperimentale e il caso d'uso adottato per il confronto tra le due strategie di adattamento. Viene descritto il dominio applicativo, un chatbot per il supporto clienti nel settore delle macchine da caffè. Segue la descrizione del processo di creazione e addestramento del modello adattato tramite LoRA, includendo la costruzione del dataset sintetico, la configurazione dell'adapter e l'analisi dell'andamento del training. Successivamente, sono illustrate le modalità di configurazione dei due approcci

a confronto. Il capitolo prosegue con la fase di inferenza e la costruzione del test set a partire da fonti reali, per poi introdurre la procedura di valutazione. Quest'ultima comprende sia una valutazione automatica, basata su metriche lessicali e semantiche (BLEU, ROUGE, BERTScore, METEOR, chrF, Exact Match, Token F1), sia una valutazione qualitativa basata su LLM-as-a-Judge, articolata su sette dimensioni e condotta in tre modalità di prompting distinte (zero-shot, one-shot, few-shot).

Il **Capitolo 5** discute i risultati ottenuti dall'esperimento, integrandone la lettura quantitativa con un'interpretazione qualitativa approfondita. Vengono analizzati i risultati delle metriche automatiche e del giudice LLM, mettendo in evidenza i punti di forza e le debolezze di ciascun approccio e interpretando i pattern di errore osservati. Il capitolo include una sezione dedicata all'analisi critica della pipeline RAG implementata in Synapsis ML, con l'obiettivo di comprenderne il funzionamento in un contesto applicativo reale e i limiti del sistema, focalizzando lo studio principalmente sui vincoli architetturali e implementativi della piattaforma, sulle criticità della fase di retrieval e sugli aspetti che influenzano la qualità delle risposte generate.

Il **Capitolo 6** trae le conclusioni del lavoro, sintetizzando i principali contributi e le implicazioni dei risultati ottenuti. Particolare enfasi è posta sul ruolo della pipeline RAG all'interno della piattaforma Synapsis ML, evidenziandone il valore in termini di flessibilità, scalabilità e integrazione in un sistema reale. Il capitolo si concentra inoltre sulle possibili direzioni di sviluppo futuro, con l'obiettivo di migliorare le prestazioni e la robustezza della piattaforma.

2. Background e stato dell'arte

I recenti progressi nel campo dell'intelligenza artificiale hanno portato all'affermazione dei *Large Language Models* (LLM) come elemento centrale di numerosi sistemi di elaborazione del linguaggio naturale. Tali modelli non si limitano alla generazione di testo, ma costituiscono oggi il nucleo computazionale di architetture più complesse, in grado di integrare conoscenza, dati eterogenei e capacità di ragionamento all'interno di pipeline applicative articolate.

Il presente capitolo ha l'obiettivo di fornire una panoramica dello stato dell'arte delle principali tecnologie alla base degli LLM, con particolare attenzione agli aspetti teorici e architetturali rilevanti per il loro impiego in contesti applicativi reali. In particolare, verrà ripercorsa l'evoluzione dei modelli linguistici, dalle prime formulazioni statistiche fino all'introduzione dell'architettura *Transformer*, fondamento dei moderni LLM.

A partire da tale architettura, verranno introdotte le principali strategie di utilizzo e adattamento dei modelli pre-addestrati, tra cui il *fine-tuning*, nonché le modalità di integrazione degli LLM all'interno di sistemi più complessi. Un'attenzione specifica sarà dedicata alla gestione di dati eterogenei e al concetto di multimodalità.

Infine, il capitolo introduce le architetture di *Retrieval-Augmented Generation* (RAG), che consentono di combinare le capacità generative degli LLM con meccanismi di recupero dell'informazione esterna, superando alcune delle limitazioni dei modelli utilizzati in modo isolato. Verranno inoltre presentate le principali metriche e i framework di valutazione utilizzati per analizzare le prestazioni di tali sistemi.

Nel complesso, questo capitolo fornisce il quadro teorico e tecnologico di riferimento necessario per comprendere le soluzioni progettate e implementate nel prosieguo del lavoro.

2.1 Large Language Models

I *Large Language Models* (LLM) sono modelli linguistici statistici basati su reti neurali profonde, pre-addestrati su larga scala su grandi quantità di dati testuali non etichettati (Minaee et al. 2024). Essi rappresentano oggi il paradigma dominante nel *Natural Language Processing* (NLP), grazie alla loro capacità di adattarsi a un'ampia varietà di compiti a valle, tra cui traduzione automatica, risposta a domande, sintesi e dialogo.

Il recente successo degli LLM, accentuato dalla diffusione di sistemi conversazionali come *ChatGPT*, rilasciato nel novembre 2022, ha evidenziato un cambiamento di paradigma nell'NLP. Tali modelli vengono oggi impiegati come modelli generali pre-addestrati

riutilizzabili e adattabili a una pluralità di compiti, in contrasto con il precedente approccio basato sull'addestramento di modelli distinti per ciascuna applicazione. Questa nuova modalità di utilizzo ha contribuito a rafforzare l'impressione di una maggiore generalità e flessibilità dei sistemi di comprensione del linguaggio naturale, come discusso da Lenci (2023), pur senza implicare necessariamente una piena comprensione del linguaggio in senso cognitivo. Nonostante le elevate prestazioni empiriche, tali modelli rimangono fondati su principi probabilistici di modellazione del linguaggio, la cui origine risale alle prime formulazioni statistiche della seconda metà del Novecento.

2.1.1 Overview ed evoluzione dei modelli linguistici

I modelli linguistici nascono con l'obiettivo di assegnare una probabilità a sequenze di parole, fornendo una rappresentazione quantitativa della plausibilità linguistica di frasi e testi. Questa formulazione risulta fondamentale in numerosi compiti di NLP caratterizzati da ambiguità e rumore, come il riconoscimento vocale, la correzione ortografica e la traduzione automatica (Jurafsky e Martin 2023).

Formalmente, data una sequenza di parole $w_1, w_2, \dots, w_n$, un modello linguistico stima la probabilità congiunta della sequenza come:

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1}) \quad (2.1)$$

Il problema della modellazione linguistica può quindi essere ricondotto alla stima della probabilità di una parola dato il contesto precedente. Tuttavia, stimare direttamente la probabilità di una parola dato l'intero contesto precedente risulta impraticabile a causa della scarsità dei dati osservabili. Una possibile strategia consiste nel limitare la dipendenza della probabilità di una parola a un numero finito di parole precedenti, un approccio che porta alla definizione dei modelli linguistici statistici a n-grammi (P. F. Brown et al. 1990).

Modello a N-grammi

I modelli linguistici a n-grammi sono modelli probabilistici basati sull'assunzione di Markov, secondo cui la probabilità di una parola può essere approssimata considerando soltanto un numero finito di parole precedenti. Applicando tale assunzione ogni termine condizionale viene stimato come:

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-(n-1)}, \dots, w_{i-1}) \quad (2.2)$$

riducendo così la dipendenza dal contesto completo a una finestra locale di dimensione fissa e rendendo il problema della stima probabilistica computazionalmente trattabile (P. F.

Brown et al. 1990). Nel caso più semplice, il modello bigramma assume che la probabilità di una parola dipenda unicamente dalla parola immediatamente precedente. Più in generale, i modelli a n -gramma estendono tale dipendenza a contesti progressivamente più ampi (Jurafsky e Martin 2023). Limitare il contesto a un numero finito di parole equivale ad assumere una forma di dipendenza locale tra di esse. Infatti, come sottolineato da Church e Mercer (1993), molte regolarità statistiche del linguaggio naturale possono essere catturate efficacemente considerando contesti brevi.

Nonostante la loro semplicità, i modelli a n -gramma hanno avuto un ruolo fondamentale, ottenendo buoni risultati in applicazioni come il riconoscimento vocale e la traduzione automatica statistica. Come osservato da Brill et al. (1998), il loro successo ha favorito un approccio basato sull'uso di grandi corpora e su metodi statistici essenziali, senza fare ricorso a una rappresentazione esplicita della struttura linguistica. Tuttavia, questi modelli presentano alcuni limiti. Poiché le probabilità degli n -grammi sono ricavate dalle frequenze relative osservate in un corpus di addestramento, all'aumentare dell'ordine n il numero di possibili sequenze cresce esponenzialmente, generando un marcato problema di sparsità dei dati: molte sequenze linguisticamente plausibili non compaiono nel corpus e ricevono quindi probabilità nulle. Al tempo stesso, tale difficoltà evidenzia un limite più profondo dell'assunzione di Markov, che restringe il contesto rilevante a una finestra fissa di parole precedenti, impedendo al modello di catturare dipendenze a lungo raggio e motivando così la ricerca verso modelli linguistici più espressivi e flessibili.

Word Embeddings

Per superare i limiti derivanti dalla rappresentazione delle parole come simboli puramente discreti nei modelli a n -grammi, la ricerca in NLP ha progressivamente riscoperto l'ipotesi distribuzionale, formulata da Harris (1954) e Firth (1957), secondo cui il significato di una parola può essere inferito osservando i contesti in cui essa compare. Questa intuizione ha dato origine ai primi modelli di semantica vettoriale, nei quali le parole vengono rappresentate tramite vettori costruiti a partire dalle loro co-occorrenze nel testo (Jurafsky e Martin 2023).

Nei primi approcci iniziali, noti come modelli distribuzionali count-based, queste rappresentazioni consentono di catturare relazioni distribuzionali significative, ma producono vettori ad altissima dimensionalità (proporzionale al vocabolario o al numero di documenti) e fortemente sparsi. Di conseguenza, esse mantengono parte dei problemi di scalabilità e generalizzazione già osservati nei modelli a n -gramma (Jurafsky e Martin 2023).

Per ovviare a tali limiti, sono state introdotte tecniche di riduzione dimensionale, come la *Singular Value Decomposition* (SVD), che proiettano queste rappresentazioni sparse in uno spazio vettoriale a dimensionalità ridotta e più denso. I vettori così ottenuti, noti come

Word Embeddings, concentrano l'informazione semantica in vettori compatti, tipicamente di poche centinaia di dimensioni, in cui ogni componente contribuisce alla codifica del significato della parola.

È in questo passaggio che le parole iniziano effettivamente a essere interpretate come punti in uno spazio geometrico continuo, in cui la vicinanza riflette una forma di similarità semantica. Questo approccio è stato ulteriormente sviluppato dai modelli predittivi neurali che apprendono direttamente rappresentazioni dense e a bassa dimensionalità ottimizzando un obiettivo predittivo sui contesti linguistici.

Neural Language Model

I *Neural Language Models* (NLM) utilizzano reti neurali artificiali per apprendere word embeddings e modellare la probabilità di sequenze linguistiche. Le reti neurali artificiali sono modelli computazionali composti da unità elementari chiamate neuroni artificiali, organizzate in strati (uno di input, uno o più strati nascosti e uno di output). Ogni neurone riceve input, li combina tramite pesi associati e applica una funzione di attivazione non lineare per produrre un output. Durante l'addestramento, tali pesi vengono ottimizzati su grandi quantità di dati per ridurre l'errore tra predizioni e valori attesi.

A differenza dei modelli a n-gramma, gli NLM imparano direttamente a predire la parola successiva a partire dal contesto osservato. Durante l'addestramento, le parole precedenti vengono trasformate in un vettore continuo, che sintetizza l'informazione rilevante del contesto e permette al modello di generare una distribuzione di probabilità sul vocabolario, trattando la predizione della parola successiva come un problema di classificazione. Il modo in cui questa codifica del contesto viene realizzata varia tra le diverse architetture neurali.

Uno dei primi approcci agli NLM è stato proposto da Bengio et al. (2003) e si basa su una rete neurale *Feed Forward*. In questo modello, le parole del contesto, rappresentate inizialmente tramite vettori one-hot, vengono proiettate in uno spazio continuo distribuito. I vettori ottenuti sono concatenati e utilizzati per predire la parola successiva. L'obiettivo è massimizzare la probabilità della parola corretta dato il contesto tramite una funzione di perdita basata sulla cross-entropy. Un limite rilevante di questo approccio è la necessità di fissare a priori la dimensione del contesto.

Un'evoluzione significativa in questa direzione è rappresentata da Word2Vec introdotto da Mikolov et al. (2013). Word2Vec è basato su due principali architetture neurali, Continuous Bag-of-Words (CBOW) e Skip-gram, entrambe progettate per apprendere automaticamente i word embedding come parametri interni della rete durante l'addestramento, ottimizzando un compito di predizione basato sul contesto.

- **CBOW**: data una finestra di contesto di ampiezza prefissata, il modello predice la parola centrale a partire dalle parole che la circondano. È un modello più veloce da addestrare e tende a funzionare bene con corpora di grandi dimensioni. Grazie alla combinazione dei contesti, produce embedding meno rumorosi, risultando particolarmente efficace nel catturare regolarità sintattiche e pattern frequenti nel linguaggio;
- **Skip-Gram**: utilizza la parola centrale per predire le parole che la circondano, all'interno di una finestra di contesto di ampiezza prefissata. È generalmente più efficace nel catturare relazioni semantiche sottili e funziona meglio con parole rare, poiché apprende molteplici predizioni a partire dalla stessa parola target. Produce embedding di qualità superiore per la rappresentazione del significato lessicale.

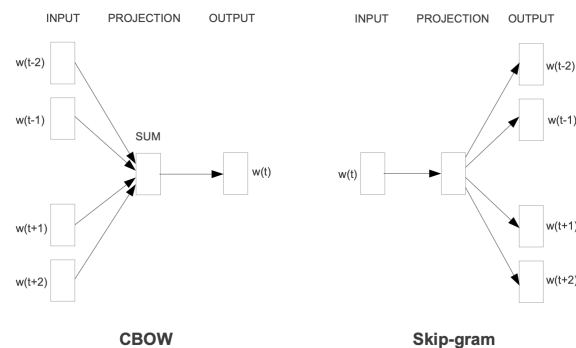


Figura 2.1: Architettura dei modelli Word2Vec: CBOW e Skip-Gram (Mikolov et al. 2013)

Nonostante la loro efficacia nella rappresentazione distribuzionale delle parole, questi modelli presentano alcuni limiti. La lunghezza del contesto deve essere predefinita, e le relazioni a lungo raggio tra parole non vengono catturate. Inoltre, i vettori appresi sono statici, ovvero ogni parola ha un unico embedding indipendentemente dai diversi significati che può assumere in contesti differenti. Queste limitazioni hanno motivato lo sviluppo di architetture più avanzate in grado di gestire sequenze di lunghezza variabile e catturare informazioni contestuali più ricche.

Reti Neurali Ricorrenti

Le Reti Neurali Ricorrenti (RNN) rappresentano un'evoluzione naturale dei modelli *Feed Forward* per la modellazione del linguaggio. Le RNN introducono una rappresentazione esplicita della dimensione temporale, consentendo l'elaborazione incrementale delle sequenze parola per parola, superando i limiti strutturali dei modelli precedenti con contesto

fisso (Jurafsky e Martin 2023). Formalmente, una RNN è caratterizzata dalla presenza di connessioni ricorrenti all'interno dello strato nascosto, che permettono allo stato interno della rete al tempo t di dipendere non solo dall'input corrente, ma anche dallo stato nascosto calcolato al passo temporale precedente, $t - 1$, il quale a sua volta dipende da $t - 2$, e così via. In questo modo, al tempo t il modello elabora il token corrente utilizzando informazioni derivate anche dai passi temporali precedenti. Tale meccanismo introduce una forma di memoria dinamica, in cui lo stato nascosto funge da rappresentazione compatta del contesto passato e consente al modello di effettuare predizioni tenendo conto dell'informazione accumulata nel tempo (Tarwani e Edem 2017). Questo permette, almeno in linea di principio, di modellare dipendenze linguistiche di lunghezza arbitraria.

Tuttavia le informazioni codificate nello stato nascosto tendono a essere fortemente influenzate dagli input più recenti, mentre i contributi provenienti da passi temporali lontani vengono progressivamente attenuati. Questo problema è causato dal fenomeno del *vanishing gradients*, che emerge durante la retropropagazione dell'errore nel tempo (*Backpropagation Through Time*, BPTT). Poiché il gradiente viene moltiplicato ripetutamente attraverso i passi temporali, esso può decrescere esponenzialmente, impedendo un apprendimento efficace delle dipendenze a lungo raggio (Jurafsky e Martin 2023).

Per affrontare questo limite, sono state introdotte architetture ricorrenti più sofisticate, tra cui le reti *Long Short-Term Memory* (LSTM), proposte da Hochreiter e Schmidhuber (1997). Le LSTM estendono le RNN standard introducendo un meccanismo esplicito di gestione della memoria, separando il problema della memorizzazione delle informazioni da quello della loro elaborazione. In particolare, le LSTM introducono celle di memoria dotate di connessioni auto-ricorrenti e unità di controllo, dette *gate*, che regolano il flusso di informazioni in ingresso, in uscita e all'interno della cella. Le LSTM apprendono dinamicamente quali informazioni mantenere nel tempo e quali scartare, consentendo un flusso di errore più stabile durante l'addestramento. Nonostante questi vantaggi, le LSTM richiedono un numero maggiore di parametri e risultano più costose da addestrare. Inoltre, faticano a risolvere compiti che richiedono il mantenimento simultaneo e combinatorio di informazioni distanti nel tempo, soprattutto in scenari non scomponibili in sotto-obiettivi incrementali (Hochreiter e Schmidhuber 1997).

Un ulteriore passo evolutivo nell'uso delle architetture ricorrenti è rappresentato dai modelli *Sequence-to-Sequence* (Seq2Seq), basati su un'architettura *Encoder-Decoder*. Proposti inizialmente da Sutskever, Vinyals e Le (2014), questi modelli affrontano il problema della mappatura tra sequenze di lunghezza variabile, come nel caso della traduzione automatica, in cui la sequenza di input e quella di output possono avere lunghezze diverse e relazioni non monotone. L'architettura prevede due LSTM distinti. L'Encoder legge l'intera sequenza di input producendo una rappresentazione vettoriale a dimensione fis-

sa che cattura le informazioni essenziali dell'input. Il Decoder utilizza questo vettore di contesto per generare la sequenza di output. Questo approccio consente di separare concettualmente il processo di comprensione dell'input da quello di generazione dell'output, mantenendo una formulazione end-to-end e indipendente dal dominio (Sutskever, Vinyals e Le 2014)..

La compressione dell'intera sequenza di input in un singolo vettore a dimensione fissa introduce però un limite informativo, penalizzando le prestazioni su sequenze lunghe o semanticamente complesse. Inoltre, la natura strettamente sequenziale del modello comporta difficoltà di parallelizzazione, rendendo l'addestramento inefficiente su grandi quantità di dati. Tali limiti hanno motivato l'introduzione dei Transformer, modelli basati su un meccanismo di attenzione in grado di catturare dipendenze a lungo raggio in maniera parallela e scalabile, aprendo una nuova era per l'elaborazione del linguaggio naturale.

Modelli Transformer

Nel 2017 Vaswani et al. (2017) hanno introdotto l'architettura Transformer nel lavoro “*Attention Is All You Need*”. Questo modello ha rappresentato un punto di svolta nella modellazione del linguaggio naturale, eliminando completamente la necessità di componenti ricorrenti o convoluzionali.

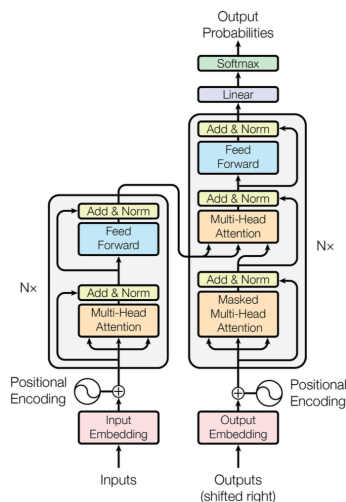


Figura 2.2: Architettura Transformer (Vaswani et al. 2017)

L'architettura Transformer mantiene la struttura encoder-decoder tipica dei modelli Seq2Seq, ma sostituisce le catene sequenziali tipiche delle RNN con stack di strati basati su meccanismi di attenzione e reti feed-forward che operano in parallelo. Sia l'encoder sia il decoder sono composti da 6 layer identici. In ciascun layer si trovano sottocomponenti di *multi-head self-attention* e reti *feed forward*. Il decoder aggiunge a questa struttura un

ulteriore sottolivello di *masked multi-head attention*, che impedisce al modello di accedere ai token futuri durante la generazione.

Il cuore del Transformer è il meccanismo di *self-attention*, che permette al modello di determinare quali parole della sequenza siano più rilevanti per interpretare ciascun token. In altre parole, ogni parola guarda tutte le altre parole della frase e assegna a ciascuna un livello di importanza, così da costruire una rappresentazione che tenga conto del contesto globale. Per realizzarlo, si utilizza una *attention function*, che prende in input tre vettori per ciascun token:

- **Query (Q)**: rappresenta le informazioni che il token sta cercando negli altri;
- **Key (K)**: descrive quali informazioni ogni token può offrire;
- **Value (V)**: contiene il contenuto informativo che verrà effettivamente combinato.

La funzione di attenzione confronta la *query* di ciascun token con le *key* di tutti gli altri token, calcolando un punteggio che misura quanto un token sia rilevante per un altro. Questi punteggi vengono poi normalizzati tramite *softmax* per ottenere pesi tra 0 e 1, che indicano l'importanza relativa di ciascun token nel contesto. Infine, questi pesi vengono usati per combinare i vettori *value*, generando una rappresentazione aggiornata di ogni token. Formalmente, l'attenzione è definita come:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.3)$$

dove d_k è la dimensione dei vettori *key* e serve a evitare che i valori del prodotto scalare diventino troppo grandi e instabili per il *softmax*. Per processare l'intera sequenza in parallelo, l'attenzione viene calcolata simultaneamente per tutte le query impacchettandole in un'unica matrice Q . Il risultato finale è una nuova rappresentazione per ogni token, ottenuta come combinazione pesata dei vettori *value* degli altri token. In questo modo, ogni parola non è più rappresentata isolatamente, ma attraverso un embedding che incorpora le informazioni rilevanti provenienti dall'intera sequenza. Si ottengono così rappresentazioni ricche di informazione contestuale, in cui il significato di ciascun termine è modellato in funzione delle parole che lo circondano (Jurafsky e Martin 2023).

Nell'architettura proposta da Vaswani et al. (2017), il meccanismo di attenzione viene esteso mediante la *multi-head attention*. Invece di applicare un'unica funzione di attenzione su vettori di dimensione d_{model} , le query, key e value vengono proiettate linearmente h volte tramite diverse trasformazioni apprese, ottenendo h rappresentazioni in sottospazi distinti. Su ciascuna proiezione si calcola in parallelo una funzione di attenzione:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V) \quad (2.4)$$

dove W_i^Q , W_i^K e W_i^V sono matrici di proiezione apprese. Le h teste ottenute vengono quindi concatenate e nuovamente proiettate:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O \quad (2.5)$$

con W^O matrice di proiezione per l'output. La *multi-head attention* consente al modello di attendere simultaneamente informazioni provenienti da diversi sottospazi di rappresentazione e da posizioni differenti della sequenza. Questo è fondamentale poiché, all'interno di una frase, le parole possono essere legate da relazioni sintattiche, semantiche o discorsive che un singolo meccanismo di attenzione difficilmente riesce a catturare contemporaneamente (Jurafsky e Martin 2023).

Poiché il meccanismo di attenzione da solo non cattura l'ordine dei token, i Transformer introducono informazioni sulla posizione tramite *positional encoding*, che vengono sommate agli embedding in ingresso dell'encoder e del decoder. Inoltre, ogni sottolivello del Transformer è avvolto da una connessione residua seguita da una normalizzazione (*LayerNorm*), permettendo un passaggio diretto dell'informazione attraverso lo stack di layer e facilitando l'ottimizzazione del modello.

La loro capacità di modellare relazioni complesse tra parole, anche a lungo raggio, e di processare le sequenze in parallelo li rende la base di tutti gli LLM attualmente in uso. Questi modelli vengono tipicamente pre-addestrati su grandi corpora di testo permettendo loro di apprendere rappresentazioni linguistiche generali che possono poi essere adattate a specifici compiti tramite *fine-tuning* o strategie di *prompting*.

2.1.2 Stato e applicazioni dei recenti LLM

I recenti progressi nei modelli linguistici di grandi dimensioni basati su architettura Transformer hanno determinato un significativo aumento delle loro capacità. Tale miglioramento è riconducibile principalmente alla crescita della quantità e varietà dei dati utilizzati nel pre-addestramento, all'aumento della scala dei modelli in termini di numero di parametri e alla progressiva ottimizzazione delle strategie di addestramento.

Modelli come *GPT-3* hanno evidenziato come l'incremento sostanziale della dimensione del modello possa tradursi in migliori capacità di generalizzazione e in prestazioni competitive. Successivamente, architetture quali *PaLM* e *Chinchilla* hanno mostrato che l'aumento del numero di parametri, se non accompagnato da una quantità adeguata di dati, non è sufficiente. In questa direzione, *LLaMA* ha dimostrato che modelli relativamente più compatti, se addestrati in modo accurato ed efficiente, possono raggiungere prestazioni altamente competitive. Un'ulteriore linea evolutiva riguarda la multimodalità. In tale contesto, *GPT-4* e, soprattutto, *GPT-4o* rappresentano un passo verso sistemi sempre più generali, capaci di elaborare non solo testo, ma anche immagini, audio e video. Allo

stesso modo, *Gemini 1.5*, grazie a finestre di contesto dell'ordine di milioni di token, migliora la capacità di ragionare su documenti di grandi dimensioni. Infine, *DeepSeek-v2* si inserisce nel filone dei modelli orientati all'efficienza computazionale, con l'obiettivo di ridurre i costi di addestramento e inferenza pur mantenendo prestazioni elevate (Naveed et al. 2023).

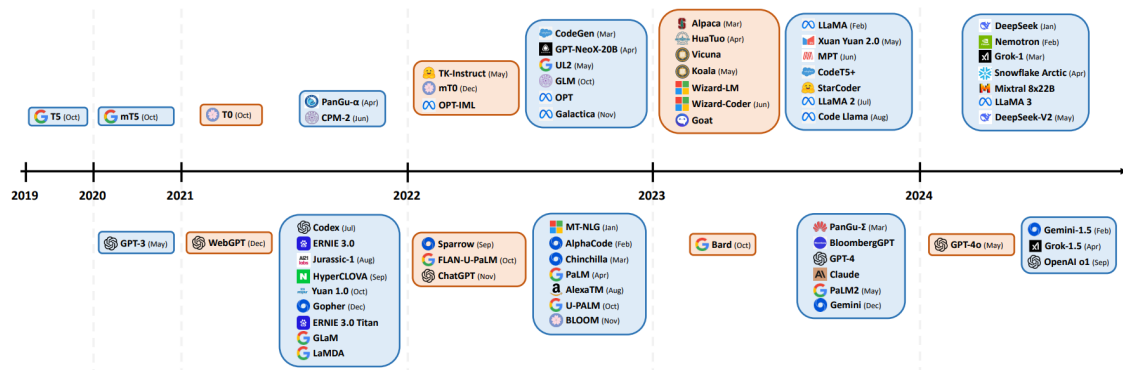


Figura 2.3: Rappresentazione cronologica dei principali rilasci di Large Language Models. I riquadri blu indicano modelli esclusivamente pre-addestrati, mentre quelli arancioni identificano modelli sottoposti a *instruction tuning*. I modelli collocati nella parte superiore del diagramma sono distribuiti come open-source, mentre quelli nella parte inferiore sono closed-source (Naveed et al. 2023).

Dal punto di vista applicativo, gli LLM costituiscono oggi il nucleo di sistemi conversazionali avanzati, strumenti di supporto alla programmazione, analisi documentale, generazione automatica di contenuti e assistenza decisionale (Naveed et al. 2023). La loro natura *general-purpose* consente di impiegarli come componenti centrali in pipeline complesse, integrando capacità di comprensione, generazione e ragionamento testuale.

Nonostante le elevate prestazioni, tali modelli presentano tuttavia alcune limitazioni strutturali. In primo luogo, non possiedono uno stato persistente, ovvero non sono in grado di ricordare informazioni fornite in interazioni precedenti al di fuori del contesto corrente. Inoltre, essendo modelli probabilistici, la generazione è intrinsecamente stocastica, a parità di prompt, possono produrre risposte differenti. Un ulteriore limite riguarda l'obsolescenza delle informazioni, gli LLM non hanno accesso diretto a dati esterni o aggiornamenti in tempo reale e possono utilizzare esclusivamente le conoscenze apprese durante il pre-addestramento. Dal punto di vista computazionale, inoltre, questi modelli sono generalmente di grandi dimensioni e richiedono risorse hardware significative. Infine, un problema ampiamente discusso in letteratura è il fenomeno delle cosiddette *hallucinations*, gli LLM non dispongono di una nozione intrinseca di verità e possono ge-

nerare risposte linguisticamente plausibili ma fattualmente errate o fuorvianti, in quanto addestrati su corpora eterogenei contenenti informazioni di qualità variabile (Minaee et al. 2024). Tali limitazioni hanno motivato lo sviluppo di strategie di integrazione e *augmentation*, volte a migliorare affidabilità, aggiornamento e controllo dei modelli. Tra queste, un ruolo centrale è ricoperto dalle architetture di *Retrieval-Augmented Generation* (RAG), che verranno discusse in dettaglio nelle sezioni successive.

2.2 Strategie di adattamento dei modelli pre-addestrati

Il pre-addestramento su larga scala consente ai Large Language Models di apprendere rappresentazioni linguistiche generali a partire da grandi quantità di testo non etichettato. Tuttavia, affinché tali modelli risultino efficaci in contesti applicativi specifici, è necessario adottare strategie di adattamento che permettano di specializzarne il comportamento rispetto a compiti, domini o vincoli particolari.

Nel corso degli anni sono state proposte diverse tecniche di adattamento, che spaziano dall'aggiornamento completo dei parametri tramite *Fine-Tuning* supervisionato, fino a metodi più leggeri o privi di aggiornamento dei pesi, come il *Prompt Engineering* e l'*In-Context Learning*.

2.2.1 Fine-Tuning

Il *fine-tuning* rappresenta la strategia più diretta e consolidata per adattare un modello linguistico pre-addestrato a un compito specifico. L'idea alla base consiste nel riutilizzare le rappresentazioni apprese durante il pre-addestramento su grandi corpora non etichettati e aggiornare i parametri tramite addestramento supervisionato su un dataset annotato relativo al task target.

Uno dei primi lavori a dimostrarne l'efficacia è stato proposto da Radford et al. (2018), che introducono un framework in due fasi. Una prima fase di pre-addestramento generativo non supervisionato di un Transformer decoder su grandi quantità di testo. Una seconda di fine-tuning discriminativo end-to-end su compiti specifici. I risultati evidenziano che un singolo modello pre-addestrato, opportunamente adattato, può raggiungere prestazioni allo stato dell'arte su diversi benchmark di inferenza linguistica naturale.

Un ulteriore passo avanti è rappresentato dal lavoro di Devlin et al. (2019), che introducono BERT (Bidirectional Encoder Representations from Transformers), un modello basato su un pre-addestramento bidirezionale tramite *Masked Language Modeling* (MLM) e *Next Sentence Prediction* (NSP). A differenza dei modelli autoregressivi unidirezionali, BERT integra simultaneamente il contesto sinistro e destro, apprendendo rappresentazioni

profonde e contestualizzate. L'adattamento al compito risulta architetturealmente semplice, è sufficiente aggiungere uno strato di output specifico ed effettuare l'aggiornamento end-to-end di tutti i parametri. Tale schema consente di impiegare un unico modello pre-addestrato per molteplici task, con benefici evidenti anche in scenari caratterizzati da quantità limitate di dati supervisionati.

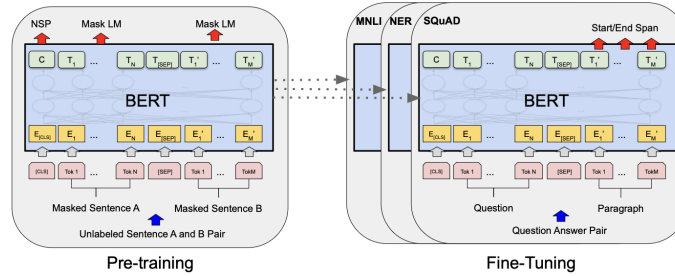


Figura 2.4: Fasi di pre-addestramento e fine-tuning per BERT (Devlin et al. 2019).

Con l'aumento della scala dei modelli, il ruolo del fine-tuning è stato in parte ridefinito. T. Brown et al. (2020) dimostrano che modelli autoregressivi di grandi dimensioni, come *GPT-3*, possono ottenere prestazioni competitive anche senza aggiornamento dei pesi, sfruttando esempi forniti nel contesto. Ciò nonostante, il fine-tuning supervisionato rimane cruciale per l'adattamento a domini specifici, per l'integrazione di dati proprietari e per la riduzione della complessità del *Prompt Engineering* (Minaee et al. 2024).

Tuttavia, il fine-tuning supervisionato diventa sempre più oneroso in termini di memoria e risorse computazionali. Questa criticità ha reso necessarie strategie di adattamento più leggere, capaci di ottenere risultati comparabili con un aggiornamento di parametri ridotto.

2.2.2 Parameter-Efficient Fine-Tuning

Il fine-tuning completo non è sempre la scelta ottimale quando si lavora con modelli da decine o centinaia di miliardi di parametri, poiché comporta costi computazionali elevati e tempi di addestramento molto lunghi. In questi casi, entrano in gioco le tecniche di *Parameter-Efficient Fine-Tuning* (PEFT), progettate per ottenere risultati simili a un fine-tuning tradizionale, ma con un impiego di risorse significativamente ridotto.

Un approccio centrale all'interno del paradigma PEFT è l'uso di moduli adapter, piccoli blocchi inseriti all'interno della rete pre-addestrata. Questi moduli aggiungono pochi parametri specifici per ciascun task, lasciando congelati i pesi originali del modello. In questo modo, è possibile adattare lo stesso modello a molteplici task downstream senza dover replicare l'intero insieme di parametri, rendendo l'adattamento più leggero, mo-

dulare ed estensibile, pur mantenendo prestazioni competitive su compiti NLP standard (Houlsby et al. 2019).

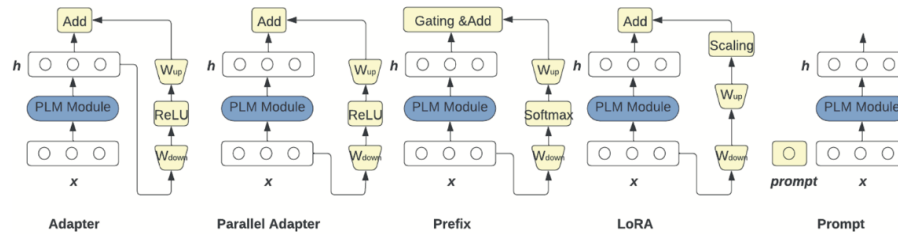


Figura 2.5: Diversi approcci di Parameter-Efficient Fine-Tuning (Naveed et al. 2023).

2.2.2.1 Low-Rank Adaptation (LoRA)

Tra le tecniche di Parameter-Efficient Fine-Tuning, una delle più rilevanti è *Low-Rank Adaptation* (LoRA), proposta da Hu et al. (2022) nel 2021. A differenza degli adapter tradizionali, che introducono nuovi moduli all'interno dell'architettura, LoRA interviene direttamente sulle matrici di peso esistenti modellandone gli aggiornamenti attraverso una struttura a rango ridotto. L'idea centrale è che le variazioni necessarie per l'adattamento a un task specifico possano essere descritte in uno spazio a dimensionalità effettiva molto inferiore rispetto a quello dei parametri originali. Ciò consente di ridurre drasticamente il numero di parametri addestrabili, mantenendo al contempo prestazioni comparabili al fine-tuning completo.

Nei modelli Transformer, LoRA viene applicata principalmente alle matrici di proiezione del meccanismo di self-attention. I risultati riportati nel lavoro originale (Hu et al. 2022) mostrano che questa strategia consente di ottenere prestazioni analoghe, e in alcuni casi superiori, rispetto al fine-tuning tradizionale, pur con un costo computazionale significativamente inferiore.

Un ulteriore vantaggio è l'assenza di latenza aggiuntiva in inferenza. Gli aggiornamenti appresi possono essere integrati nei pesi originali al termine dell'addestramento. Grazie a tali proprietà, LoRA è divenuta una delle tecniche PEFT più adottate per l'adattamento di Large Language Models di grandi dimensioni.

2.2.3 Adattamento senza aggiornamento dei pesi

Oltre alle tecniche di fine-tuning, negli ultimi anni si è diffuso un insieme di strategie che consentono di adattare i modelli pre-addestrati senza modificarne i pesi. In questo caso, il comportamento del modello viene orientato intervenendo esclusivamente sugli input forniti o sui parametri di generazione in fase di inferenza. Questi approcci sfruttano le capacità

emergenti degli LLM di condizionare la propria risposta sulla base del contesto, eliminando completamente i costi di addestramento e rendendo l'adattamento immediatamente applicabile.

2.2.3.1 Prompt Engineering

Il *Prompt Engineering* costituisce una delle modalità più diffuse di adattamento in fase di inferenza. Consiste nella progettazione e ottimizzazione dei prompt al fine di guidare il comportamento del modello verso output coerenti con uno specifico obiettivo (Marvin et al. 2023).

Un prompt è un input, tipicamente testuale ma potenzialmente anche multimodale, che può includere una direttiva, esplicita o implicita, volta a definire il compito da svolgere. Può inoltre contenere esempi dimostrativi che illustrano il comportamento atteso, vincoli relativi al formato dell'output e istruzioni di stile riguardanti tono o registro linguistico. In alcuni casi viene assegnato al modello un ruolo o una specifica prospettiva, così da orientarne la risposta. Infine, possono essere fornite informazioni di contesto aggiuntive, utili a delimitare l'ambito del problema e ridurre ambiguità interpretative. La qualità del prompt influisce in modo significativo sulle prestazioni del modello. Obiettivi chiari, contesto sufficiente e una formulazione concisa favoriscono risposte più accurate e controllabili (Schulhoff et al. 2024).

L'uso sempre più esteso del prompting ha stimolato lo sviluppo di tecniche avanzate, tra cui il *prompt chaining*, l'estensione a scenari multimodali e metodi di ottimizzazione automatica come l'*Automatic Prompt Engineer*, che generano e selezionano istruzioni in base a criteri di efficacia (Marvin et al. 2023). Nel complesso, il prompt engineering rappresenta una strategia fondamentale per sfruttare il potenziale dei modelli pre-addestrati in modo flessibile ed efficiente.

2.2.3.2 In-Context Learning

L'*In-Context Learning* (ICL) è un ulteriore paradigma di adattamento in cui un LLM esegue un compito senza aggiornare i propri pesi, ma sfruttando esempi dimostrativi inclusi direttamente nel prompt. Come mostrato da T. Brown et al. (2020), l'apprendimento in contesto è un paradigma che permette ai modelli di linguaggio di apprendere compiti dati solo pochi esempi sotto forma di dimostrazione. Operativamente, il prompt include un insieme ristretto di esempi dimostrativi in linguaggio naturale, concatenati alla query target. Il modello è quindi chiamato a inferire il pattern sottostante agli esempi e a generalizzarlo alla nuova richiesta, senza alcuna modifica dei parametri interni. Questo meccanismo consente di ottenere buone prestazioni su un'ampia gamma di compiti, inclusi proble-

mi di ragionamento complesso, riducendo i costi computazionali rispetto al fine-tuning supervisionato (Dong et al. 2024).

Le prestazioni dell'ICL risultano tuttavia sensibili alla progettazione del prompt, alla selezione e all'ordine degli esempi e alla lunghezza del contesto. Rimangono aperte questioni teoriche relative al suo meccanismo interno, interpretato alternativamente come forma di inferenza bayesiana implicita o apprendimento algoritmico emergente (Dong et al. 2024). Nonostante tali limiti, l'ICL rappresenta una strategia centrale per sfruttare le capacità emergenti degli LLM senza costi di riaddestramento.

2.2.3.3 Strategie di decodifica

Il processo di decodifica è cruciale per la generazione di testo da modelli linguistici, poiché determina come il modello seleziona i token successivi in base a probabilità calcolate. Le scelte fatte durante questo processo influenzano significativamente la qualità e la coerenza dell'output generato. Tra le strategie più comuni ci sono:

- **Greedy Search:** seleziona il token con la massima probabilità a ogni passo, ma può portare a risultati incoerenti e ripetitivi, poiché non considera l'impatto globale sulla sequenza (Minaee et al. 2024).
- **Beam Search:** tiene traccia dei migliori B candidati. Sebbene migliori la qualità dell'output, tende a generare sequenze simili, trascurando la diversità. Questo approccio è computazionalmente intensivo e può risultare inefficace per compiti complessi (Vijayakumar et al. 2018).
- **Top-k Sampling:** questa tecnica introduce casualità, selezionando un token tra i k più probabili. Tuttavia, come evidenziato da Holtzman et al. (2019), può produrre testi generati che risultano banali se il valore di k non è adeguato, limitando la diversità e la creatività.
- **Nucleus Sampling:** proposto da Holtzman et al. (2019), supera le limitazioni del Top-k troncando dinamicamente la coda inaffidabile della distribuzione di probabilità. Selezionando i token in base a una soglia p , questo metodo migliora la qualità e la diversità del testo generato, risultando la strategia più efficace per la generazione di testi lunghi e coerenti (Holtzman et al. 2019).
- **Diverse Beam Search:** variante della Beam Search ottimizza la diversità nell'output, permettendo di esplorare soluzioni diverse e migliorando le prestazioni senza un significativo aumento del carico computazionale (Vijayakumar et al. 2018).

Nonostante i progressi nelle strategie di decodifica e nelle tecniche di adattamento, tali approcci non modificano la natura intrinsecamente parametrica degli LLM. La conoscenza acquisita durante il pre-addestramento rimane infatti incorporata nei pesi del modello e non può essere aggiornata dinamicamente senza un ulteriore processo di addestramento. Inoltre, la generazione continua a essere un processo probabilistico, privo di un meccanismo esplicito di verifica rispetto a fonti esterne.

Questi limiti risultano particolarmente evidenti nei compiti ad alta intensità di conoscenza, in cui sono richieste informazioni aggiornate, specifiche di dominio o verificabili. Tali criticità hanno motivato lo sviluppo di architetture in grado di integrare meccanismi esterni di accesso all'informazione all'interno del processo generativo.

2.3 Retrieval-Augmented Generation

Il superamento dei limiti associati a una memoria puramente parametrica ha condotto alla definizione di architetture ibride. Tra queste, la *Retrieval-Augmented Generation* (RAG), formalmente introdotta da P. Lewis et al. (2020), rappresenta uno dei contributi più significativi. Tale paradigma combina un modello generativo pre-addestrato con una componente di recupero dell'informazione, introducendo una forma di memoria non-parametrica integrata nel processo inferenziale.

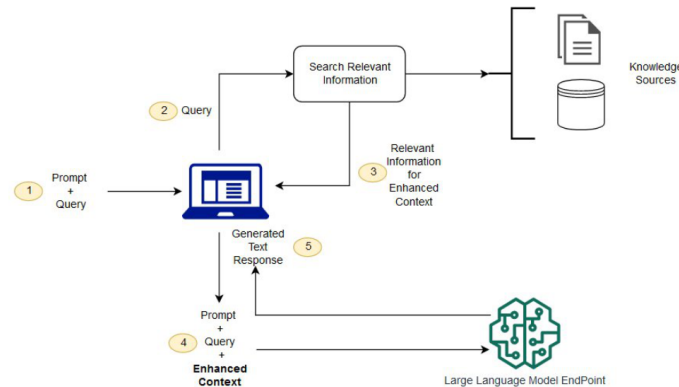


Figura 2.6: Esempio di architettura RAG per il question answering open-domain (Minaee et al. 2024).

Se da un lato la conoscenza parametrica è incorporata nei pesi del modello e riflette l'informazione appresa durante il pre-addestramento, dall'altro la conoscenza non-parametrica risiede in risorse esterne interrogabili dinamicamente. La prima garantisce capacità di generalizzazione, ma non può essere aggiornata senza un nuovo processo di addestramento. La seconda, sebbene non internalizzata nel modello, può essere modificata o ampliata in modo flessibile (Y. Gao et al. 2023). In conclusione, la memoria non

parametrica può essere estesa a piacimento, archiviando grandi volumi di documenti o dati proprietari e accedendo a qualsiasi tipo di fonte. In questo modo, la RAG permette di integrare negli LLM una conoscenza potenzialmente illimitata (Kimothei 2025).

2.3.1 Pipeline di Indicizzazione

Un sistema RAG funziona al meglio se le informazioni provenienti da diverse fonti vengono raccolte in un'unica posizione, normalizzate in un formato uniforme e suddivise in frammenti di dimensioni gestibili. La creazione di una base di conoscenza consolidata è necessaria per gestire la natura eterogenea delle fonti esterne e garantire che il modello acceda a dati rilevanti e aggiornati (Kimothei 2025). La pipeline di indicizzazione comprende tipicamente i seguenti passaggi:

1. **Caricamento dei dati (data loading):** connessione alle fonti esterne identificate, estrazione dei documenti e parsing del testo. I dati possono essere in formati eterogenei come PDF, Word, CSV o HTML e archiviati in data lake, data warehouse, file store, object store o persino fonti online di terze parti. Sebbene il caricamento sembri semplice, vanno considerati metadati, trasformazioni e eventuali operazioni di oscuramento.

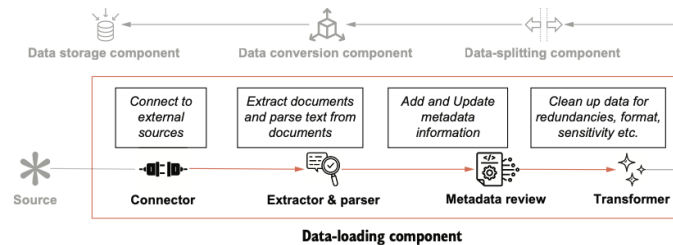


Figura 2.7: Fasi del caricamento dei dati nella pipeline di indicizzazione di un sistema RAG (Kimothei 2025).

2. **Suddivisione dei dati (data splitting o chunking):** i testi lunghi vengono segmentati in unità più piccole e gestibili, operazione fondamentale per superare i limiti intrinseci degli LLM, come la finestra di contesto massima e il problema noto come *lost-in-the-middle*, dove informazioni rilevanti situate nel mezzo del documento rischiano di essere trascurate. La suddivisione dei dati facilita anche la ricerca tramite retriever, che operano in modo più efficiente su frammenti compatti rispetto a documenti interi. Il chunking viene generalmente eseguito in tre fasi: segmentazione del testo in unità significative, combinazione di queste unità fino a raggiungere la dimensione desiderata, e inserimento di sovrapposizioni tra chunk consecutivi

per garantire continuità contestuale. Esistono diverse metodologie, che variano sia nella modalità di segmentazione sia nella definizione della dimensione dei chunk, adattandosi così alle specifiche esigenze della base di conoscenza.

3. **Conversione dei dati (embedding):** i chunk vengono trasformati in embedding, rendendoli compatibili con il retrieval semantico.
4. **Archiviazione (vector store):** gli embedding vengono memorizzati in database vettoriali per consentire ricerche semantiche rapide.

La fase di archiviazione rappresenta quindi il punto di collegamento tra la pipeline di indicizzazione e il processo di retrieval vero e proprio. Gli embedding generati devono essere memorizzati in strutture dati in grado di supportare ricerche per similarità efficienti anche su larga scala. A questo scopo vengono utilizzati i *vector store*.

2.3.1.1 Vector Store

I vector store costituiscono l'infrastruttura deputata alla memorizzazione e organizzazione degli embedding. Essi permettono di strutturare tali rappresentazioni in modo da supportare operazioni efficienti di calcolo della similarità. Dal punto di vista architetturale, si distinguono due livelli principali. Il primo livello è rappresentato dagli indici vettoriali, librerie specializzate nell'organizzazione e gestione efficiente di vettori tramite strutture dati e algoritmi dedicati. Un esempio ampiamente utilizzato è Faiss (Douze et al. 2025), sviluppata da Meta AI, che supporta indicizzazione sia su CPU sia GPU, a seconda dell'algoritmo. Il secondo livello è costituito dai database vettoriali, come ChromaDB (Kimothi 2025), che integrano uno o più indici vettoriali con funzionalità aggiuntive come persistenza, gestione dei metadati e interfacce di interrogazione. Gli indici vettoriali utilizzano strategie diverse per ottimizzare la ricerca dei vicini più prossimi. Alcuni si basano su strutture di grafi, come *Hierarchical Navigable Small World* (HNSW) (Malkov e Yashunin 2018), altri su partizionamento o clustering come l'*Inverted File* (IVF) (Douze et al. 2025). Tecniche complementari, come la *Product Quantization*, comprimono i vettori riducendo memoria e accelerando il calcolo delle distanze (Jegou, Douze e Schmid 2010).

Una volta archiviati e organizzati nel vector store, gli embedding possono essere interrogati dinamicamente nella fase successiva della pipeline RAG, dedicata al retrieval dei contenuti rilevanti e alla generazione della risposta.

2.3.2 Pipeline di Generazione

La pipeline di generazione nei sistemi RAG è responsabile della produzione della risposta contestuale a partire dalla query dell'utente. Basandosi sulle informazioni raccolte nella

pipeline di indicizzazione, essa integra tre componenti principali: *retrieval*, *augmentation* e *generation*. Queste fasi lavorano insieme per consentire interazioni in tempo reale, senza che l'utente debba conoscere la struttura interna della knowledge base (Kimothi 2025).

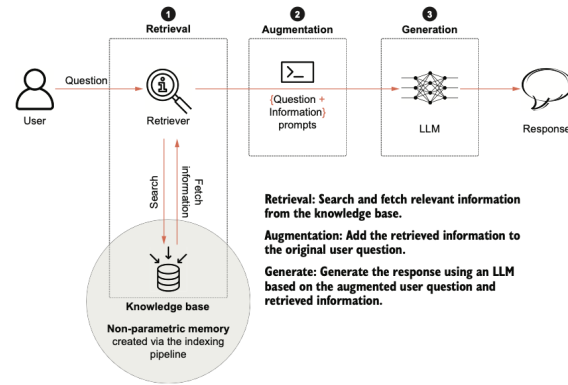


Figura 2.8: Processo di generazione in un sistema RAG (Kimothi 2025).

Retrieval

Il retrieval consiste nel processo di individuazione e selezione di informazioni rilevanti da un corpus o da una knowledge base. Formalmente rappresenta un problema di ricerca e ordinamento. Dato un input dell'utente, il retriever restituisce i primi k documenti classificati in base alla loro rilevanza rispetto alla query. La qualità di tale selezione incide direttamente sull'efficacia del sistema RAG, poiché il modello generativo può produrre una risposta corretta solo se il contesto recuperato è pertinente (Kimothi 2025). Tradizionalmente, il retrieval si basava su metodi sparsi come BM25, che assegnano punteggi ai documenti in base alla frequenza e distribuzione dei termini della query. Sebbene semplici e scalabili, questi approcci non catturano relazioni semantiche profonde, come sinonimie o parafrasi, limitandone l'efficacia nei sistemi RAG (Abdallah et al. 2025). Per superare tali limiti sono stati introdotti i *dense retrievers*, come il *Dense Passage Retriever* (DPR), che rappresentano query e documenti nello stesso spazio vettoriale continuo tramite encoder basati su BERT e stimano la rilevanza mediante prodotto scalare (Karpukhin et al. 2020). Poiché i modelli densi possono risultare meno efficaci in presenza di termini rari o altamente specifici, molti sistemi adottano strategie di *hybrid retrieval*, combinando punteggi lessicali (BM25) e semantici (DPR) (Abdallah et al. 2025). La fusione dei risultati può avvenire tramite tecniche come la *Reciprocal Rank Fusion* (RRF). Inoltre, il retrieval è spesso organizzato in due stadi, un primo recupero rapido tramite modelli *bi-encoder* come DPR, seguito da una fase di *re-ranking* con un *cross-encoder*, che analizza con maggiore profondità l'interazione tra query e documento, migliorando la precisione com-

plexiva. I documenti recuperati in questa fase costituiscono il contesto che sarà integrato nella successiva fase di augmentation.

Augmentation

Le informazioni recuperate dal retriever vengono integrate nella query dell'utente e inviate all'LLM sotto forma di prompt in linguaggio naturale, in un processo noto come augmentation. In pratica, l'augmentation trasforma i risultati della knowledge base in input comprensibili e utilizzabili dal modello. Questo passaggio rientra principalmente nell'ambito del prompt engineering (cfr. Sezione 2.2.3.1), che concerne la progettazione e l'ottimizzazione dei prompt al fine di guidare il comportamento dell'LLM e ottenere risposte accurate e coerenti con il contesto fornito. Tra gli approcci più comuni troviamo il *contextual prompting*, che integra le informazioni più rilevanti dal retriever direttamente nel prompt; il *controlled generation prompting*, che specifica vincoli sul formato o sullo stile dell'output; il *few-shot prompting*, in cui si presentano esempi simili per guidare la generazione del modello; e il *chain-of-thought prompting*, che incoraggia un ragionamento passo-passo, utile per problemi complessi.

È importante sottolineare che, sebbene il prompt sia scritto in linguaggio naturale, i principi logici alla base della costruzione dei prompt ricordano quelli della programmazione. La chiarezza delle istruzioni e la struttura logica determinano l'efficacia dell'interazione con l'LLM (Kimothei 2025). In conclusione, l'augmentation offre ampio margine di creatività nella progettazione dei prompt, e il tipo di approccio scelto dipenderà sia dal caso d'uso sia dalla natura delle informazioni presenti nella knowledge base.

Generation

La generazione rappresenta l'ultimo passaggio della pipeline RAG e consiste nella produzione dell'output finale a partire dalle informazioni recuperate, che vengono trasformate dall'LLM in testo naturale, coerente e pertinente (Y. Gao et al. 2023). Prima della generazione, i documenti selezionati vengono elaborati per ridurre il rumore, condensare i contenuti e rispettare i limiti della finestra di contesto. La loro eventuale riordinazione consente di dare priorità alle informazioni più rilevanti, ottimizzando l'input fornito al modello. Il generatore deve quindi interpretare congiuntamente la query e i documenti recuperati, che possono includere informazioni sia strutturate sia non strutturate. In questo contesto, il modello può essere ulteriormente ottimizzato tramite *fine-tuning*, ad esempio attraverso architetture Joint-Encoder o Dual-Encoder, così da migliorare l'integrazione tra query e contenuto recuperato. Tecniche come il *contrastive learning* contribuiscono a rafforzare l'allineamento tra le entità rilevanti e a migliorare accuratezza e capacità di generalizzazione (Y. Gao et al. 2023).

La scelta dell'LLM incide in modo significativo sulle prestazioni complessive del sistema. I *foundation models* offrono ampia copertura tematica e rapido deployment, mentre modelli *fine-tuned* risultano più efficaci in domini specialistici. Ulteriori considerazioni riguardano la distinzione tra modelli open source e proprietari, la disponibilità di risorse computazionali e la dimensione del modello. Soluzioni più grandi gestiscono meglio compiti complessi e contesti estesi, mentre modelli più piccoli garantiscono maggiore efficienza e rapidità di inferenza. In generale, la selezione dell'LLM richiede un processo iterativo volto a bilanciare capacità del modello, qualità del retrieval e obiettivi applicativi (Kimothi 2025). L'efficacia della generazione dipende non solo dalle capacità dell'LLM, ma anche dalla modalità di integrazione con il sistema di retrieval, da cui derivano le diverse configurazioni architetturali del RAG.

2.3.3 Paradigmi architetturali del RAG

L'evoluzione del paradigma RAG ha progressivamente ridefinito le modalità con cui retrieval e generazione vengono orchestrati all'interno del processo inferenziale. Nel tempo si sono consolidate tre principali configurazioni architetturali, differenziate per grado di complessità, livello di integrazione tra i moduli e strategie di controllo del flusso informativo. Una formulazione lineare di base (*Naive RAG*), un'estensione ottimizzata della pipeline (*Advanced RAG*) e un'impostazione modulare e componibile (*Modular RAG*) (Y. Gao et al. 2023).

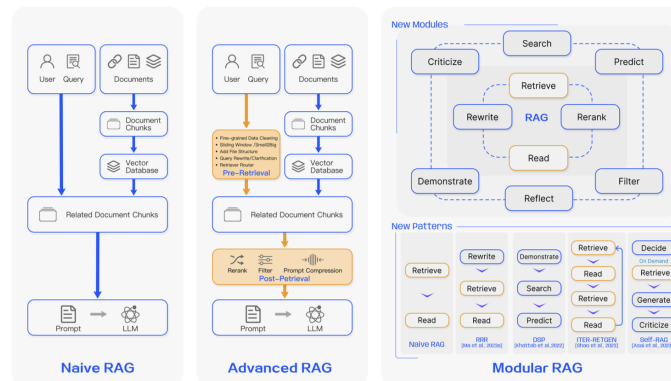


Figura 2.9: Confronto tra le principali configurazioni architetturali: Naive RAG, Advanced RAG e Modular RAG (Y. Gao et al. 2023).

Naive RAG

Il paradigma Naive RAG rappresenta la formulazione più elementare del framework RAG, diffusasi in seguito all'adozione su larga scala dei modelli conversazionali. Esso segue

una pipeline lineare di tipo *Retrieve–Read* (Y. Gao et al. 2023). In questa configurazione, retrieval e generazione sono organizzati in modo sequenziale, la query viene utilizzata per recuperare i documenti più rilevanti, che vengono poi concatenati al prompt e forniti al modello generativo.

L’architettura è semplice ed efficace, ma presenta limitazioni legate alla qualità del retrieval e all’assenza di meccanismi di ottimizzazione tra le diverse fasi. Errori nella selezione dei documenti si propagano direttamente alla generazione, compromettendo l’affidabilità dell’output. Più nel dettaglio, le criticità possono emergere già nella fase di retrieval, dove si possono verificare problemi di bassa precisione o recall, nonché la presenza di dati obsoleti o ridondanti. Sul fronte della generazione, il modello può produrre allucinazioni, risposte incoerenti o influenzate da bias. Inoltre, un’integrazione non ottimale dei passaggi recuperati può dar luogo a testi frammentati, ripetitivi o privi di adeguata sintesi critica.

Advanced RAG

L’Advanced RAG introduce miglioramenti strutturali volti a superare le limitazioni del paradigma Naive, intervenendo sia prima sia dopo la fase di retrieval (Y. Gao et al. 2023). L’obiettivo è ottimizzare precisione, rilevanza e coerenza dell’intero sistema.

Nella fase di *pre-retrieval*, l’attenzione si concentra sull’ottimizzazione dell’indicizzazione. Tra le strategie adottate vi sono tecniche di segmentazione più sofisticate, utilizzo di metadati per filtrare i documenti, integrazione di strutture a grafo per modellare relazioni tra entità e migliorare le query multi-hop, nonché approcci di ricerca ibrida che combinano metodi keyword-based e semantici. Particolare rilievo assume il fine-tuning dei modelli di embedding, che consente un migliore allineamento tra query e documenti in domini specialistici. Nella fase di *post-retrieval*, vengono applicate tecniche di riordinamento (re-ranking) e compressione del prompt per ridurre il rumore informativo e rispettare i limiti di contesto del modello. Alcuni approcci impiegano modelli ausiliari per stimare l’importanza dei segmenti recuperati e selezionare solo quelli realmente funzionali alla generazione. Ulteriori strategie includono il retrieval ricorsivo, la decomposizione della query in sotto-task, l’utilizzo di prompt che favoriscono ragionamenti astratti e l’approccio HyDE, che genera una risposta ipotetica per migliorarne il successivo recupero documentale. Complessivamente, l’Advanced RAG si configura come un’estensione sistematica del modello base, caratterizzata da un controllo più fine delle diverse fasi della pipeline.

Modular RAG

Il Modular RAG supera la rigidità della pipeline lineare retrieve–generate proponendo un’architettura componibile e dinamica (Y. Gao et al. 2023). In questo approccio, i mo-

duli funzionali possono essere aggiunti, sostituiti o riorganizzati in base alle esigenze del compito. Tra i principali moduli vi è il *Search Module*, che esegue ricerche dirette su fonti esterne come motori di ricerca, database strutturati o knowledge graph, e il *Memory Module*, dedicato al recupero da memorie interne o costruite iterativamente. Altri moduli rilevanti includono l'*Extra Generation Module*, che crea contesto supplementare per ridurre rumore e ridondanza, il *Task Adaptable Module* per adattare il sistema a compiti specifici, l'*Alignment Module* che migliora la coerenza tra query e documenti e il *Validation Module*, incaricato di verificare la qualità e l'affidabilità delle informazioni recuperate prima della generazione finale.

Il flusso operativo può assumere forma iterativa o riflessiva. Il modello può generare contenuti per migliorare il retrieval, oppure recuperare informazioni per affinare la generazione, in cicli successivi. Ne deriva un'architettura flessibile, capace di combinare pipeline serializzate e approcci end-to-end, con un grado di adattabilità superiore rispetto alle configurazioni precedenti. Il Modular RAG rappresenta l'evoluzione più avanzata del paradigma, trasformando il framework da semplice sequenza di fasi a sistema modulare orientato al compito.

2.3.4 RAG Multimodale

Nel tempo, le applicazioni basate su RAG hanno ampliato il paradigma oltre il solo testo, includendo modalità eterogenee quali immagini, audio, video e dati strutturati. In questo contesto emerge il *RAG multimodale*, una variante del framework standard progettata per gestire e integrare più formati di dati all'interno della stessa pipeline inferenziale (Kimothi 2025). I sistemi RAG tradizionali, essendo fondati su retrieval testuale, risultano limitati quando l'informazione rilevante è contenuta in rappresentazioni non testuali. Il RAG multimodale nasce dunque dall'esigenza di estendere le capacità di retrieval e generazione a basi di conoscenza eterogenee, sempre più comuni in ambito aziendale e scientifico. La costruzione di un sistema RAG multimodale richiede adattamenti nelle fasi di indicizzazione, embedding e retrieval. In particolare, la creazione di rappresentazioni vettoriali per dati multimodali può seguire tre strategie principali:

- Utilizzo di modelli di embedding condivisi, che mappano diverse modalità in uno spazio vettoriale comune abilitando il retrieval cross-modale.
- Impiego di embedding specifici per ciascuna modalità, con spazi vettoriali distinti e meccanismi di retrieval multipli.
- Conversione preliminare dei contenuti non testuali in testo, riconducendo la pipeline a un'impostazione RAG tradizionale.

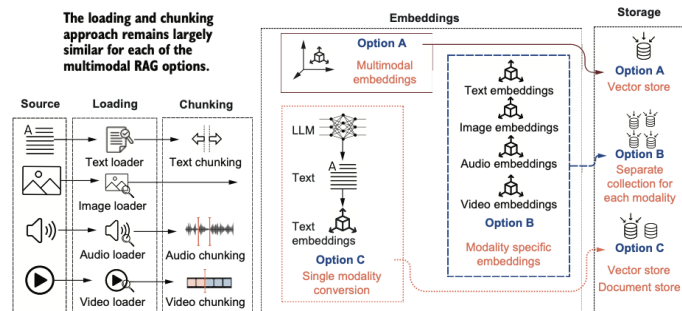


Figura 2.10: Tre approcci per la pipeline di indicizzazione nel RAG multimodale (Kimothi 2025).

Un esempio significativo di integrazione multimodale è rappresentato da *MuRAG*, proposto da W. Chen et al. (2022), che estende il paradigma RAG consentendo il recupero congiunto di testi e immagini per il question answering a dominio aperto. Il modello utilizza un encoder multimodale basato su T5 per il testo e Vision Transformer per le immagini, producendo rappresentazioni condivise memorizzate in una memoria esterna non parametrica. Durante l'inferenza, la query viene codificata nello stesso spazio vettoriale e utilizzata per recuperare contenuti rilevanti, successivamente forniti al generatore per la produzione della risposta (W. Chen et al. 2022). I risultati sperimentali dimostrano che l'integrazione di conoscenze visive e testuali può migliorare significativamente le prestazioni rispetto ai modelli basati esclusivamente su retrieval testuale.

Nonostante i vantaggi, il RAG multimodale introduce nuove sfide, tra cui l'allineamento semantico tra modalità diverse, l'aumento dei costi computazionali, la gestione di latenze maggiori e la complessità dell'orchestrazione tra moduli eterogenei (Kimothi 2025). La progettazione di pipeline robuste di preprocessing e la selezione mirata delle modalità effettivamente rilevanti per il caso d'uso risultano quindi elementi centrali per garantire scalabilità ed efficienza.

2.3.5 Framework di sviluppo per sistemi RAG

La realizzazione di sistemi RAG, in particolare quando si vogliono integrare più modalità di dati, richiede strumenti che facilitino la costruzione di pipeline complesse di retrieval, indicizzazione, embedding e generazione. Diversi framework open-source forniscono componenti modulari per semplificare questi compiti, permettendo agli sviluppatori di concentrarsi sulla progettazione del flusso logico e sull'integrazione dei modelli piuttosto che sulla gestione dei dettagli di basso livello. Tra questi, LangChain si distingue come uno degli strumenti più utilizzati per orchestrare sistemi RAG sia testuali sia multimodali.

2.3.5.1 Langchain

LangChain è un framework modulare progettato per facilitare lo sviluppo di applicazioni basate su LLM, con particolare supporto per pipeline RAG. La struttura modulare di LangChain permette di gestire diverse fasi del ciclo di vita delle applicazioni LLM, inclusi caricamento dei dati, embedding, retrieval e generazione contestualizzata, semplificando la creazione di sistemi scalabili, persistenti e consapevoli del contesto (LangChain Development Team 2024).

Il framework integra componenti fondamentali per la costruzione di pipeline RAG:

- **LLM Interface:** interfacce per collegare e interrogare modelli linguistici come GPT, Gemini o LLaMA.
- **Prompt Templates:** template strutturati per standardizzare le query e guidare il modello verso output coerenti.
- **Memory:** moduli per mantenere informazioni da interazioni precedenti, essenziali per conversazioni multi-turn.
- **Indexes e Retrievers:** consentono l'organizzazione dei dati in archivi strutturati e il recupero basato su similarità vettoriale.
- **Vector Store:** archiviazione e gestione degli embedding, cuore del retrieval per pipeline RAG.
- **Output Parsers, Agents e Callbacks:** strumenti per orchestrare workflow complessi, controllare l'esecuzione delle catene e monitorare eventi.

LangChain supporta inoltre pipeline multimodali, permettendo l'integrazione di dati testuali, immagini e audio, con l'uso di loaders specifici, embedding multimodali e retrieval cross-modale. La modularità del framework si estende a componenti avanzati come *LangGraph*, *LangServe* e *LangSmith*, che forniscono rispettivamente gestione dello stato e flussi multi-agente, deployment scalabile di API e monitoraggio/valutazione delle applicazioni (Mavroudis 2024; LangChain Development Team 2024).

Questo approccio rende LangChain uno strumento ideale per implementare pipeline RAG moderne, consentendo di costruire sistemi in grado di recuperare informazioni da database esterni o documenti multimodali e generare risposte linguisticamente coerenti e contestualizzate, riducendo i tempi di sviluppo e aumentando la robustezza delle applicazioni.

2.4 Metriche di valutazione

La valutazione dei sistemi basati su intelligenza artificiale costituisce un passaggio fondamentale per misurare l'efficacia, la robustezza e l'affidabilità dei modelli e delle architetture più complesse. In particolare, per applicazioni che coinvolgono LLM e pipeline RAG, la varietà dei task, la complessità dei dati e la natura generativa dei modelli richiedono un approccio articolato, in grado di integrare diverse dimensioni di misurazione.

La scelta delle metriche deve essere attenta e bilanciata, combinando indicatori quantitativi, riproducibili e misurabili con osservazioni qualitative, contestuali e spesso basate su giudizio umano (Minaee et al. 2024). In questo modo è possibile valutare non solo la correttezza delle risposte generate dai modelli, ma anche la coerenza, la rilevanza e la capacità del sistema di gestire contesti complessi o interazioni multi-turn.

Un approccio di valutazione strutturato diventa quindi uno strumento strategico per guidare lo sviluppo dei modelli e dei sistemi, identificare limiti e bias, ottimizzare le prestazioni complessive e garantire che le applicazioni siano affidabili e sicure anche in scenari dinamici e multi-modalità.

2.4.1 Metriche per LLM

La valutazione degli LLM richiede metriche in grado di analizzare diversi aspetti delle risposte generate, tra cui correttezza, coerenza linguistica, rilevanza semantica e capacità del modello di adattarsi a contesti differenti. Gli LLM producono spesso output generativi aperti, nei quali non esiste necessariamente una singola risposta corretta. Per questo motivo la valutazione richiede un insieme eterogeneo di metriche che permettano di analizzare il comportamento del modello da prospettive differenti (Minaee et al. 2024).

La scelta delle metriche dipende fortemente dal tipo di task per cui il modello viene utilizzato. Nei task di classificazione del testo, come sentiment analysis o classificazione tematica, le prestazioni possono essere valutate tramite metriche standard della classificazione supervisionata. Tra queste rientrano l'*accuracy*, che misura la percentuale di predizioni corrette, la *precision* e il *recall*, che analizzano rispettivamente la correttezza delle predizioni positive e la capacità del modello di identificare tutti i casi rilevanti, e l'*F1 score*, che combina precision e recall in una misura bilanciata (Y. Chang et al. 2024).

Per i task di generazione di testo, come riassunto automatico, traduzione, dialogo o question answering generativo, la valutazione risulta più complessa. In questi casi vengono spesso utilizzate metriche basate sulla sovrapposizione tra il testo generato e uno o più testi di riferimento. Metriche come *BLEU* e *ROUGE* stimano il grado di similarità tra le sequenze di parole presenti nei due testi, calcolando la sovrapposizione di n-grammi. Sebbene queste metriche siano largamente utilizzate nei benchmark, esse presentano al-

cune limitazioni, poiché non sempre riescono a catturare la similarità semantica tra frasi che esprimono lo stesso contenuto utilizzando parole diverse (Y. Chang et al. 2024). Per superare questi limiti sono state introdotte metriche basate su rappresentazioni distribuite del linguaggio. Un esempio è *BERTScore* (Zhang et al. 2019), che utilizza embeddings contestuali derivati da modelli pre-addestrati per confrontare semanticamente le parole presenti nel testo generato e in quello di riferimento. In questo modo è possibile valutare la similarità semantica anche quando le frasi utilizzano formulazioni lessicali differenti.

Nel caso di applicazioni legate alla generazione di codice, la valutazione si basa spesso su criteri funzionali. Le soluzioni prodotte dal modello vengono eseguite all'interno di suite di test automatici, che permettono di verificare se il codice soddisfa i requisiti richiesti dal problema. In questo contesto vengono utilizzate metriche come *Exact Match (EM)*, che verifica la corrispondenza esatta tra l'output e una soluzione di riferimento, e *Pass@k*, che misura la probabilità che almeno una delle k soluzioni generate dal modello superi i test di validazione (Y. Chang et al. 2024).

Accanto alle metriche automatiche, la valutazione degli LLM include spesso anche analisi qualitative effettuate da annotatori umani. Questo tipo di valutazione è particolarmente importante nei sistemi generativi, dove aspetti come la coerenza del discorso, la pertinenza rispetto alla richiesta dell'utente e la naturalezza linguistica del testo non possono essere completamente catturati da indicatori numerici (Minaee et al. 2024). Gli annotatori analizzano quindi le risposte prodotte dal modello secondo criteri definiti, che possono includere accuratezza informativa, rilevanza, chiarezza espositiva e qualità complessiva del linguaggio. L'integrazione tra metriche automatiche e valutazione umana permette di ottenere un quadro più completo delle capacità dei modelli linguistici. Le metriche quantitative consentono infatti confronti sistematici e replicabili tra diversi modelli o configurazioni, mentre l'analisi qualitativa permette di cogliere aspetti più complessi legati alla qualità del contenuto generato e all'esperienza complessiva dell'utente.

2.4.2 Metriche per RAG

I sistemi basati su RAG introducono ulteriori complessità rispetto ai modelli linguistici tradizionali. In una pipeline RAG, la risposta generata dipende non solo dalle capacità del modello linguistico, ma anche dalla qualità del processo di recupero delle informazioni (Y. Gao et al. 2023). Le metriche possono quindi essere suddivise in tre categorie principali: retrieval dei documenti, qualità della generazione e coerenza tra risposta e contesto recuperato.

Per il modulo di retrieval si utilizzano metriche tipiche dei sistemi di information retrieval, come *Precision@k*, *Recall@k* e *Mean Reciprocal Rank (MRR)*, che misurano rispettivamente la proporzione di documenti rilevanti tra i primi k risultati, la capacità di re-

cuperare tutti i documenti pertinenti e la rapidità nel trovare il primo documento rilevante (Y. Gao et al. 2023).

La valutazione della generazione testuale include metriche specifiche per verificare l'uso corretto delle informazioni recuperate. Particolarmente importanti sono la *faithfulness*, ossia la fedeltà della risposta rispetto ai documenti forniti per evitare fenomeni di allucinazione (Es et al. 2024), e la pertinenza della risposta rispetto alla query, valutata tramite metriche di *answer relevance*. Framework recenti, come *RAGAS*, integrano metriche per valutare simultaneamente retrieval, generazione e fedeltà al contesto (Es et al. 2024).

Accanto alle metriche automatiche, rimane fondamentale l'integrazione con valutazioni qualitative. L'analisi manuale da parte di annotatori permette di individuare problemi difficilmente rilevabili con metriche quantitative, come interpretazioni errate del contesto, risposte parziali o formulazioni linguistiche poco chiare. La combinazione di metriche automatiche e valutazione umana fornisce così una visione più completa delle prestazioni, supportando pipeline RAG affidabili ed efficaci (Y. Gao et al. 2023).

2.4.3 Framework di monitoraggio

Il monitoraggio delle pipeline di modelli linguistici e dei sistemi RAG è essenziale per garantire prestazioni affidabili, confrontabili e riproducibili. Un framework di monitoraggio consente di registrare in modo strutturato metriche, parametri di configurazione, versioni dei modelli e risultati sperimentali, facilitando l'analisi comparativa tra diverse iterazioni e il rilevamento di eventuali anomalie. Tra i framework più utilizzati in questo contesto vi è *MLflow*, che permette di tracciare esperimenti, loggare metriche e gestire i modelli in un workflow coerente (Zaharia et al. 2018).

2.4.3.1 MLflow

MLflow è una piattaforma open source per la gestione centralizzata di esperimenti, metriche, parametri e artefatti dei modelli (Zaharia et al. 2018). È particolarmente utile nelle pipeline di LLM e RAG, permettendo di monitorare sia il fine-tuning dei modelli sia la qualità delle risposte generate, mantenendo una tracciabilità completa di ogni esperimento.

MLflow offre tre componenti principali:

- **MLflow Tracking:** consente di registrare esperimenti sotto forma di *runs*, loggare parametri, metriche e artefatti, e consultare i risultati tramite API o interfaccia web. Permette di confrontare iterazioni, organizzare esperimenti in gruppi e monitorare le performance durante il ciclo di sviluppo.

- **MLflow Projects:** definisce un formato standard per pacchettizzare codice e dipendenze di un esperimento, garantendo riproducibilità e coerenza tra ambienti locali e cluster remoti.
- **MLflow Models:** fornisce un formato standard per salvare e distribuire modelli, includendo informazioni su versione, dipendenze e interfacce compatibili (*flavors*), facilitando l'integrazione in pipeline di inferenza batch e real-time.

L'interfaccia web e le API di MLflow permettono di analizzare i risultati, confrontare versioni differenti e riprodurre esperimenti passati, supportando lo sviluppo iterativo e il miglioramento continuo delle pipeline (MLflow Contributors 2026). L'utilizzo di MLflow consente così di avere una panoramica chiara e organizzata degli esperimenti, facilitando l'interpretazione dei risultati e la pianificazione dei successivi sviluppi.

2.5 Lavori correlati

Negli ultimi anni sono state sviluppate diverse piattaforme e framework che facilitano la realizzazione di applicazioni basate su Large Language Models, con particolare attenzione alla costruzione di assistenti conversazionali personalizzati. Questi strumenti mirano a semplificare l'integrazione tra modelli linguistici, basi di conoscenza e fonti di dati eterogenee, offrendo ambienti che permettono di gestire il recupero delle informazioni, la definizione dei prompt e la regolazione dei parametri di generazione senza richiedere implementazioni software complesse.

Tra le soluzioni più recenti in questo ambito vi sono piattaforme che permettono di progettare sistemi basati su RAG tramite ambienti visuali. Un esempio è FlowiseAI, una piattaforma open-source progettata per lo sviluppo di applicazioni basate su LLM attraverso un editor grafico che consente di definire workflow conversazionali mediante componenti modulari. La piattaforma permette di integrare modelli linguistici, sistemi di embedding e database vettoriali all'interno di architetture configurabili, supportando l'utilizzo di documenti e basi di conoscenza come contesto per la generazione delle risposte (FlowiseAI Contributors 2024). Attraverso un sistema visuale basato su nodi, gli utenti possono definire i diversi passaggi del processo, gestire i prompt e controllare i parametri di generazione dei modelli.

Un'altra piattaforma ampiamente utilizzata nello sviluppo di sistemi conversazionali è IBM Watson Assistant, parte dell'ecosistema di intelligenza artificiale sviluppato da IBM. Watson Assistant fornisce strumenti per progettare e distribuire chatbot tramite un ambiente grafico che consente di definire intenzioni, entità e flussi di dialogo, facilitando la realizzazione di assistenti virtuali per diversi contesti applicativi (IBM 2024). La

piattaforma integra tecniche di NLP e strumenti di gestione del dialogo, permettendo inoltre di collegare il sistema a basi di conoscenza e servizi esterni per fornire risposte contestualizzate.

Queste piattaforme evidenziano una crescente diffusione di strumenti che rendono più accessibile lo sviluppo di applicazioni basate su modelli linguistici avanzati. In particolare, l'introduzione di ambienti visuali e strumenti di configurazione consente di semplificare la costruzione di sistemi che combinano modelli generativi, meccanismi di recupero delle informazioni e diverse fonti di dati.

3. Architettura del sistema e piattaforma Synapsis ML

L'evoluzione delle tecnologie digitali ha favorito la diffusione di sistemi complessi in grado di integrare *Internet of Things* (IoT), analisi dei dati e intelligenza artificiale all'interno di un'unica infrastruttura. Per supportare le crescenti esigenze di digitalizzazione e connettività, Mative S.r.l. propone un ecosistema integrato per la gestione di progetti IoT aziendali. Realtà tecnologica il cui nome deriva dalla combinazione dei termini *automation* e *automotive*, a sottolineare la vocazione dell'azienda verso soluzioni innovative in ambito digitale e industriale. Mative pone particolare attenzione agli ambiti della telematica e delle applicazioni in contesti quali *Smart Industry*, *Smart Mobility*, *Smart Farming* e *Smart City*.

L'infrastruttura di Mative è organizzata in diversi livelli funzionali tra loro interconnessi: Mative Cloud, Synapsis IoT, Synapsis Analysis e Synapsis ML. Questi comprendono i sistemi di acquisizione dei dati, l'infrastruttura cloud, gli strumenti di analisi e i moduli dedicati all'intelligenza artificiale. L'integrazione di tali componenti consente di supportare un'ampia gamma di applicazioni, che spaziano dalla gestione di macchinari industriali e flotte di veicoli fino al monitoraggio di infrastrutture e sistemi distribuiti.

Il presente capitolo approfondisce l'architettura dell'intero ecosistema Mative, con particolare attenzione a Synapsis ML, elemento centrale dell'intero sistema e principale oggetto di studio di questo lavoro. Synapsis ML rappresenta una piattaforma dedicata all'intelligenza artificiale e al machine learning, progettata per facilitare lo sviluppo, la gestione e l'implementazione di modelli in modo integrato. In particolare, l'analisi si concentra sulle caratteristiche architettoniche, sull'implementazione e sulle modalità di utilizzo della piattaforma, evidenziando il ruolo che essa riveste all'interno dell'infrastruttura complessiva. Una parte significativa dell'attività svolta personalmente nel corso del progetto ha riguardato la progettazione e lo sviluppo della componente LLM e della pipeline RAG descritte nella Sezione 3.3. Il lavoro svolto ha incluso l'implementazione del sistema di routing e orchestrazione delle capability, della gestione del contesto condiviso della pipeline e dei meccanismi di integrazione delle differenti sorgenti informative. L'analisi si concentra sulle scelte progettuali adottate, sulle soluzioni tecniche implementate e sulle motivazioni alla base di tali decisioni, con l'obiettivo di evidenziare le modalità di integrazione tra i diversi componenti del sistema e le strategie utilizzate per garantire flessibilità, scalabilità ed efficienza.

3.1 Introduzione all'ecosistema

L'architettura proposta da Mative è progettata per gestire in modo integrato l'intero ciclo di vita dei dati generati da dispositivi connessi, dalla fase di acquisizione fino alla loro elaborazione avanzata. L'obiettivo è fornire un ecosistema distribuito e modulare, composto da diversi componenti interconnessi, ciascuno responsabile di specifiche funzionalità, che cooperano per garantire affidabilità, scalabilità e interoperabilità.

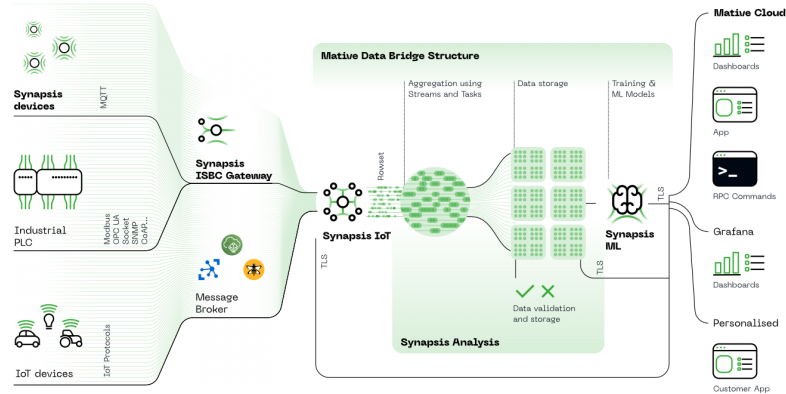


Figura 3.1: Architettura generale dell'ecosistema Mative. Il diagramma illustra l'integrazione tra i dispositivi IoT, l'infrastruttura cloud e le piattaforme Synapsis dedicate alla gestione dei dati, all'analisi e all'intelligenza artificiale.

La piattaforma Mative combina componenti hardware, infrastrutture software e moduli avanzati di analisi, configurandosi come una soluzione completa per applicazioni IoT e basate su intelligenza artificiale. I dati possono provenire da una varietà di sorgenti eterogenee: sensori industriali, dispositivi telematici, macchinari, veicoli o API di terze parti. Queste informazioni includono parametri operativi, indicatori ambientali, stato di funzionamento dei sistemi e telemetria di flotta, che insieme costituiscono una base informativa ricca e diversificata.

L'infrastruttura è organizzata in più livelli funzionali che rispecchiano il paradigma edge-to-cloud. Il livello periferico, rappresentato dai dispositivi edge, raccoglie e pre-elabora i dati, riducendo latenza e traffico di rete. Il livello di orchestrazione e integrazione, Synopsis IoT, garantisce normalizzazione, gestione bidirezionale e interoperabilità tra sistemi eterogenei. Infine, la piattaforma cloud centrale persiste, organizza e rende disponibili i dati per strumenti di analisi avanzata e moduli di intelligenza artificiale.

Questo approccio modulare consente di scalare l'infrastruttura in base alle esigenze, integrare nuove sorgenti dati e applicazioni, e supportare scenari complessi quali il monitoraggio remoto di impianti industriali, la gestione di flotte aziendali o il controllo di

dispositivi distribuiti su larga scala. Inoltre, la struttura distribuita assicura continuità operativa anche in caso di connettività intermittente, permettendo alle aziende di utilizzare i dati in tempo reale per decisioni rapide e basate su evidenze quantitative.

3.1.1 Synapsis IoT

Mative Synapsis IoT è un'architettura progettata per la comunicazione e l'integrazione di hardware edge, software e API OEM di terze parti. La piattaforma consente la trasformazione di diversi protocolli industriali, automotive e standard in payload compatibili con Mative Cloud, permettendo l'integrazione di microcontrollori, hardware telematico e soluzioni OEM. Synapsis IoT rappresenta il componente centrale dell'infrastruttura IoT Mative, responsabile della raccolta, normalizzazione, orchestrazione e messa a disposizione dei dati provenienti da sorgenti eterogenee.

La piattaforma opera come sistema di ingestione e integrazione dei dati sia a livello cloud sia in modalità intermedia, gestendo la connessione simultanea di migliaia o milioni di dispositivi tramite protocolli IoT standard come MQTT, HTTP/HTTPS e CoAP. Supporta inoltre tecnologie di connettività quali TCP, TLS e LPWAN, incluso LoRaWAN, mettendo a disposizione endpoint sicuri per la trasmissione dei messaggi di telemetria. In questo modo le unità periferiche possono inviare continuamente informazioni verso piattaforme backend scalabili. I dati raccolti vengono convertiti in un formato uniforme e normalizzato, rendendoli immediatamente utilizzabili da servizi di storage, strumenti di analisi e moduli intelligenti. La piattaforma è inoltre in grado di decodificare protocolli industriali complessi e di trasformare semanticamente i campi dati, facilitando l'integrazione tra dispositivi e sistemi eterogenei.

Synapsis IoT supporta la comunicazione bidirezionale tra cloud e dispositivi, consentendo non solo la raccolta di dati ma anche l'invio di comandi, configurazioni e aggiornamenti ai dispositivi connessi. Questa funzionalità abilita scenari avanzati di gestione delle flotte e di provisioning dei dispositivi. La piattaforma espone inoltre API di integrazione verso sistemi aziendali come ERP, CRM o altri software gestionali, favorendo lo scambio automatico di informazioni e l'estensione delle applicazioni industriali all'interno dei sistemi informativi aziendali.

Architettura e affidabilità del sistema

Dal punto di vista architetturale, Synapsis IoT si colloca tra il livello periferico edge e il cloud di elaborazione avanzata, fungendo da punto di interconnessione tra i dispositivi sul campo e i servizi cloud centrali. Adotta un'architettura progettata per garantire reattività, scalabilità e sicurezza nella gestione dei dati IoT. La piattaforma sfrutta un modello *event-*

driven, che permette di reagire in tempo reale agli eventi generati dai dispositivi e di orchestrare automaticamente le operazioni necessarie alla gestione dei flussi informativi distribuiti.



Figura 3.2: Architettura della piattaforma Synapsis IoT

L'architettura cloud è scalabile orizzontalmente ed è in grado di bilanciare il carico di lavoro, gestendo picchi di traffico provenienti da un numero elevato di dispositivi connessi. L'utilizzo di broker per la gestione dei messaggi consente di smistare e processare grandi volumi di dati in modo efficiente, garantendo al tempo stesso flessibilità e adattabilità alle diverse esigenze applicative.

Dal punto di vista della sicurezza, la piattaforma implementa meccanismi avanzati per la protezione delle comunicazioni e la gestione delle identità dei dispositivi. I dispositivi connessi adottano metodologie di autenticazione reciproca (*Mutual Authentication*), mentre i messaggi in transito sono protetti tramite crittografia end-to-end. La gestione delle identità e delle autorizzazioni consente di controllare l'accesso ai servizi e di garantire la confidenzialità e l'integrità delle informazioni. L'ecosistema supporta inoltre funzionalità avanzate di gestione dei dispositivi, tra cui provisioning sicuro, aggiornamenti OTA (*Over-The-Air*), monitoraggio dello stato di salute dei dispositivi e strumenti di diagnostica remota. Queste funzionalità risultano fondamentali per mantenere operativo e controllabile un parco di dispositivi distribuiti su larga scala, riducendo la necessità di interventi manuali e migliorando l'efficienza della gestione operativa.

Database

Synapsis IoT utilizza un'architettura di database ibrida per gestire i diversi tipi di dati generati dai dispositivi IoT. I dati di telemetria ad alta frequenza vengono memorizzati in database NoSQL scalabili, come Cassandra o MongoDB, progettati per gestire grandi volumi di dati e accessi rapidi. Configurazioni, metadati e informazioni strutturate sono

invece conservati in database relazionali, come PostgreSQL, che garantiscono integrità e consistenza transazionale. Questa combinazione consente di bilanciare le esigenze di scalabilità e flessibilità tipiche dei dati non strutturati con la necessità di mantenere integrità e coerenza per i dati più critici.

L'utilizzo di un'architettura ibrida permette quindi alla piattaforma di gestire in modo efficiente sia grandi quantità di dati telemetrici generati in tempo reale sia informazioni strutturate relative alla configurazione dei dispositivi e dei sistemi, assicurando un sistema affidabile per l'analisi dei dati e la gestione operativa dell'infrastruttura IoT.

3.1.2 Mative Cloud

Mative Cloud rappresenta il livello infrastrutturale centrale dell'ecosistema Mative ed è progettato per fornire un ambiente cloud-native scalabile, sicuro ed efficiente per la gestione di dispositivi, dati e applicazioni IoT. La piattaforma costituisce il punto di raccolta dei dati e delle informazioni elaborate dai sistemi periferici, consentendo alle organizzazioni di analizzare grandi volumi di informazioni e di supportare applicazioni avanzate di monitoraggio e controllo.

La piattaforma adotta un paradigma orientato allo streaming dei dati, che consente di gestire flussi informativi continui e di supportare applicazioni in tempo reale. In questo modo, Mative Cloud trasforma le informazioni raccolte in dati fruibili da strumenti di analisi avanzata e moduli di intelligenza artificiale, permettendo la generazione di insight operativi e decisionali.

Gestione della piattaforma e dashboard

Mative Cloud mette a disposizione un'interfaccia web centralizzata attraverso la quale gli utenti possono monitorare lo stato dei sistemi e configurare le funzionalità operative. L'interfaccia include dashboard dinamiche basate su widget, che permettono di visualizzare indicatori di stato, grafici temporali, mappe e report interattivi. I widget possono essere organizzati e personalizzati, consentendo agli utenti di adattare la visualizzazione delle informazioni alle proprie esigenze operative e di consultare dati storici o approfondimenti specifici sui dispositivi e sulle flotte monitorate.

Gestione delle flotte e monitoraggio degli asset

Tra le principali applicazioni della piattaforma vi è il monitoraggio in tempo reale di flotte di veicoli e asset distribuiti sul territorio. Dashboard dedicate consentono di seguire parametri come posizione, velocità, stato di funzionamento e percorso storico dei dispositivi, supportando l'analisi degli eventi e la rilevazione di eventuali anomalie operative.

La piattaforma integra mappe interattive e strumenti di geofencing per gestire l'ingresso e l'uscita dei dispositivi da specifiche aree operative, ottimizzando così la gestione logistica e la supervisione delle risorse distribuite.

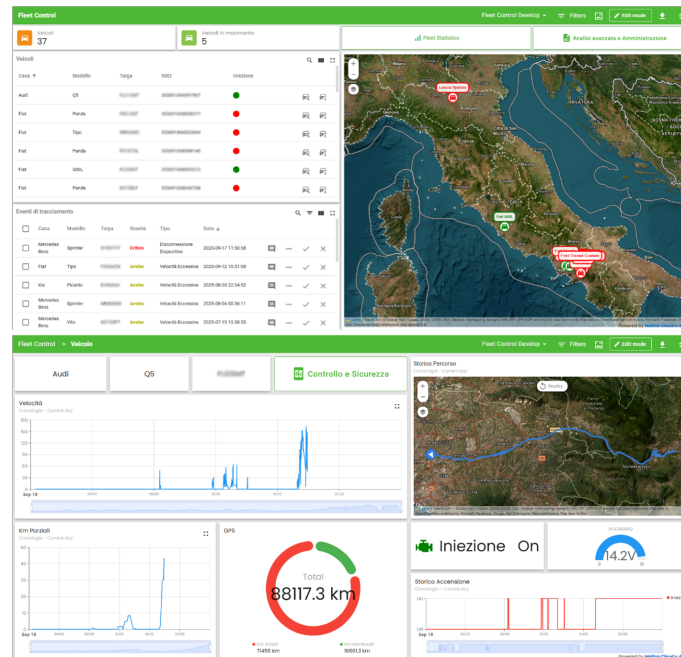


Figura 3.3: Interfaccia della piattaforma Mative Cloud per il monitoraggio e la gestione di dispositivi e flotte tramite dashboard interattive

Gestione degli accessi e sicurezza

Mative Cloud prevede un sistema di gestione delle identità e degli accessi centralizzato, basato su un'infrastruttura IAM denominata Mative Accounts. Il sistema permette di definire ruoli, politiche di accesso e Single Sign-On per utenti e team, garantendo sicurezza e separazione dei dati tra i diversi tenant della piattaforma.

Grazie a questa architettura multi-tenant, ogni organizzazione dispone di uno spazio dedicato per la gestione di dispositivi, configurazioni e dashboard operative, mantenendo sicurezza, privacy e controllo completo sulle proprie informazioni.

Ambiti applicativi

La piattaforma può essere utilizzata in diversi contesti che richiedono il monitoraggio e la gestione di dispositivi connessi. Tra i principali ambiti di utilizzo si annoverano:

- **Smart Industry:** supervisione di impianti e macchinari industriali;

- **Smart Mobility:** gestione di flotte di veicoli e sistemi di mobilità connessa;
- **Smart Farming:** monitoraggio di attività agricole tramite sensori distribuiti;
- **Smart City:** gestione di infrastrutture urbane e sistemi distribuiti di raccolta dati.

La flessibilità della piattaforma consente di integrare dispositivi e sorgenti dati eterogenee, sviluppando soluzioni IoT personalizzate per le esigenze operative di ciascuna organizzazione.

3.1.3 Synapsis Analysis

Synapsis Analysis rappresenta il modulo dell'ecosistema Mative dedicato all'elaborazione, all'analisi e alla valorizzazione dei dati raccolti dai dispositivi e dai sistemi connessi alla piattaforma. Il suo obiettivo principale è trasformare i dati grezzi provenienti dall'infrastruttura IoT in informazioni strutturate e interpretabili, utili per attività di monitoraggio, analisi e supporto alle decisioni operative.

All'interno dell'architettura complessiva della piattaforma, Synapsis Analysis si colloca a valle dei sistemi di raccolta e gestione dei dati, fornendo strumenti avanzati per l'esplorazione e l'analisi delle informazioni archiviate nel cloud. Attraverso questo modulo è possibile analizzare dataset complessi, individuare pattern nei dati e costruire indicatori utili per valutare le prestazioni dei sistemi monitorati.

Integrazione, modellazione e pipeline dei dati

Synapsis Analysis consente di integrare informazioni provenienti da sorgenti eterogenee, creando uno strato dati unificato che facilita le attività analitiche. Tra le fonti supportate rientrano database relazionali, database NoSQL, file strutturati come CSV o JSON e flussi di telemetria generati dai dispositivi IoT. Questa connettività multi-fonte permette di consolidare dati distribuiti tra diversi sistemi informativi senza richiedere modifiche sostanziali all'infrastruttura esistente.

Per rendere i dati facilmente utilizzabili, la piattaforma mette a disposizione strumenti di modellazione che permettono di strutturare e organizzare le informazioni secondo le esigenze applicative. Gli utenti possono definire dataset logici, combinare informazioni provenienti da diverse sorgenti e creare viste aggregate che semplificano l'analisi di strutture dati complesse. Questo processo di astrazione semantica migliora leggibilità e riutilizzabilità dei dati all'interno delle applicazioni analitiche.

Synapsis Analysis integra inoltre pipeline di elaborazione basate su processi ETL (*Extract, Transform, Load*) ed ELT (*Extract, Load, Transform*). Queste pipeline consentono di raccogliere, trasformare e preparare i dati, eseguendo operazioni di pulizia, filtraggio

e normalizzazione, oltre a calcolare indicatori derivati e metriche specifiche. Le pipeline possono operare sia in modalità programmata sia in tempo reale, adattandosi a scenari e esigenze diverse.

Analisi, visualizzazione e reporting

Il motore analitico di Synapsis Analysis permette di interrogare dataset complessi attraverso query avanzate, con sintassi simile a SQL, e di gestire grandi volumi di informazioni grazie a meccanismi di caching, indicizzazione e viste materializzate. Queste ottimizzazioni garantiscono prestazioni elevate anche su dataset di grandi dimensioni.

Gli strumenti di visualizzazione integrati consentono di rappresentare i dati tramite grafici, tabelle e dashboard interattive. È possibile costruire visualizzazioni personalizzate combinando elementi grafici per ottenere una panoramica completa dello stato dei sistemi monitorati. I grafici temporali permettono l'analisi delle serie storiche, mentre gli indicatori sintetici e le tabelle dinamiche supportano il monitoraggio dettagliato delle principali metriche di performance. Attraverso le funzionalità di business intelligence, Synapsis Analysis trasforma i dati elaborati in informazioni utili per il supporto alle decisioni. La piattaforma consente di generare report personalizzabili, esportabili in PDF, CSV o fogli di calcolo, e programmabili a intervalli temporali definiti. È inoltre possibile definire e monitorare indicatori chiave di performance (KPI), valutando l'andamento delle attività operative e identificando eventuali criticità.

Particolare attenzione è riservata ai dati temporali generati dai dispositivi IoT. Synapsis Analysis gestisce grandi volumi di telemetria ad alta frequenza, consentendo il monitoraggio in tempo reale e l'analisi storica dei dati. Grazie ad aggregazioni temporali e analisi su finestre configurabili, il sistema individua trend, anomalie e variazioni nel comportamento dei dispositivi, supportando la manutenzione predittiva e l'ottimizzazione dei processi operativi.

Nel complesso, i componenti descritti cooperano per garantire la gestione completa del ciclo di vita del dato, dalla fase di acquisizione fino alla sua trasformazione in informazione utile. Su tale infrastruttura si innesta il livello di intelligenza artificiale rappresentato da Synapsis ML, che consente di valorizzare ulteriormente i dati attraverso l'applicazione di tecniche avanzate di machine learning e modelli linguistici.

Nel seguito, verrà quindi analizzata nel dettaglio la piattaforma Synapsis ML, evidenziandone l'architettura, le modalità di integrazione e le scelte implementative adottate nel contesto del presente lavoro.

3.2 Synopsis ML

Il livello più avanzato dell'ecosistema è costituito da Synopsis ML, il modulo dedicato alla gestione e all'esecuzione di modelli di intelligenza artificiale. Esso consente di applicare tecniche di machine learning e modelli linguistici avanzati ai dati raccolti, con l'obiettivo di generare previsioni, identificare anomalie e automatizzare processi decisionali. A differenza dei sistemi tradizionali basati su regole statiche, Synopsis ML è in grado di apprendere dai dati storici, adattarsi a condizioni variabili e fornire insight anche in scenari complessi.

La piattaforma si distingue per modularità e flessibilità, permettendo di gestire diversi tipi di modelli in pipeline integrate per training, validazione, deploy e monitoraggio. L'integrazione con Synopsis Analysis e Synopsis IoT consente di utilizzare dati normalizzati e flussi in tempo reale, rendendo possibili analisi predittive e decisioni automatizzate in scenari distribuiti. Include strumenti per monitoraggio e gestione dei modelli in produzione, versioning dei modelli, gestione dei permessi e rollback automatici in caso di anomalie, garantendo sicurezza, affidabilità e continuità operativa. La piattaforma supporta anche pipeline automatizzate che riducono l'intervento manuale, migliorando efficienza e scalabilità nell'implementazione di soluzioni AI.

La piattaforma si basa su due pilastri tecnologici fondamentali:

- Modelli di Machine Learning, capaci di apprendere dai dati storici e migliorare progressivamente le proprie prestazioni grazie all'esperienza acquisita.
- Large Language Model, attivati tramite la tecnica RAG (Retrieval-Augmented Generation), per elaborare e comprendere testi complessi, offrendo risposte puntuali e contestualizzate.

3.2.1 Architettura della piattaforma

La piattaforma Synopsis ML è progettata secondo un'architettura modulare orientata ai servizi, con l'obiettivo di supportare il processo di sviluppo e deployment dei modelli di intelligenza artificiale, dalla configurazione fino alla loro esecuzione e integrazione nei flussi applicativi. L'architettura è concepita per operare in contesti distribuiti e multi-tenant, garantendo isolamento tra ambienti differenti e permettendo la gestione simultanea di più modelli e pipeline.

A livello generale, il sistema si articola in più componenti principali: un livello di interfaccia utente per l'interazione con gli operatori, un livello applicativo responsabile della logica di orchestrazione e un livello di persistenza dedicato alla gestione delle configura-

zioni e dei metadati. Questi elementi cooperano per costruire un’infrastruttura in grado di descrivere, configurare ed eseguire modelli in modo dinamico.

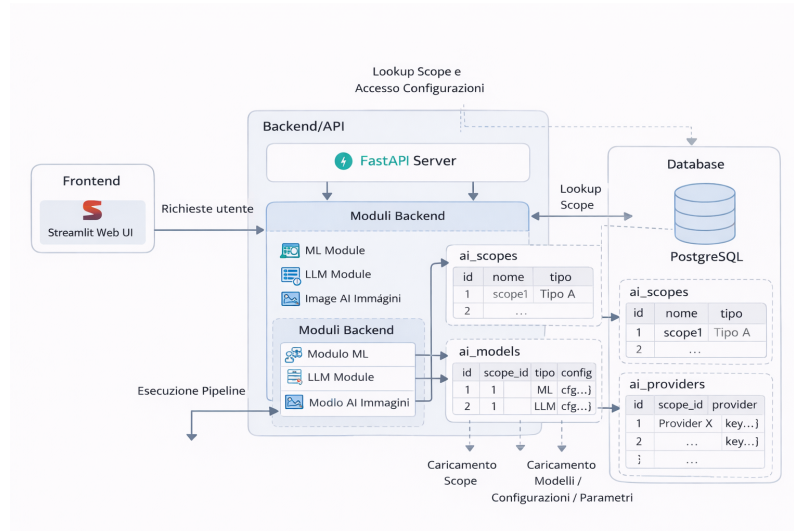


Figura 3.4: Architettura della piattaforma Synapsis ML

Il livello di presentazione è realizzato tramite un frontend sviluppato in *Streamlit*, una tecnologia che consente di costruire interfacce web interattive per applicazioni di data science e machine learning. Tale scelta permette di sviluppare rapidamente dashboard e strumenti operativi per la configurazione dei modelli, l’esecuzione delle pipeline e la visualizzazione dei risultati, mantenendo al tempo stesso un’elevata flessibilità nell’interazione con i dati. Il livello di backend è implementato tramite *FastAPI*, un framework Python progettato per la realizzazione di API REST ad alte prestazioni. FastAPI gestisce la logica applicativa della piattaforma, incluse le operazioni di orchestrazione dei modelli, la validazione dei parametri, la gestione degli scope e l’interazione con i servizi esterni. Grazie al supporto nativo per la programmazione asincrona, il sistema è in grado di gestire richieste concorrenti e flussi di lavoro distribuiti in modo efficiente. La comunicazione tra frontend e backend avviene tramite API REST, consentendo una separazione netta tra i due livelli e facilitando l’integrazione con altri servizi o interfacce future. Questo approccio rende la piattaforma facilmente estendibile e interoperabile. Il livello di persistenza è affidato a un database relazionale, utilizzato per memorizzare configurazioni, metadati, definizioni dei modelli e ulteriori informazioni. L’utilizzo di strutture JSON all’interno del database permette inoltre di rappresentare configurazioni dinamiche e adattarsi a modelli con caratteristiche differenti senza modifiche strutturali allo schema.

L’architettura multilivello consente a Synapsis ML di combinare flessibilità e robustezza, supportando l’integrazione di modelli eterogenei e l’evoluzione del sistema senza impatti significativi sulle componenti esistenti. La piattaforma è progettata per adattarsi a

diversi scenari applicativi, mantenendo al tempo stesso coerenza e controllo centralizzato delle configurazioni. Un elemento distintivo del sistema è la gestione dei modelli come entità configurabili, descritte tramite metadati e parametri persistiti. Questo approccio consente di introdurre nuovi modelli o varianti attraverso configurazione, senza intervenire direttamente sulla logica applicativa, favorendo così estendibilità, riusabilità e rapidità di evoluzione.

Nel complesso, l'architettura adottata permette di integrare in modo coerente i diversi componenti della piattaforma, supportando scenari applicativi complessi e garantendo al tempo stesso modularità, scalabilità e adattabilità alle diverse esigenze operative.

3.2.2 Livello di persistenza e modellazione dei dati

Il database rappresenta il livello di persistenza centrale della piattaforma Synapsis ML e svolge un ruolo attivo nella definizione del comportamento runtime del sistema. Al suo interno non vengono memorizzati soltanto dati anagrafici o metadati amministrativi, ma anche configurazioni, definizioni dei modelli, associazioni con i provider e informazioni necessarie alla ricostruzione del contesto operativo. Dal punto di vista implementativo, il backend sviluppato in *FastAPI* interagisce con il database tramite un livello di accesso dedicato, responsabile della gestione delle connessioni e dell'esecuzione delle query. Questo approccio consente di mantenere separata la logica applicativa dalla struttura fisica del database, facilitando la manutenzione e l'evoluzione del sistema. Il database assume quindi il ruolo di sorgente primaria della configurazione, permettendo al sistema di adattare dinamicamente il proprio comportamento senza introdurre logiche statiche.

3.2.2.1 Organizzazione delle tabelle

La struttura del database è organizzata in insiemi di tabelle che riflettono le principali responsabilità della piattaforma. Un primo gruppo riguarda la gestione dei contesti operativi, includendo entità legate agli scope e alle relative configurazioni. Un secondo gruppo descrive i modelli disponibili e i provider associati, definendo le risorse utilizzabili dal sistema. Un ulteriore insieme di tabelle è dedicato a parametri, metadati e informazioni di supporto, utilizzati per controllare il comportamento delle pipeline in fase di esecuzione. Questa organizzazione consente di separare in modo chiaro le informazioni strutturali da quelle configurative. Gli scope definiscono il contesto di lavoro, i modelli rappresentano le capacità disponibili e i provider identificano le risorse computazionali o i servizi esterni utilizzati.

3.2.2.2 Registry dei modelli

Tra le tabelle più rilevanti della piattaforma, `ai_models` occupa una posizione centrale, in quanto rappresenta il catalogo dei modelli disponibili nel sistema. Essa non si limita a memorizzare informazioni descrittive, ma contiene una rappresentazione strutturata del modello, comprensiva degli elementi necessari alla sua configurazione ed esecuzione. Le colonne *name*, *description*, *engine* e *tags* permettono di classificare funzionalmente i modelli, distinguendo ad esempio modelli di machine learning tradizionale da modelli linguistici avanzati. I campi *info_attributes* e *configuration_attributes*, memorizzati in formato JSON, consentono invece di rappresentare metadati tecnici e parametri configurabili in modo flessibile. Questa scelta progettuale permette di descrivere con un'unica struttura modelli anche molto diversi tra loro. Un classificatore supervisionato basato su librerie locali e un large language model fruito tramite provider esterno possono essere gestiti attraverso lo stesso schema, variando esclusivamente il contenuto degli attributi. In questo modo la piattaforma può introdurre nuovi modelli o aggiornare quelli esistenti senza modificare la logica applicativa. In quest'ottica, `ai_models` può essere interpretata come un vero e proprio registry applicativo, in cui ogni record definisce non solo un modello disponibile, ma anche le modalità con cui esso deve essere esposto, configurato ed eseguito all'interno del sistema.

id	name	engine	tags	info_attributes	configuration_attributes
efbca293-9c20-4d26-a5d4-91f992e557d2	Random Forest	ML	machine learning	{"library": "sklearn.ensemble", "class": "RandomForestClassifier"}	{"n_estimators": {"min": 10, "max": 1000}, "max_depth": {"min": 1, "max": 50}}
313aa7a9-15bc-4f8a-9503-911e462dc584	Qwen3-30B-A3B	LLM	41K context; reasoning; code; math	{"creator": "Qwen", "quality": 90, "tensor_type": "BF16", "size": 30.05}	{"temperature": {"min": 0.0, "max": 2.0}, "max_tokens": {"min": 1, "max": 4096}, "top_p": {"min": 0.0, "max": 1.0}}

Tabella 3.1: Estratto rappresentativo della tabella `ai_models`.

3.2.3 Gestione dei contesti operativi (Scope)

Uno degli elementi centrali del design della piattaforma è il concetto di *scope*, che rappresenta il meccanismo fondamentale di isolamento logico delle risorse e delle configurazioni. A differenza di approcci monolitici in cui le configurazioni sono globali o condivise, Synopsis ML adotta una modellazione multi-contesto in cui ogni operazione è eseguita all'interno di uno specifico scope. Uno scope può essere interpretato come un contesto computazionale autosufficiente, che incapsula tutte le informazioni necessarie per l'esecuzione delle pipeline AI. Questo include configurazioni dei modelli, provider esterni,

parametri operativi e metadati relativi alle esecuzioni. Tale astrazione consente di separare in modo rigoroso ambienti diversi, ad esempio associati a differenti clienti, progetti o casi d'uso, evitando interferenze tra configurazioni e garantendo coerenza nelle esecuzioni. Dal punto di vista architetturale, lo scope non è un semplice identificatore, ma una chiave primaria che governa l'accesso a tutte le risorse applicative. Ogni richiesta elaborata dal sistema è associata a uno scope e viene risolta attraverso una sequenza di lookup nel database, che consente di ricostruire dinamicamente il contesto operativo.

3.2.4 Integrazione con MLflow

Per supportare la tracciabilità degli esperimenti e la gestione delle esecuzioni dei modelli, la piattaforma Synapsis ML integra MLflow come componente dedicato al monitoraggio e alla registrazione delle informazioni rilevanti. Tale integrazione consente di mantenere uno storico strutturato delle esecuzioni, facilitando il confronto tra diverse configurazioni e garantendo la riproducibilità dei risultati.

All'interno della piattaforma, MLflow viene utilizzato sia nel contesto dei modelli di machine learning tradizionali sia nell'ambito dei large language model. In entrambi i casi, il sistema registra automaticamente parametri di configurazione, metriche di valutazione e metadati associati alle esecuzioni, permettendo di analizzare le prestazioni dei modelli in modo sistematico.

Nel caso dei modelli di machine learning, MLflow consente di monitorare le metriche classiche di valutazione, come accuratezza, precisione o F1-score, insieme ai parametri utilizzati durante l'addestramento. Per quanto riguarda i large language model, l'integrazione è stata estesa al tracciamento di metriche specifiche legate alla qualità delle risposte generate, come misure di similarità, coerenza o valutazioni basate su criteri applicativi. Questo permette di confrontare diversi modelli o configurazioni anche in scenari non deterministici, tipici dei sistemi basati su LLM.

L'utilizzo di MLflow avviene a livello di backend, dove le pipeline di esecuzione interagiscono con il sistema di tracciamento in modo trasparente rispetto alla logica applicativa. Ogni esecuzione viene registrata come entità indipendente, contenente tutte le informazioni necessarie per analizzare e replicare il comportamento del modello.

Questa integrazione introduce un approccio strutturato alla gestione degli esperimenti, migliorando la visibilità sulle prestazioni dei modelli e supportando le attività di validazione, confronto e ottimizzazione all'interno della piattaforma Synapsis ML.

3.2.5 Esecuzione dei modelli e gestione dei provider

La piattaforma Synapsis ML gestisce l'esecuzione dei modelli attraverso una pipeline orchestrata che consente di trattare in modo uniforme modelli eterogenei, indipendentemente dalla loro natura o dal provider utilizzato.

L'esecuzione di un modello avviene all'interno di uno specifico scope, che definisce il contesto operativo e le configurazioni associate. A partire da tale contesto, il sistema recupera dinamicamente le informazioni necessarie, incluse la definizione del modello, i parametri configurabili e il provider associato.

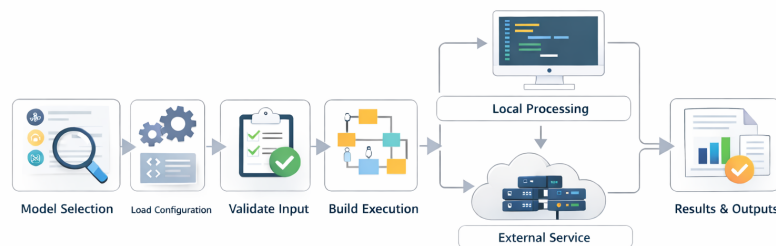


Figura 3.5: Workflow di esecuzione dei modelli in Synapsis ML

Un elemento chiave dell'architettura è l'astrazione del concetto di provider, che permette di separare la logica applicativa dai dettagli implementativi delle risorse computazionali impiegate. In particolare, la piattaforma supporta diverse modalità di esecuzione dei modelli:

- **Provider managed**, che includono servizi esterni accessibili tramite API, come OpenAI, Groq o Hugging Face, utilizzati principalmente per l'esecuzione di large language model;
- **Provider self-hosted**, basati su infrastrutture cloud configurabili, come ambienti su Amazon SageMaker o Azure, che permettono un maggiore controllo sulle risorse e sulle modalità di esecuzione;
- **Provider on-premise**, eseguiti direttamente all'interno dell'infrastruttura Synapsis ML, tramite endpoint dedicati basati su tecnologie come MLflow, Ollama o vLLM, utilizzati per garantire maggiore controllo, riduzione della latenza e gestione locale dei dati.

Indipendentemente dalla modalità di esecuzione, la piattaforma espone un'interfaccia unificata per l'invocazione dei modelli, così da integrare risorse diverse all'interno di uno stesso flusso operativo. Questa impostazione facilita l'introduzione di nuove tecnologie e garantisce al tempo stesso flessibilità, estendibilità e adattabilità del sistema a differenti scenari applicativi.

3.3 Synopsis ML: componente LLM e architettura RAG

La componente LLM di Synopsis ML è stata progettata per integrare modelli linguistici di grandi dimensioni nei flussi applicativi della piattaforma, consentendo l'elaborazione di richieste in linguaggio naturale e la generazione di risposte contestualizzate. In questo contesto, l'adozione di un'architettura RAG permette di affiancare alle capacità generative del modello l'accesso a fonti informative esterne, migliorando la pertinenza e l'aderenza delle risposte al contesto operativo.

La generazione della risposta non dipende quindi esclusivamente dalla conoscenza parametrica del modello, ma da un processo che combina recupero delle informazioni e costruzione dinamica del contesto. Questo approccio risulta particolarmente adatto in ambito applicativo, dove è necessario interrogare dati eterogenei, sia strutturati sia non strutturati, mantenendo coerenza con il dominio di riferimento.

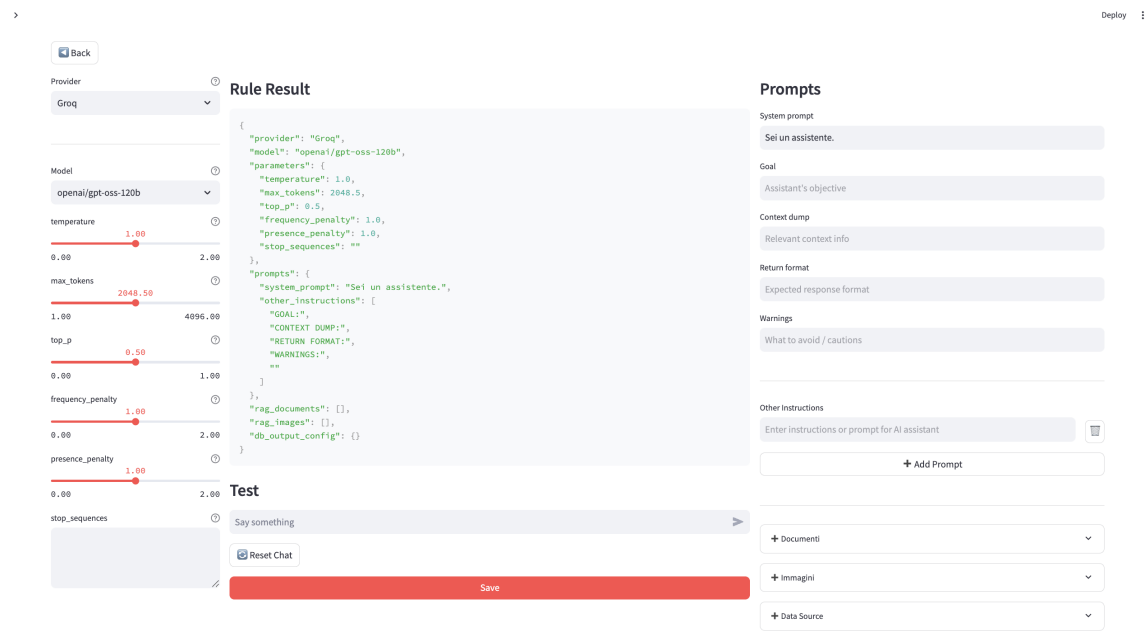


Figura 3.6: Interfaccia della componente LLM di Synopsis ML. La schermata consente di configurare il comportamento del sistema selezionando modello e provider, definendo i parametri di generazione e integrando diverse sorgenti informative, quali documenti, immagini e basi di dati, che vengono utilizzate durante il processo di generazione della risposta.

L'interfaccia rappresenta il punto di accesso alla configurazione del sistema e riflette direttamente la natura modulare della componente LLM. L'utente può infatti definire non solo il modello da utilizzare, ma anche le risorse informative e i parametri che influenzano il comportamento del sistema, contribuendo alla costruzione del contesto operativo

della richiesta. La componente LLM non è implementata come una pipeline lineare, ma come un sistema orchestrato, in cui diverse unità funzionali cooperano per produrre la risposta finale. In questo modello, il flusso di esecuzione non è predefinito, ma viene determinato dinamicamente sulla base della richiesta e delle informazioni disponibili. La gestione dell'elaborazione è affidata a un orchestratore centrale, che coordina le diverse fasi del processo: analisi della richiesta, attivazione dei moduli necessari e integrazione dei risultati. Tutti i componenti operano su un contesto condiviso, che include il prompt dell'utente, la cronologia conversazionale e i riferimenti alle risorse disponibili.

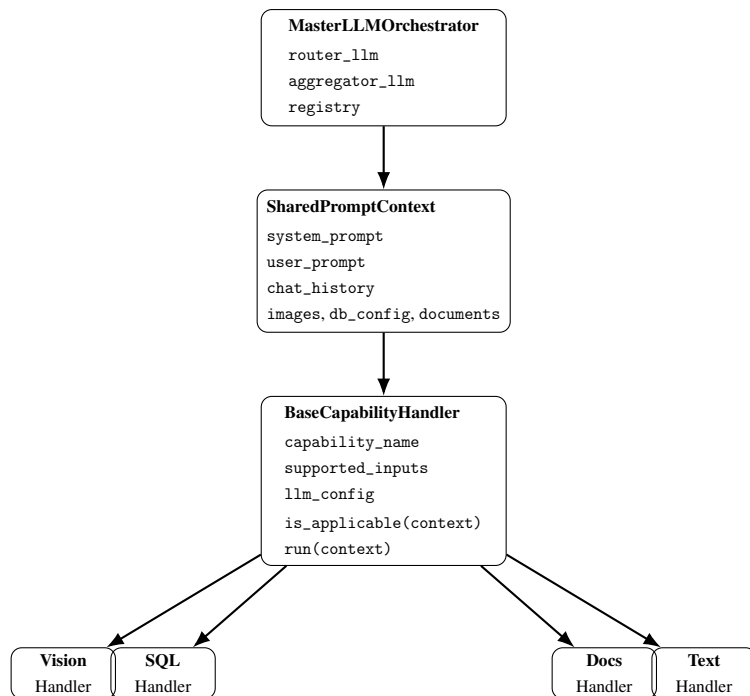


Figura 3.7: Schema semplificato dell'architettura della componente LLM. L'orchestratore coordina il flusso di esecuzione operando su un contesto condiviso e delegando le operazioni ai moduli specializzati.

Come mostrato nello schema, l'orchestratore non esegue direttamente la logica applicativa, ma coordina una serie di moduli specializzati che operano sul contesto condiviso. Questo approccio consente di separare il controllo del flusso dalla logica di elaborazione, migliorando modularità ed estendibilità del sistema.

3.3.1 Routing e orchestrazione delle capability

La generazione della risposta all'interno della componente LLM è governata da un meccanismo di orchestrazione dinamica, progettato per selezionare ed eseguire in modo adattivo i moduli applicativi più pertinenti rispetto alla richiesta dell'utente. A differenza delle

pipeline RAG tradizionali, caratterizzate da una sequenza statica di operazioni, il sistema adotta un approccio flessibile in cui il flusso di esecuzione viene determinato a *runtime*, sulla base del contenuto della richiesta e del contesto disponibile. Questo modello consente di gestire scenari complessi e multimodali, nei quali la risposta non può essere ottenuta tramite una singola strategia di retrieval, ma richiede la combinazione di più capacità operative, quali l'accesso a documenti, le interrogazioni strutturate o l'analisi di contenuti visuali. In questo senso, l'orchestrazione non coincide con una semplice sequenza di chiamate, ma rappresenta il livello di controllo che decide quali moduli coinvolgere, in quale combinazione e con quali informazioni di contesto.

In tale contesto, il sistema introduce il concetto di *capability*, ovvero unità funzionali specializzate che implementano specifiche modalità di elaborazione della richiesta. Una *capability* può essere interpretata come un modulo autonomo incaricato di eseguire una determinata operazione, ad esempio il recupero di informazioni da documenti, l'interrogazione di basi di dati strutturate o l'analisi di contenuti multimodali. Le *capability* sono progettate per essere indipendenti tra loro, riutilizzabili e componibili all'interno dello stesso flusso di esecuzione. Ciascuna di esse opera su un contesto condiviso e produce un risultato strutturato, che può essere successivamente integrato con quelli generati da altre *capability*. In questo modo, la risposta finale emerge dalla cooperazione di più moduli specializzati, selezionati dinamicamente in funzione della richiesta dell'utente.

Un aspetto centrale di questa impostazione è la separazione tra il livello decisionale e il livello esecutivo. La decisione su quali *capability* attivare viene demandata a un componente specializzato di routing, mentre l'effettiva esecuzione è affidata a un orchestratore che coordina moduli indipendenti. Tale distinzione consente di mantenere separata la logica di selezione dalla logica operativa, evitando che ciascuna *capability* debba conoscere il comportamento delle altre. Il processo di orchestrazione, infatti, si articola in tre fasi principali: il routing della richiesta, l'esecuzione delle *capability* selezionate e l'aggregazione dei risultati. Tali fasi sono implementate rispettivamente dai componenti *RouterLLM*, *MasterLLMOrchestrator* e *AggregatorLLM*, che operano su un contesto condiviso contenente tutte le informazioni necessarie all'elaborazione.

Il contesto condiviso costituisce l'elemento che garantisce continuità tra i diversi moduli della pipeline. Esso non contiene soltanto il prompt dell'utente, ma anche la cronologia conversazionale, i riferimenti alle risorse disponibili e gli eventuali parametri necessari per accedere a sorgenti esterne. In questo modo, ciascuna *capability* riceve una rappresentazione coerente dello stato della richiesta, senza dover ricostruire autonomamente le informazioni di cui ha bisogno.

3.3.1.1 Routing della richiesta

La prima fase consiste nella selezione delle capability da attivare. Questa decisione non è basata su regole statiche, ma viene delegata a un modello linguistico, che analizza la richiesta dell'utente e la cronologia conversazionale per determinare quali operazioni siano necessarie.

Il componente responsabile è *RouterLLM*, il cui comportamento è illustrato di seguito.

Logica di routing delle capability

```
1 class RouterLLM:
2     async def route(self, context):
3         result = await (self.prompt | self.llm).ainvoke({
4             "input": context.user_prompt,
5             "chat_history": self._to_history(context.chat_history)
6         })
7
8         try:
9             decision = json.loads(
10                 re.search(r"\{.*\}", result.content, re.DOTALL).group(0)
11             )
12         except Exception:
13             decision = {}
14
15         return decision
```

Il metodo `route` riceve in input il contesto della richiesta e costruisce un prompt che include sia la query dell'utente sia la cronologia delle interazioni precedenti. Il modello linguistico viene quindi invocato per produrre una decisione strutturata, espressa in formato JSON. Questa scelta è particolarmente rilevante dal punto di vista progettuale, poiché consente di trasformare un output generativo in una struttura formalizzata e direttamente utilizzabile dal backend.

La fase di parsing successiva consente di trasformare l'output testuale del modello in una struttura dati interpretabile dal sistema. In caso di errore, viene restituito un oggetto vuoto, garantendo robustezza rispetto a output non conformi. Sebbene questa logica sia implementativamente semplice, essa svolge una funzione importante, introduce un livello di tolleranza agli errori che evita il fallimento completo della pipeline in presenza di risposte mal formate da parte del router.

Il risultato del routing assume tipicamente la forma di una struttura JSON contenente flag booleani associati alle diverse capability, ad esempio:

```
{
    "vision": false,
    "sql": true,
    "docs": true
}
```

Questa rappresentazione permette di identificare in modo esplicito le capability da attivare, eliminando la necessità di logiche di branching *hard-coded*. Inoltre, rende osservabile la decisione del router, facilitando il debugging e l'analisi del comportamento del sistema. In altre parole, il routing non è soltanto una decisione interna del modello, ma un passaggio intermedio esplicito e tracciabile.

3.3.1.2 Selezione ed esecuzione delle capability

Una volta determinata la configurazione di routing, il controllo passa al componente centrale del sistema, *MasterLLMOrchestrator*. Questo modulo ha la responsabilità di selezionare, istanziare ed eseguire le capability appropriate.

Il processo è illustrato di seguito.

```
Logica di routing delle capability
1 routing = await self.router.route(context)
2
3 handlers = []
4 for handler_cls in self.registry.get_all():
5     if routing.get(handler_cls.capability_name):
6         handler = handler_cls(...)
7         if handler.is_applicable(context):
8             handlers.append(handler)
9
10 results = await asyncio.gather(
11     *(h.run(context) for h in handlers)
12 )
```

In questa fase, il sistema interroga il *Capability Registry*, che rappresenta il catalogo delle funzionalità disponibili. Per ciascuna capability, viene verificata la presenza nel risultato del routing e successivamente l'effettiva applicabilità al contesto corrente. In questo modo, il router non seleziona direttamente istanze operative, ma individua classi di comportamento potenzialmente rilevanti, lasciando all'orchestratore il compito di risolvere la selezione concreta.

Questo doppio livello di selezione consente di evitare esecuzioni non pertinenti e di garantire che ogni modulo venga attivato solo quando necessario. La presenza del metodo `is_applicable` introduce infatti un ulteriore controllo semantico, una capability può essere teoricamente compatibile con la richiesta, ma non esserlo con i dati effettivamente disponibili nel contesto. Un esempio tipico è il caso di una richiesta che richieda analisi visuale ma in assenza di immagini, oppure un'interrogazione strutturata quando non è presente alcuna configurazione di accesso a basi di dati.

Le capability selezionate vengono quindi eseguite in parallelo tramite meccanismi asincroni. Tale scelta architetturale consente di ridurre la latenza complessiva del sistema e di combinare in modo efficiente informazioni provenienti da sorgenti eterogenee.

L'esecuzione concorrente è particolarmente utile nei casi in cui i moduli abbiano tempi di risposta differenti o debbano interagire con servizi esterni.

Ogni capability produce un risultato strutturato, che include una risposta testuale e metadati associati all'elaborazione. Questa uniformità dei risultati è un elemento essenziale per il funzionamento della fase successiva di aggregazione, poiché consente al sistema di trattare in modo omogeneo contributi provenienti da moduli eterogenei.

3.3.1.3 Aggregazione della risposta

L'ultima fase del processo consiste nella costruzione della risposta finale. I risultati prodotti dalle diverse capability vengono combinati attraverso il componente *AggregatorLLM*, come mostrato di seguito.

```
Logica di routing delle capability
1 async def aggregate(self, context, results):
2     formatted = "\n\n".join(
3         f"[{r.capability.upper()}]\n{r.answer}"
4         for r in results
5     )
6
7     out = await (self.prompt | self.llm).ainvoke({
8         "question": context.user_prompt,
9         "results": formatted
10    })
11
12    return out.content
```

I risultati parziali vengono prima trasformati in una rappresentazione testuale uniforme, che viene poi fornita al modello linguistico insieme alla domanda originale. Il modello ha quindi il compito di sintetizzare le informazioni disponibili in una risposta coerente e completa. In questa fase il ruolo del modello non è più quello di decidere cosa fare, bensì di integrare contributi già prodotti, armonizzandoli in un testo unitario.

A differenza di una semplice concatenazione, questa fase realizza una vera e propria fusione semantica, in cui il modello seleziona le informazioni rilevanti ed elimina eventuali ridondanze o incongruenze. Questo passaggio è particolarmente importante quando due capability producono contributi complementari ma formulati in modo differente, oppure quando una risposta finale deve combinare contenuti documentali e dati strutturati in un'unica esposizione coerente.

L'aggregazione svolge quindi una funzione duplice: da un lato migliora la leggibilità dell'output finale, dall'altro evita che l'utente riceva una risposta frammentata o eccessivamente legata alla struttura interna della pipeline. In questo senso, il componente di aggregazione agisce come livello di astrazione finale tra l'architettura interna del sistema e l'interfaccia conversazionale esposta all'utente.

3.3.1.4 Flusso complessivo

Il comportamento del sistema può essere interpretato come una pipeline dinamica composta da tre passaggi:

1. analisi della richiesta e selezione delle capability tramite *RouterLLM*;
2. esecuzione parallela dei moduli selezionati tramite *MasterLLMOrchestrator*;
3. aggregazione dei risultati tramite *AggregatorLLM*.

Tutti i componenti operano su un contesto condiviso, che rappresenta lo stato della richiesta e include prompt, cronologia conversazionale e riferimenti alle risorse disponibili. La presenza di questo contesto comune consente di mantenere coerenza tra le fasi della pipeline e di evitare duplicazioni nella gestione delle informazioni di input.

L'adozione di un meccanismo di orchestrazione basato su LLM consente di superare i limiti delle pipeline tradizionali, introducendo un comportamento adattivo e modulare. Questo approccio risulta particolarmente efficace in scenari applicativi complessi, nei quali è necessario combinare diverse strategie di elaborazione. Tuttavia, tale flessibilità introduce anche alcune criticità, tra cui la dipendenza dalla correttezza del routing e la necessità di garantire coerenza tra i risultati prodotti dai diversi moduli. A ciò si aggiunge il fatto che l'osservabilità del sistema diventa più importante rispetto a pipeline lineari. Per comprendere il comportamento complessivo non basta analizzare l'output finale, ma è spesso necessario ispezionare anche il routing intermedio, i risultati parziali e la fase di aggregazione.

Nonostante ciò, l'architettura adottata rappresenta un compromesso efficace tra flessibilità, estendibilità e controllo del sistema, risultando particolarmente adatta a contesti in cui la richiesta dell'utente può richiedere l'attivazione di percorsi elaborativi differenti a seconda del contenuto, del contesto e delle risorse disponibili.

3.3.2 Implementazione della pipeline RAG

L'implementazione della pipeline RAG rappresenta il livello operativo attraverso cui la componente LLM integra sorgenti informative esterne nel processo di generazione della risposta. In questa fase, il sistema organizza gli input disponibili in una rappresentazione interna uniforme e li rende disponibili ai moduli che si occupano del recupero delle informazioni e della generazione del testo.

Dal punto di vista esecutivo, la pipeline comprende tre passaggi principali: la normalizzazione degli input, la costruzione del contesto operativo e l'invocazione dei componenti responsabili della risposta finale. Il sistema acquisisce il prompt dell'utente, la

cronologia conversazionale e le eventuali risorse associate, come documenti, immagini o configurazioni di accesso a basi di dati, trasformandoli in strutture interne omogenee.

Su questa base viene costruito un contesto condiviso che raccoglie tutte le informazioni necessarie all'elaborazione, tra cui il prompt, la memoria conversazionale, i riferimenti alle sorgenti informative e gli eventuali contenuti multimodali. Tale contesto costituisce il punto di raccordo tra i diversi moduli della pipeline e consente di trattare in modo uniforme dati eterogenei. A partire da questo livello comune, il sistema può attivare i meccanismi di retrieval documentale, costruire la chain di generazione e integrare i contributi provenienti dalle diverse sorgenti informative.

3.3.2.1 Indicizzazione e retrieval documentale

Quando la richiesta coinvolge contenuti documentali, la pipeline RAG attiva un processo di retrieval semantico basato su rappresentazioni vettoriali. In questa fase il sistema non lavora direttamente sui documenti grezzi, ma su una loro trasformazione in embedding, ovvero vettori numerici che catturano il significato semantico del contenuto. Questo consente di superare i limiti del matching lessicale tradizionale, permettendo il recupero di informazioni rilevanti anche in presenza di variazioni linguistiche.

Il processo di retrieval si articola in due passaggi principali: la costruzione degli embedding e la creazione di un indice vettoriale interrogabile. Per la generazione degli embedding testuali, l'implementazione utilizza modelli della famiglia *sentence-transformers*, eseguiti localmente tramite *HuggingFaceEmbeddings*. In particolare, il modello adottato è *all-MiniLM-L12-v2*, che rappresenta un compromesso tra qualità semantica e costo computazionale, risultando adeguato per scenari applicativi in cui è necessario mantenere bassa la latenza.

Costruzione degli embedding testuali

```
1 MODEL_MEDIUM = "sentence-transformers/all-MiniLM-L12-v2"
2
3 def build_embeddings(offline: bool = True):
4     if offline:
5         return HuggingFaceEmbeddings(
6             model_name=MODEL_MEDIUM,
7             model_kwargs={"device": "cpu"},
8             encode_kwargs={"normalize_embeddings": True}
9         )
10    else:
11        return OpenAIEmbeddings()
```

L'utilizzo di embedding normalizzati consente di migliorare la stabilità del confronto tra vettori, tipicamente basato su misure di similarità come il coseno. Una volta costruito lo spazio vettoriale, il retrieval avviene confrontando l'embedding della query con quelli dei documenti indicizzati. In particolare, il sistema:

- genera l'embedding della query dell'utente;
- calcola la similarità tra tale embedding e quelli presenti nell'indice;
- seleziona i k documenti più simili, che vengono restituiti come contesto rilevante.

Questo meccanismo consente di recuperare contenuti semanticamente affini anche in assenza di corrispondenze lessicali esplicite, migliorando la qualità delle informazioni fornite al modello generativo.

Una volta generati gli embedding, il sistema costruisce un indice vettoriale che permette di effettuare query per similarità. L'architettura distingue due modalità operative, una temporanea, utilizzata per richieste estemporanee, e una persistente, adottata per basi documentali stabili.

Nel caso di richieste temporanee, il sistema utilizza FAISS per creare un indice in memoria. FAISS è una libreria sviluppata da Meta progettata per la ricerca efficiente di vettori ad alta dimensionalità, particolarmente adatta a scenari in cui è necessario eseguire query di similarità in modo rapido su dataset anche di grandi dimensioni. Nel contesto della pipeline, FAISS viene utilizzato come vector store in memoria, senza meccanismi di persistenza.

Creazione del retriever temporaneo in memoria

```
1 def create_ephemeral_retriever(docs: List[Document]):  
2     text_embeddings = build_embeddings()  
3     vectordb = FAISS.from_documents(docs, text_embeddings)  
4  
5     return vectordb.as_retriever(search_kwargs={"k": 4})
```

L'utilizzo di un indice in memoria consente di minimizzare i tempi di costruzione e accesso, rendendo questa soluzione particolarmente efficiente per interazioni interattive. Il parametro $k = 4$ definisce il numero massimo di documenti recuperati per ciascuna query e rappresenta un compromesso tra ricchezza del contesto e contenimento del rumore informativo.

Per scenari in cui è necessario mantenere una base documentale nel tempo, il sistema utilizza invece un archivio persistente basato su Chroma. Chroma è una vector store progettata per applicazioni di retrieval semantico, che consente di memorizzare embedding su disco e di riutilizzarli tra diverse esecuzioni, supportando inoltre metadati e filtri sulle query.

Creazione del retriever persistente

```
1 def create_persistent_store(persist_dir: str) -> Chroma:
2     embeddings = build_embeddings()
3
4     return Chroma(
5         persist_directory=persist_dir,
6         embedding_function=embeddings
7     )
8
9 def create_persistent_retriever(persist_dir: str, rule_id: str = None):
10     vectordb = create_persistent_store(persist_dir)
11
12     if rule_id:
13         retriever = vectordb.as_retriever(
14             search_kwargs={"k": 4, "filter": {"rule_id": str(rule_id)}}
15         )
16     else:
17         retriever = vectordb.as_retriever(search_kwargs={"k": 4})
18
19     return retriever
```

La distinzione tra indice temporaneo e persistente rappresenta una scelta architetturale rilevante. Da un lato, l'approccio in memoria garantisce flessibilità e rapidità nelle richieste ad hoc, dall'altro, l'utilizzo di uno store persistente consente di costruire basi documentali riutilizzabili e coerenti nel tempo. Inoltre, l'introduzione del filtro su `rule_id` permette di isolare logicamente i documenti associati a specifici contesti applicativi, evitando interferenze tra diverse configurazioni.

L'indicizzazione dei documenti persistenti prevede infatti l'arricchimento dei metadati con identificatori contestuali, che vengono poi utilizzati in fase di retrieval per limitare lo spazio di ricerca ai soli contenuti rilevanti.

Indicizzazione dei documenti nello store persistente

```
1 def index_document_to_chroma(persist_dir: str, rule_id: str, docs: List[Document]):
2     for doc in docs:
3         doc.metadata['rule_id'] = rule_id
4
5     vectordb = create_persistent_store(persist_dir)
6     vectordb.add_documents(docs)
```

Nel complesso, il retrieval documentale è progettato come un componente indipendente e riutilizzabile della pipeline RAG. Esso non è integrato rigidamente nella fase di generazione, ma viene reso disponibile attraverso il contesto condiviso e attivato solo quando necessario. Questa separazione consente di mantenere il sistema modulare ed estendibile, facilitando l'introduzione di nuove strategie di indicizzazione o di modelli di embedding senza impattare la logica complessiva della pipeline.

3.3.2.2 Costruzione della chain di generazione

Una volta definito il retriever documentale, il passo successivo consiste nella costruzione della chain di generazione, ossia della struttura esecutiva che collega il modello linguistico, il contesto della richiesta e i moduli già descritti nella sezione precedente. In questa fase il sistema non introduce nuove componenti concettuali, ma istanzia operativamente gli elementi necessari all'esecuzione della richiesta, traducendo configurazioni astratte e risorse disponibili in un flusso realmente invocabile.

La funzione centrale di questa fase è `build_history_aware_rag`, che riceve in ingresso il provider selezionato, il modello, la configurazione del sistema e il retriever documentale. A partire da questi elementi, essa costruisce una chain unificata che restituisce la risposta finale sotto forma di `RunnableLambda`. Il termine *history-aware* evidenzia come la generazione non dipenda soltanto dalla query corrente, ma tenga conto anche della cronologia conversazionale e delle eventuali sorgenti informative associate alla richiesta.

Il primo passaggio riguarda l'inizializzazione del modello linguistico. Tale fase dipende dal provider configurato e consente di astrarre le differenze tra endpoint esterni e modelli eseguiti localmente o tramite infrastrutture proprietarie. In altri termini, la chain viene costruita in modo indipendente dal backend computazionale effettivamente utilizzato. Ciò che varia è il componente LLM istanziato, mentre la struttura generale del flusso rimane invariata. Nel caso di endpoint esterni, ad esempio, il modello viene inizializzato tramite i client `LangChain` corrispondenti.

Inizializzazione del modello in funzione del provider

```
1 if provider.name.lower() == "openai":
2     llm = ChatOpenAI(
3         model=model.name,
4         temperature=config["parameters"]["temperature"],
5         max_tokens=config["parameters"]["max_tokens"],
6         top_p=config["parameters"]["top_p"],
7         frequency_penalty=config["parameters"]["frequency_penalty"],
8         presence_penalty=config["parameters"]["presence_penalty"]
9     )
10
11 elif provider.name.lower() == "groq":
12     llm = ChatGroq(
13         model=model.name,
14         temperature=config["parameters"]["temperature"],
15         max_tokens=config["parameters"]["max_tokens"]
16     )
```

Questa parte della pipeline svolge un ruolo importante dal punto di vista architetturale, poiché separa la logica applicativa dalla tecnologia specifica utilizzata per la generazione. La chain non dipende quindi da un singolo provider, ma può essere costruita dinamicamente a partire dalla configurazione selezionata, mantenendo invariata la logica complessiva di elaborazione.

In alcuni casi il modello non viene invocato tramite un provider API esterno, ma viene caricato a partire da artifact registrati in MLflow all'interno dell'infrastruttura della piattaforma. In questa situazione la funzione recupera i file del modello, inizializza tokenizer e pesi e costruisce una pipeline Hugging Face compatibile con il resto del sistema. Sebbene questa parte sia più articolata sul piano implementativo, il suo ruolo all'interno della chain rimane lo stesso, ovvero produrre un oggetto LLM conforme all'interfaccia attesa dal resto del sistema.

Dopo l'inizializzazione del modello, la funzione prepara le configurazioni da associare ai diversi moduli specializzati. Questa fase non ridefinisce il meccanismo di orchestrazione, ma fornisce ai moduli le dipendenze necessarie per operare correttamente nel contesto corrente. Nel caso della capability documentale, ad esempio, vengono passati il retriever e la configurazione RAG, per le capability SQL e visuale viene invece fornito direttamente il modello linguistico.

Configurazione delle capability nella chain

```
1 capability_configs = {  
2     "docs": {  
3         "rag_config": {  
4             "retriever": retriever,  
5             "provider": provider,  
6             "model": model,  
7             "config": config,  
8         }  
9     },  
10    "sql": {  
11        "llm": llm  
12    },  
13    "vision": {  
14        "llm": llm  
15    },  
16 }
```

Questa struttura consente di associare a ciascun modulo soltanto le informazioni di cui ha effettivamente bisogno, evitando di concentrare tutta la logica in un unico punto e semplificando la gestione delle dipendenze. Le capability, infatti, non accedono direttamente a risorse globali o a variabili esterne, ma ricevono in ingresso una configurazione coerente con il contesto della richiesta. In questo modo, la chain funge da livello di assemblaggio delle dipendenze, facilitando il controllo del comportamento dei singoli moduli e la manutenibilità dell'implementazione.

Una volta preparate le configurazioni, vengono istanziati i componenti necessari all'esecuzione. In questa fase la funzione costruisce il router, l'aggregatore, il registry delle capability e l'orchestratore centrale, collegandoli tra loro all'interno della pipeline operativa.

Assemblaggio dei componenti della chain

```
1 router = RouterLLM(llm)
2 aggregator = AggregatorLLM(llm)
3
4 registry = CapabilityRegistry()
5 registry.register(DocsRAGCapabilityHandler)
6 registry.register(SQLCapabilityHandler)
7 registry.register(VisionCapabilityHandler)
8
9 orchestrator = MasterLLMOrchestrator(
10     router=router,
11     aggregator=aggregator,
12     registry=registry,
13     llm_factory=lambda cap: llm,
14     capability_configs=capability_configs,
15 )
```

La chain non è tuttavia completa finché non viene costruito il contesto condiviso della richiesta. È proprio all'interno di questo oggetto che confluiscono gli elementi eterogenei necessari all'elaborazione: prompt di sistema, prompt utente, cronologia conversazionale, retriever, connessione al database, schema relazionale ed eventuali immagini codificate in base64. In questo senso, il contesto rappresenta il punto di convergenza tra la fase di preparazione degli input e quella di esecuzione vera e propria.

Costruzione del contesto condiviso della richiesta

```
1 context = SharedPromptContext(
2     system_prompt=system_prompt,
3     other_prompt=other,
4     user_prompt=inputs["input"],
5     chat_history=chat_history,
6     retriever=inputs.get("retriever"),
7     db=inputs.get("db"),
8     db_schema=db_schema,
9     images_base64=images_base64,
10    rag_config={
11        "provider": provider,
12        "model": model,
13        "config": config,
14    },
15 )
```

La presenza esplicita di questo oggetto rende la pipeline più leggibile e controllabile. Invece di distribuire le informazioni della richiesta tra più parametri scollegati, il sistema raccoglie tutto in una struttura unificata, che può poi essere passata ai diversi moduli senza ambiguità. Ciò semplifica anche eventuali estensioni future, poiché nuovi campi possono essere aggiunti al contesto senza alterare in modo sostanziale la logica di composizione della chain.

Infine, dopo il controllo dell'applicabilità delle capability, la funzione incapsula l'intero comportamento in una `RunnableLambda`, che rappresenta l'oggetto effettivamente esposto alla pipeline applicativa. In questo modo la chain può essere invocata come un'unità autonoma, pur includendo al proprio interno la gestione del contesto, l'esecuzione dei

moduli e la produzione della risposta finale.

Restituzione della chain invocabile

```
1 async def run(inputs: dict) -> dict:
2     ...
3     result = await orchestrator.run(context)
4     return {"answer": result.final_answer}
5
6 return RunnableLambda(run)
```

Nel complesso, la costruzione della chain corrisponde al momento in cui configurazioni, risorse informative e moduli della pipeline vengono composti in un flusso eseguibile. Se la sezione precedente ha descritto il comportamento generale del sistema, questa fase mostra invece come tale comportamento venga concretamente predisposto a runtime, rendendo possibile l'invocazione della pipeline RAG all'interno della piattaforma.

3.3.3 Gestione delle sorgenti informative

La gestione delle sorgenti informative rappresenta un passaggio centrale nella pipeline RAG, in quanto determina il modo in cui gli input dell'utente vengono acquisiti, trasformati e resi disponibili ai moduli di elaborazione.

A livello di interfaccia, l'utente può fornire diverse tipologie di input, tra cui prompt testuali, documenti, immagini e configurazioni per l'accesso a basi di dati. Tali informazioni vengono raccolte dal frontend e inviate al backend sotto forma di payload strutturato, che include sia il contenuto della richiesta sia le eventuali risorse associate. Nel backend, questi input vengono elaborati e trasformati in una rappresentazione interna uniforme, che confluisce nel contesto condiviso della richiesta. In questa fase non avviene ancora alcuna elaborazione semantica, ma esclusivamente una preparazione dei dati che consente ai moduli successivi di operare su strutture coerenti.

Le modalità di gestione dipendono dalla tipologia di sorgente, i documenti vengono convertiti in oggetti indicizzabili, le immagini vengono trasformate in rappresentazioni compatibili con modelli multimodali, mentre le basi di dati vengono esposte attraverso configurazioni di connessione e schemi interrogabili.

3.3.3.1 Documenti testuali

I documenti testuali rappresentano una delle principali sorgenti informative gestite dalla pipeline RAG di Synapsis ML. Il loro ruolo è fornire contenuto esterno al modello linguistico, così da integrare la generazione con informazioni contestuali non contenute nella sola conoscenza parametrica del modello. La gestione dei documenti non si limita al semplice caricamento dei file, ma comprende una sequenza di operazioni che va dall'ac-

quisizione degli input alla loro trasformazione in oggetti documentali uniformi, fino alla successiva indicizzazione nel retriever vettoriale.

La piattaforma supporta diversi formati documentali, tra cui file PDF, documenti Word, file di testo semplice, file Markdown, pagine HTML, CSV ed Excel. Questa eterogeneità riflette i contesti applicativi in cui il sistema viene utilizzato, nei quali le informazioni possono provenire da manuali tecnici, documentazione operativa, tabelle esportate, report o dataset strutturati in formato tabellare. Per rendere possibile un trattamento uniforme di tali sorgenti, il sistema adotta una fase di parsing basata su loader specifici per estensione, ognuno incaricato di estrarre il contenuto nel modo più adatto al formato di origine.

La funzione che realizza questo comportamento è `create_document_splits`, la quale riceve in ingresso una collezione di file caricati dall'utente sotto forma di coppie (`filename`, `content_bytes`). Per ciascun file, il sistema individua l'estensione, salva temporaneamente il contenuto su disco e seleziona il loader corrispondente. In questo modo, file strutturalmente diversi possono essere convertiti in una collezione comune di oggetti `Document`, compatibili con il resto della pipeline.

Caricamento dei documenti tramite loader specifici

```
1 def create_document_splits(files: Iterable[tuple[str, bytes]]) -> List[Document]:
2     docs: List[Document] = []
3     loader: BaseLoader
4
5     for fname, content in files:
6         ext = os.path.splitext(fname)[1].lower()
7
8         tmp_path = os.path.join(PROJECT_TMP_DIR, os.path.basename(fname))
9         with open(tmp_path, "wb") as f:
10             f.write(content)
11
12         if ext == ".csv":
13             loader = CSVLoader(tmp_path)
14         elif ext == ".pdf":
15             loader = PyPDFLoader(tmp_path)
16         elif ext in (".docx", ".doc"):
17             loader = Docx2txtLoader(tmp_path)
18         elif ext in (".md", ".txt"):
19             loader = TextLoader(tmp_path, encoding="utf-8")
20         elif ext in (".html", ".htm"):
21             loader = UnstructuredHTMLLoader(tmp_path)
22         elif ext in (".xls", ".xlsx"):
23             loader = UnstructuredExcelLoader(tmp_path)
24         else:
25             loader = None
26
27         if loader is not None:
28             file_docs = loader.load()
29             for d in file_docs:
30                 d.metadata["source"] = fname
31             docs.extend(file_docs)
32
33     return docs
```

In questo modo il sistema non definisce un unico parser generico per tutti i file, ma delega l'estrazione del contenuto a componenti specializzati. Tale approccio migliora la

robustezza della pipeline, poiché consente di trattare ogni formato con strumenti adeguati, riducendo la necessità di logiche ad hoc distribuite nel codice applicativo. Inoltre, il risultato dell'operazione non dipende più dal formato originario del file, ma da una rappresentazione documentale uniforme, che costituisce il vero input per le fasi successive.

Un aspetto rilevante è che il caricamento documentale non produce direttamente testo pronto per il modello, ma una sequenza di oggetti arricchiti con metadati. In particolare, per ogni frammento caricato viene mantenuta almeno l'informazione relativa alla sorgente di provenienza, tramite il campo `source`. Questo consente di preservare un legame tra contenuto elaborato e file originale, facilitando sia la tracciabilità interna sia eventuali estensioni future, ad esempio per la citazione delle fonti o per il filtraggio dei documenti in fase di retrieval. Nel caso dei documenti multipagina o strutturati in più unità logiche, il loader restituisce più oggetti `Document`, che vengono poi aggregati nella lista finale. Di conseguenza, la pipeline lavora naturalmente su una granularità più fine rispetto al file intero. Questo comportamento è particolarmente utile nei PDF o nei fogli elettronici, in cui contenuti distinti possono essere recuperati separatamente senza dover necessariamente trattare l'intero file come un blocco unico.

La scelta di scrivere temporaneamente i file su disco prima del caricamento risponde a esigenze di compatibilità con i loader utilizzati. Molti dei componenti di parsing adottati, infatti, operano su percorsi file piuttosto che su stream in memoria. Sebbene questa soluzione introduca un passaggio intermedio, essa semplifica l'integrazione con l'ecosistema di strumenti documentali già disponibili e consente di supportare più formati con un numero limitato di adattamenti custom. In prospettiva, tale meccanismo potrebbe essere sostituito o affiancato da una gestione completamente in memoria, ma nella configurazione attuale rappresenta un compromesso efficace tra semplicità implementativa e compatibilità operativa.

Accanto a questa modalità basata su file temporanei, è stata considerata anche una strategia basata sull'elaborazione in memoria. Il codice include, infatti, anche una funzione alternativa, `load_files_to_docs`, che realizza una conversione in memoria per alcuni formati. In questo caso il contenuto binario viene letto direttamente come stream e trasformato in oggetti `Document` senza passare dal filesystem. Tale approccio risulta più efficiente in termini di accesso ai dati, ma è applicabile solo a un sottoinsieme di formati e richiede una gestione più specifica dei parser. Per questo motivo, nella pipeline operativa è stata privilegiata la soluzione basata su file temporanei, mentre la versione in memoria rimane una possibile estensione per scenari futuri o per casi d'uso specifici.

Esempio di caricamento documentale in memoria

```
1 def load_files_to_docs(files: Iterable[tuple[str, bytes]]) -> List[Document]:
2     docs: List[Document] = []
3
4     for fname, content in files:
5         ext = os.path.splitext(fname)[1].lower()
6
7         if ext in (".md", ".txt"):
8             s = content.decode("utf-8", errors="ignore")
9             if s.strip():
10                 docs.append(Document(page_content=s, metadata={"source": fname}))
11
12         elif ext == ".pdf":
13             reader = PdfReader(io.BytesIO(content))
14             for i, page in enumerate(reader.pages):
15                 text = page.extract_text() or ""
16                 if text.strip():
17                     docs.append(Document(
18                         page_content=text,
19                         metadata={"source": fname, "page": i}
20                     ))
21
22     return docs
```

Il processo evidenzia inoltre come la trasformazione del contenuto vari in funzione del formato. Nei file di testo il contenuto viene decodificato direttamente e inserito come `page_content`, mentre nei PDF l'estrazione avviene a livello di singola pagina, con l'aggiunta di metadati che ne identificano la posizione all'interno del documento. Questo approccio consente di preservare informazioni strutturali utili al retrieval, mantenendo al contempo una rappresentazione interna uniforme.

Una volta trasformati in oggetti `Document`, i contenuti testuali diventano l'input per la fase di indicizzazione vettoriale, rendendo le informazioni accessibili al retriever in forma compatibile con la ricerca semantica. In fase di esecuzione, i documenti recuperati vengono quindi utilizzati dalla capability documentale, che costruisce un prompt vincolato basato esclusivamente sulle informazioni estratte.

Utilizzo dei documenti nella capability RAG

```
1 docs = context.retriever.get_relevant_documents(
2     context.user_prompt
3 )
4
5 docs_text = "\n\n".join(
6     f"Fonte: {d.metadata}\n{d.page_content}" for d in docs
7 )
8
9 prompt = f"Sei un assistente che risponde SOLO usando le informazioni fornite. Documenti: {
10     docs_text} Domanda: {context.user_prompt}"
11
12 out = await self.llm.ainvoke(prompt)
```

Nel complesso, il flusso descritto consente di acquisire documenti in formati differenti, normalizzarli in una struttura comune e renderli utilizzabili dal retriever documentale. In

questo modo, la pipeline può incorporare conoscenza esterna in maniera controllata, senza vincolare il processo di generazione al formato originario delle sorgenti.

3.3.3.2 Immagini e contenuti multimodali

Oltre ai documenti testuali, la piattaforma supporta l'elaborazione di contenuti visivi, consentendo di integrare immagini all'interno della richiesta e di sfruttare modelli multimodali per l'analisi del loro contenuto. Questo tipo di sorgente introduce una complessità aggiuntiva rispetto ai documenti testuali, poiché richiede una rappresentazione compatibile sia con il processo di retrieval sia con i modelli linguistici in grado di gestire input visivi.

Dal punto di vista dell'acquisizione, le immagini vengono caricate dal frontend in modo analogo ai documenti e trasmesse al backend come contenuti binari. In particolare, il file viene letto come sequenza di byte e inviato all'interno della richiesta senza alcuna trasformazione preventiva. A differenza dei file testuali tali contenuti non vengono convertiti in testo, ma mantenuti nella loro forma originaria e associati a oggetti Document attraverso metadati specifici.

La gestione delle immagini avviene già in fase di costruzione della collezione documentale, dove esse vengono rappresentate come documenti privi di contenuto testuale ma arricchiti con informazioni necessarie per il processamento successivo.

Rappresentazione delle immagini come documenti

```
1 elif ext in (".png", ".jpg", ".jpeg"):  
2     doc = Document(  
3         page_content="",  
4         metadata={  
5             "source": fname,  
6             "image_bytes": content,  
7             "path": tmp_path  
8         }  
9     )  
10    docs.append(doc)
```

In questo modo, non è necessario introdurre una struttura separata per le immagini: il sistema le integra nel medesimo modello dati utilizzato per i documenti testuali. Ciò consente all'intera pipeline di operare su una rappresentazione uniforme, distinguendo i diversi tipi di contenuto attraverso i metadati. In particolare, la presenza del campo `image_bytes` permette di identificare i documenti di tipo visivo e di attivare comportamenti specifici nelle fasi successive.

Una volta acquisite, le immagini non vengono utilizzate nel processo di retrieval semantico, ma sono destinate a essere elaborate direttamente dai modelli multimodali. Per rendere possibile tale integrazione, il sistema adotta una fase di codifica in formato ba-

se64, che consente di trasformare il contenuto binario in una rappresentazione testuale compatibile con le interfacce dei modelli linguistici.

Conversione delle immagini in base64

```
1 def encode_image(image_path):  
2     with open(image_path, 'rb') as image_file:  
3         return base64.b64encode(image_file.read()).decode('utf-8')
```

La codifica base64 è necessaria perché i modelli multimodali non ricevono direttamente file binari, ma richiedono che le immagini siano incluse all'interno del prompt in formato testuale. In particolare, il contenuto dell'immagine viene incapsulato in una stringa che può essere trasmessa come *data URL*, mantenendo compatibilità con le API dei modelli linguistici. Le immagini codificate vengono quindi incluse nel contesto condiviso della richiesta, rendendole disponibili alle capability specializzate. In fase di esecuzione, la presenza di contenuti visivi abilita automaticamente l'attivazione di un modulo dedicato all'elaborazione multimodale.

Esecuzione della capability vision

```
1 image_prompts = [  
2     {  
3         "type": "image_url",  
4         "image_url": {  
5             "url": f"data:image/png;base64,{b64}",  
6             "detail": "low",  
7         }  
8     }  
9     for b64 in context.images_base64  
10 ]  
11  
12 prompt = ChatPromptTemplate.from_messages([  
13     ("system", context.system_prompt),  
14     ("user", image_prompts),  
15 ])  
16  
17 out = await (prompt | self.llm).ainvoke({})
```

Le immagini vengono così integrate direttamente nella struttura del prompt secondo il formato richiesto dai modelli multimodali, consentendo di combinare input visivi e testuali all'interno della stessa interazione. La gestione delle immagini permette quindi di supportare scenari in cui il contenuto visivo contribuisce direttamente alla costruzione della risposta, estendendo la pipeline verso contesti multimodali.

3.3.3.3 Basi di dati relazionali

Quando la richiesta richiede l'accesso a informazioni strutturate, il sistema utilizza una capability dedicata all'interrogazione di basi di dati relazionali. In questo caso il contesto

include sia l'oggetto di connessione al database sia lo schema delle tabelle, che viene fornito al modello per guidare la generazione della query.

La configurazione fornita dall'utente viene utilizzata per costruire dinamicamente una connessione al database. Questa operazione è gestita dalla funzione `create_db_connection`, che traduce i parametri di configurazione in una stringa di connessione compatibile con il sistema utilizzato.

Creazione della connessione al database

```
1 def create_db_connection(db_config: str) -> SQLiteDatabase:
2     config = json.loads(db_config)
3
4     db_type = config["db_type"].lower()
5
6     if db_type == "postgresql":
7         db_uri = (
8             f"postgresql+psycopg2://{config['db_username']}:"
9             f"{config['db_password']}@{config['host']}:"
10            f"{config['port']}/{config['db_name']}"
11        )
12
13     elif db_type == "mysql":
14         db_uri = (
15             f"mysql+pymysql://{config['db_username']}:"
16             f"{config['db_password']}@{config['host']}:"
17             f"{config['port']}/{config['db_name']}"
18        )
19
20     elif db_type == "sqlite":
21         db_uri = f"sqlite://{config['db_path']}"
22
23     else:
24         raise ValueError(f"Tipo di DB non supportato: {db_type}")
25
26     return SQLiteDatabase.from_uri(db_uri)
```

La funzione consente di supportare diversi sistemi di gestione di basi di dati attraverso un'unica interfaccia, delegando la costruzione della connessione alla configurazione fornita a runtime. In questo modo, il sistema rimane indipendente dal tipo di database utilizzato, favorendo la flessibilità e l'estendibilità della piattaforma.

L'approccio adottato rientra nel paradigma *text-to-SQL*, in cui il modello linguistico viene utilizzato per tradurre una richiesta in linguaggio naturale in una query SQL eseguibile. A differenza del retrieval documentale, in cui il recupero avviene per similarità semantica, qui l'accesso ai dati è determinato da una struttura logica esplicita. La capability SQL costruisce un prompt vincolato che guida il modello nella generazione della query, limitando le operazioni alle sole istruzioni di lettura.

Generazione della query SQL

```
1 sql_prompt = ChatPromptTemplate.from_messages([
2     (
3         "system",
4         "Sei un generatore SQL. Regole:\n"
5         "- Rispondi SOLO con SQL\n"
6         "- SOLO SELECT\n"
7         "- NIENTE markdown\n\n"
8         f"{context.system_prompt}\nSchema:\n{context.db_schema}"
9     ),
10    ("human", context.user_prompt),
11 ])
12
13 sql_out = await (sql_prompt | self.llm).ainvoke({})
```

Una volta generata, la query viene estratta e validata tramite espressioni regolari, così da isolare esclusivamente la porzione SQL rilevante ed evitare output non conformi. Successivamente, la query viene eseguita sul database e i risultati vengono trasformati in una rappresentazione intermedia.

Esecuzione della query e generazione della risposta

```
1 rows = context.db.run(sql)
2
3 answer_prompt = ChatPromptTemplate.from_messages([
4     ("system", context.system_prompt),
5     ("human", f"Risultati SQL:\n{rows}\n\nRispondi in linguaggio naturale.")
6 ])
7
8 answer = await (answer_prompt | self.llm).ainvoke({})
```

In questo modo si mantiene separata la fase di generazione della query dalla fase di formulazione della risposta. Il modello viene utilizzato inizialmente come traduttore verso SQL e successivamente come generatore di linguaggio naturale, sfruttando i risultati ottenuti dal database.

L'integrazione delle basi di dati consente di estendere le capacità della pipeline RAG oltre il recupero documentale, introducendo un accesso diretto a informazioni strutturate. Di conseguenza, il sistema è in grado di combinare contenuti descrittivi e dati operativi all'interno dello stesso processo di generazione, adattando la risposta in funzione della natura delle informazioni richieste.

3.3.3.4 Cronologia conversazionale

La cronologia conversazionale rappresenta una sorgente informativa implicita che consente al sistema di mantenere continuità tra i diversi turni dell'interazione. A differenza delle altre sorgenti, essa non viene fornita come contenuto esterno, ma viene costruita dinamicamente a partire dalle interazioni precedenti tra utente e sistema.

La cronologia viene acquisita dal frontend sotto forma di lista di messaggi e successivamente normalizzata in una rappresentazione interna uniforme. Questo processo consente di rendere i dati compatibili con i componenti della pipeline e con le interfacce dei modelli linguistici.

Normalizzazione della cronologia conversazionale

```
1 def to_langchain_history(chat_history):
2     msgs = []
3     for item in chat_history:
4         if item.get("user"):
5             msgs.append(HumanMessage(content=item["user"]))
6         if item.get("ai"):
7             msgs.append(AIMessage(content=item["ai"]))
8     return msgs
```

La sequenza di messaggi ottenuta viene inclusa nel contesto condiviso della richiesta e fornita ai modelli linguistici durante le diverse fasi della pipeline. In questo modo, la generazione della risposta non dipende esclusivamente dalla query corrente, ma tiene conto anche delle interazioni precedenti. La cronologia consente di gestire riferimenti impliciti, mantenere coerenza tra le risposte e supportare interazioni multi-turno, ad esempio nei casi in cui l'utente utilizzi espressioni anaforiche o ometta parte del contesto già discusso.

La memoria conversazionale è limitata alla sessione corrente e viene ricostruita a ogni richiesta a partire dai messaggi forniti dal frontend. Non viene quindi mantenuta una persistenza autonoma lato modello, ma è il contesto stesso a veicolare le informazioni necessarie alla continuità dell'interazione.

3.3.4 Scelte progettuali e vincoli implementativi

L'implementazione della piattaforma Synapsis ML è stata fortemente influenzata dal contesto tecnologico adottato, in particolare dalla necessità di integrare un insieme di librerie e framework appartenenti a domini differenti, quali sviluppo web, data science, machine learning e sistemi basati su modelli linguistici.

Un aspetto rilevante che ha guidato le scelte implementative riguarda il fatto che lo sviluppo è avvenuto su una base software già esistente. La piattaforma Synapsis ML, infatti, non è stata progettata come sistema isolato, ma come estensione di un'infrastruttura backend già operativa e integrata con altri moduli dell'ecosistema Mative. Questo ha comportato la necessità di adattarsi a versioni specifiche di alcune librerie già adottate, in particolare per quanto riguarda il framework *FastAPI* e le componenti correlate. Di conseguenza, le decisioni implementative non sono state guidate esclusivamente da criteri di ottimalità tecnica, ma anche da vincoli di compatibilità con il sistema esistente. L'introduzione di nuove dipendenze o l'aggiornamento di librerie già in uso avrebbe

potuto compromettere il funzionamento di altri servizi connessi, rendendo necessario un approccio conservativo nella gestione delle versioni.

Una delle principali sfide affrontate durante lo sviluppo ha riguardato la gestione delle dipendenze software. La pipeline RAG si basa infatti su un ecosistema articolato che include framework per la costruzione di API (*FastAPI*), strumenti per il machine learning (*PyTorch*, *scikit-learn*) e componenti del framework *LangChain* per l'orchestrazione dei modelli linguistici e delle pipeline di retrieval. L'integrazione di queste tecnologie ha richiesto una particolare attenzione alla compatibilità tra le versioni delle librerie. In diversi casi, aggiornamenti di singoli componenti introducevano modifiche nelle interfacce o nei comportamenti interni, rendendo necessario il fissaggio esplicito delle versioni per garantire stabilità e riproducibilità dell'ambiente di esecuzione.

In particolare, il framework *LangChain* ha rappresentato uno dei principali punti critici. Le diverse componenti utilizzate nel progetto, come *langchain-community*, *langchain-openai* e le integrazioni con i provider, presentano una forte dipendenza dalle versioni specifiche della libreria principale. L'aggiornamento a versioni più recenti avrebbe consentito l'accesso a funzionalità aggiuntive, ma avrebbe introdotto incompatibilità con il codice esistente e con le dipendenze già integrate nel backend.

Per questo motivo, non è stato possibile adottare direttamente alcune soluzioni più recenti, ed è stato necessario individuare alternative compatibili o adattare l'implementazione alle API disponibili nelle versioni in uso. Questa attività ha richiesto un processo iterativo di selezione e verifica delle librerie, con l'obiettivo di trovare un equilibrio tra funzionalità richieste e stabilità del sistema. A tal fine, è stato adottato un approccio basato su gestione dichiarativa delle dipendenze tramite *Poetry*, che consente di definire in modo preciso le versioni delle librerie e di replicare l'ambiente di sviluppo in modo consistente, evitando aggiornamenti non controllati.

Dal punto di vista dell'elaborazione dei modelli, la scelta di utilizzare librerie come *transformers* e *sentence-transformers* è stata motivata dalla necessità di supportare sia modelli locali sia integrazioni con provider esterni, mantenendo al contempo compatibilità con l'infrastruttura esistente. In questo contesto, si è privilegiato un approccio flessibile, che consente di eseguire embedding e modelli anche su CPU, riducendo la dipendenza da configurazioni hardware specifiche.

Per quanto riguarda il livello di persistenza e l'accesso ai dati, l'utilizzo di *SQLAlchemy* e driver asincroni come *asyncpg* ha richiesto un'attenta gestione del paradigma asincrono, al fine di garantire coerenza tra le operazioni di accesso al database e le chiamate ai servizi esterni.

Infine, l'integrazione di sistemi di tracciamento come *MLflow* e di provider esterni ha richiesto una progettazione orientata alla modularità, in modo da isolare le dipendenze e

ridurre l'impatto di eventuali incompatibilità o aggiornamenti futuri.

Nel complesso, le scelte implementative adottate riflettono un compromesso tra evoluzione tecnologica e stabilità del sistema. La necessità di operare su una base software esistente ha reso centrale il problema della compatibilità tra librerie, trasformando la selezione delle versioni e delle soluzioni tecniche in una parte fondamentale del processo progettuale.

4. Setup sperimentale e presentazione di un caso d'uso

Il presente capitolo descrive il setup sperimentale adottato per valutare e confrontare due differenti strategie di adattamento dei Large Language Models a un dominio applicativo specifico, ovvero l'adattamento parametrico mediante tecnica LoRA (Low-Rank Adaptation) e l'integrazione di conoscenza esterna tramite una pipeline di Retrieval-Augmented Generation (RAG). Dopo aver introdotto nei capitoli precedenti i fondamenti teorici di tali approcci e aver descritto l'architettura della piattaforma Synapsis ML, l'obiettivo è ora analizzarne il comportamento in un contesto applicativo concreto. In particolare, la pipeline RAG implementata all'interno di Synapsis ML viene confrontata con un modello adattato tramite LoRA, addestrato separatamente in ambiente Colab al fine di sfruttare risorse computazionali adeguate.

Il confronto è definito come un esperimento controllato, LoRA e RAG operano a parità di condizioni, utilizzando lo stesso modello base, gli stessi prompt e gli stessi parametri di generazione. Questo consente di isolare l'effetto della strategia di adattamento, permettendo di analizzare le differenze tra conoscenza appresa nei pesi del modello e conoscenza recuperata dinamicamente tramite retrieval. L'analisi delle prestazioni viene effettuata mediante un insieme di metriche di valutazione, sia automatiche che qualitative, con l'obiettivo di giudicare l'efficacia dei due approcci nel contesto applicativo considerato.

Viene innanzitutto introdotto il caso d'uso considerato, insieme al dominio applicativo di riferimento. Successivamente vengono descritte le scelte relative al modello adottato e il setup sperimentale comune ai due approcci, includendo la definizione dei prompt e dei parametri di generazione utilizzati. Si passa quindi alla descrizione delle due strategie di adattamento considerate, da un lato l'adattamento parametrico tramite LoRA, dall'altro l'integrazione di conoscenza esterna mediante pipeline RAG. Per ciascun approccio vengono illustrate le modalità di configurazione e utilizzo nel contesto dell'esperimento. Infine, viene presentata la fase di generazione e valutazione, in cui vengono riportati i risultati ottenuti dai due metodi.

4.1 Dominio di applicazione

Il presente lavoro si inserisce nell'ambito dei sistemi di question answering applicati a domini specifici. Il caso considerato riguarda lo sviluppo di un chatbot aziendale per il supporto clienti di un negozio specializzato nella vendita di macchine da caffè e prodotti

correlati. Il dominio è caratterizzato da una conoscenza fortemente orientata al prodotto e relativamente strutturata. Le informazioni rilevanti comprendono le caratteristiche delle macchine da caffè, le specifiche tecniche, la compatibilità tra dispositivi e consumabili (come capsule e cialde), oltre agli aspetti legati all'utilizzo e alla manutenzione. Questi contenuti sono generalmente distribuiti tra diverse fonti, tra cui schede prodotto, documentazione tecnica e materiali informativi destinati agli utenti.

Il sistema ha l'obiettivo di fornire risposte accurate e pertinenti, supportando l'utente nella comprensione dei prodotti, nel confronto tra alternative disponibili e nella risoluzione dei dubbi più comuni. La gestione della compatibilità tra prodotti rappresenta un aspetto particolarmente delicato, poiché richiede informazioni precise e costantemente aggiornate. La qualità delle risposte assume quindi un ruolo centrale. Contenuti incompleti o errati possono compromettere l'esperienza dell'utente e portare a decisioni di acquisto non adeguate. È necessario che il sistema mantenga coerenza con il dominio e garantisca affidabilità dal punto di vista informativo.

Questo scenario si presta all'analisi delle tecniche di adattamento dei Large Language Models. Il dominio è sufficientemente circoscritto da permettere una specializzazione mirata, ma presenta al tempo stesso la necessità di gestire informazioni aggiornate e specifiche, rendendolo adatto al confronto tra approcci basati su conoscenza parametrica, come LoRA, e soluzioni che integrano conoscenza esterna, come la Retrieval-Augmented Generation.

4.2 Scelta del modello

Per entrambi gli approcci considerati è stato utilizzato il modello meta-llama/Llama-3.1-8B-Instruct, appartenente alla famiglia LLaMA sviluppata da Meta. L'adozione di un modello comune costituisce un elemento fondamentale del disegno sperimentale, in quanto consente di isolare l'effetto delle diverse strategie di adattamento (LoRA e RAG), mantenendo invariata la componente generativa di base.

La scelta di questo modello è guidata da diverse considerazioni. In primo luogo, si tratta di un modello di dimensione intermedia (8 miliardi di parametri), che rappresenta un buon compromesso tra capacità espressive e requisiti computazionali. Modelli di dimensioni inferiori potrebbero non essere sufficientemente performanti nel gestire richieste articolate, mentre modelli più grandi comporterebbero costi e requisiti hardware significativamente maggiori. In secondo luogo, la variante Instruct è specificamente ottimizzata per seguire istruzioni in linguaggio naturale, risultando particolarmente adatta a contesti conversazionali e sistemi di question answering. Ciò risulta particolarmente rilevante nel caso d'uso considerato, in cui il modello deve interagire con utenti finali fornendo rispo-

ste chiare, coerenti e pertinenti rispetto al dominio applicativo. Infine, la scelta è stata influenzata anche da considerazioni legate alla disponibilità del modello all'interno degli strumenti utilizzati e alla possibilità di integrarlo in contesti differenti, sia in fase di addestramento sia in fase di inferenza.

4.2.1 Utilizzo del modello in ambiente di addestramento (LoRA)

Per l'adattamento parametrico tramite LoRA, il modello è stato utilizzato in ambiente Google Colab, sfruttando le risorse GPU disponibili per eseguire il processo di fine-tuning. Il modello è stato caricato tramite la piattaforma Hugging Face, che fornisce accesso ai pesi pre-addestrati previa accettazione delle condizioni di utilizzo del repository ufficiale Meta. Una volta ottenuto l'accesso, il modello è stato integrato all'interno della pipeline di addestramento utilizzando le librerie *transformers* e *peft*, che consentono di applicare la tecnica LoRA in modo efficiente.

L'utilizzo di Google Colab risponde a esigenze di accessibilità e riduzione dei costi, permettendo di eseguire esperimenti su hardware accelerato senza la necessità di disporre di infrastrutture locali dedicate. Tuttavia, tale ambiente introduce alcuni vincoli, tra cui la disponibilità limitata di memoria GPU, la durata delle sessioni e la necessità di ottimizzare attentamente i parametri di addestramento. Questa configurazione risulta comunque adeguata per l'applicazione di tecniche di Parameter-Efficient Fine-Tuning, come LoRA, che richiedono un numero ridotto di parametri addestrabili rispetto al fine-tuning completo.

4.2.2 Utilizzo del modello in fase di inferenza (RAG in Synopsis ML)

Nel contesto della piattaforma Synopsis ML, il modello è stato utilizzato esclusivamente in fase di inferenza, all'interno della pipeline RAG, tramite l'integrazione con provider esterni. L'utilizzo di un provider esterno è coerente con un contesto di sperimentazione privo di risorse hardware dedicate per l'esecuzione locale di modelli di grandi dimensioni. L'accesso a modelli tramite API rappresenta una soluzione pratica ed efficiente, ampiamente adottata in ambito applicativo. In questo caso, è stato impiegato il servizio offerto da Groq, che consente l'accesso a modelli della famiglia LLaMA attraverso API ad alte prestazioni. Questo approccio permette di delegare l'esecuzione del modello a un'infrastruttura esterna ottimizzata, eliminando la necessità di gestire direttamente risorse hardware dedicate.

L'utilizzo di Groq presenta diversi vantaggi tra cui tempi di risposta ridotti, semplicità di integrazione e assenza di costi infrastrutturali diretti nella fase di sperimentazione. Tuttavia, il servizio gratuito è soggetto a limitazioni, tra cui vincoli sul numero di richieste effettuabili, sul throughput e sulla lunghezza massima delle richieste gestibili. Tali limitazioni rendono necessario progettare con attenzione le chiamate al modello, in particolare

nel contesto di pipeline RAG che possono richiedere più interazioni con l’LLM. Nonostante questi vincoli, l’approccio basato su API risulta particolarmente adatto per scenari applicativi reali e per l’integrazione all’interno di architetture software complesse, come quella di Synapsis ML.

4.3 Adattamento tramite LoRA

L’adattamento al dominio applicativo è stato realizzato creando un adapter LoRA a partire dal modello base. In questo modo, durante il training vengono aggiornati esclusivamente i parametri introdotti dall’adapter, mentre i pesi originali del modello rimangono invariati.

L’intervento è stato applicato ai layer di attenzione del modello, consentendo di modificarne il comportamento rispetto alle richieste del dominio senza alterarne la struttura complessiva. Questo approccio permette di ottenere una specializzazione mirata, limitando il numero di parametri addestrabili e mantenendo inalterate le capacità generali del modello pre-addestrato.

4.3.1 Dataset di addestramento

Il dataset utilizzato per l’addestramento è composto da 720 esempi ed è stato suddiviso in training set (518 esempi), validation set (58 esempi) e test set (144 esempi). I dati sono stati costruiti sinteticamente a partire da documentazione tecnica e manuali relativi a macchine da caffè e prodotti correlati. Le informazioni sono state riorganizzate sotto forma di coppie domanda-risposta, con l’obiettivo di simulare interazioni realistiche tra utente e assistente.

Ogni istanza del dataset è inizialmente rappresentata in forma strutturata attraverso un insieme di campi che descrivono il prodotto e la richiesta dell’utente. La Tabella 4.1 riporta un esempio rappresentativo nel formato originale.

brand	prodotto	categoria	domanda	risposta
Spinel	Pinocchio	problema	Perché dalla Pinocchio esce poco caffè?	Controllare che il serbatoio sia pieno, che l’erogatore non sia ostruito e che la capsula o la cialda sia inserita correttamente nella Pinocchio.

Tabella 4.1: Esempio di istanza del dataset nel formato strutturato

Al fine di renderlo compatibile con il modello, ogni esempio è stato trasformato in formato instruction-following, integrando le informazioni strutturate all’interno di un prompt testuale. Un esempio della rappresentazione finale è riportato di seguito:

Prompt:

Sei un assistente esperto di macchine da caffè e manutenzione.

Brand: Spinel

Prodotto: Pinocchio

Categoria: problema

Domanda: Perché dalla Pinocchio esce poco caffè?

Risposta:

Risposta:

Controllare che il serbatoio sia pieno, che l'erogatore non sia ostruito e che la capsula o la cialda sia inserita correttamente nella Pinocchio.

Durante la fase di preprocessing, prompt e risposta vengono concatenati in un'unica sequenza. La funzione di loss viene tuttavia calcolata esclusivamente sui token della risposta, mentre i token del prompt vengono mascherati. In questo modo, il prompt viene utilizzato come contesto per la generazione dell'output, mentre l'ottimizzazione dei parametri è guidata unicamente dalla qualità della risposta prodotta rispetto al target.

4.3.2 Training dell'adapter LoRA

Il training dell'adapter LoRA è stato effettuato utilizzando il dataset descritto nella sezione precedente, con l'obiettivo di specializzare il modello rispetto alle tipologie di richieste presenti nel dominio applicativo.

La fase di addestramento è stata configurata tenendo conto sia delle dimensioni contenute del dataset sia dei vincoli dell'ambiente di esecuzione. È stato necessario adottare, infatti, una configurazione che garantisse una buona convergenza, mantenendo al tempo stesso contenuto il consumo di memoria.

Per rendere il training compatibile con le risorse disponibili, è stata adottata una strategia di quantizzazione a 4 bit tramite la libreria *bitsandbytes*. Il modello viene quindi caricato in formato quantizzato, riducendo significativamente il consumo di memoria rispetto alla rappresentazione a 16 o 32 bit. Questa configurazione consente di eseguire il fine-tuning di modelli di grandi dimensioni anche in ambienti con risorse limitate, mantenendo prestazioni adeguate.

Configurazione LoRA e quantizzazione

```
1 bnb_config = BitsAndBytesConfig(  
2     load_in_4bit=True,  
3     bnb_4bit_compute_dtype=torch.float16,  
4     bnb_4bit_quant_type="nf4",  
5     bnb_4bit_use_double_quant=True,  
6 )  
7  
8 lora_config = LoraConfig(  
9     r=8,  
10    lora_alpha=32,  
11    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],  
12    lora_dropout=0.05,  
13    bias="none",  
14    task_type=TaskType.CAUSAL_LM,  
15 )
```

L'intervento sui moduli di proiezione delle query, key, value e output consente di influenzare direttamente il meccanismo di attenzione, che rappresenta il componente principale nella modellazione delle dipendenze tra token. Inoltre, l'utilizzo di LoRA comporta una significativa riduzione del numero di parametri addestrabili rispetto al fine-tuning completo, rendendo il processo più efficiente sia in termini di memoria sia di tempo di addestramento.

4.3.2.1 Configurazione del training

L'addestramento è stato eseguito per un totale di 5 epoche, utilizzando un learning rate pari a 1×10^{-4} e batch size pari a 1.

Parametri di training

```
1 training_args = TrainingArguments(  
2     output_dir="./chatbot_lora",  
3     num_train_epochs=5,  
4     per_device_train_batch_size=1,  
5     per_device_eval_batch_size=1,  
6     learning_rate=1e-4,  
7     logging_strategy="epoch",  
8     eval_strategy="epoch",  
9     fp16=True,  
10    save_total_limit=2  
11 )
```

La scelta del numero di epoche è stata effettuata sulla base di prove preliminari. Configurazioni con un numero maggiore di epoche (ad esempio 10) hanno mostrato una convergenza molto rapida già nelle prime iterazioni, seguita da una fase in cui la loss rimaneva prossima allo zero. Considerata la dimensione ridotta del dataset, tale comportamento è indicativo di una possibile tendenza alla memorizzazione, rendendo quindi preferibile limitare il numero di epoche a 5.

Il learning rate è stato selezionato in modo da garantire una convergenza stabile. Valori più elevati tendevano a introdurre instabilità durante il training, mentre valori più bassi rallentavano significativamente il processo di apprendimento.

La valutazione sul validation set è stata effettuata al termine di ogni epoca, consentendo di monitorare l'andamento delle metriche e di individuare eventuali segnali di overfitting. La lunghezza massima della sequenza è stata impostata a 512 token, valore sufficiente a contenere sia il prompt sia la risposta nella maggior parte dei casi. L'utilizzo di batch size pari a 1 è invece motivato dai vincoli di memoria dell'ambiente di esecuzione, risultando comunque adeguato nel contesto del fine-tuning con LoRA.

4.3.2.2 Valutazione del training

L'andamento della loss durante il training evidenzia una rapida convergenza del modello già nelle prime epoche. In particolare, la training loss passa da valori iniziali pari a 0.34 a valori prossimi allo zero dopo poche iterazioni, mentre la validation loss segue un andamento analogo.

La Figura 4.1 mostra come entrambe le curve decrescano rapidamente e tendano a stabilizzarsi a partire dalla terza epoca.

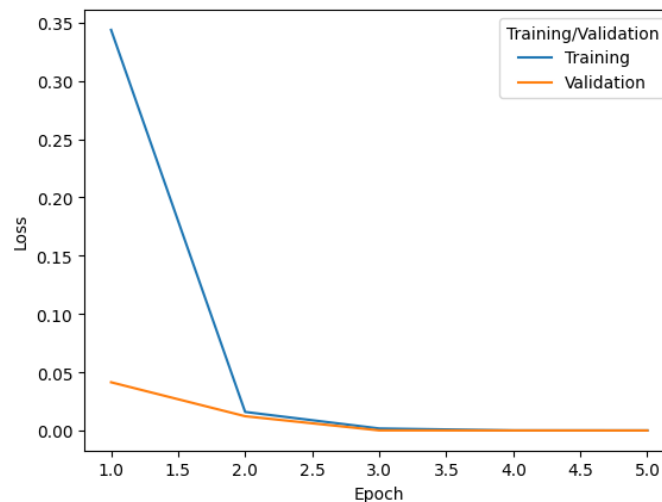


Figura 4.1: Andamento della training loss e validation loss durante il training LoRA

Oltre alla loss, sono state monitorate anche diverse metriche di valutazione sul validation set, riportate in Figura 4.2.

Le metriche osservate nel corso delle epoche mostrano un comportamento complessivamente stabile, suggerendo che il modello raggiunge rapidamente una fase di convergenza. In particolare, i valori di ROUGE-L e F1 si attestano rispettivamente intorno a 0.67 e tra 0.62 e 0.64, indicando una buona sovrapposizione tra le risposte generate e

Epoch	Training Loss	Validation Loss	Rouge1	Exact Match	F1	Bleu
1	0.343745	0.041457	0.677154	0.000000	0.643717	0.467272
2	0.015987	0.012313	0.671178	0.000000	0.626634	0.478852
3	0.001805	0.000165	0.675039	0.000000	0.632403	0.484000
4	0.000132	0.000118	0.675039	0.000000	0.632564	0.483867
5	0.000105	0.000106	0.675039	0.000000	0.632704	0.483867

Figura 4.2: Metriche di valutazione calcolate durante il training LoRA

quelle di riferimento. Tali risultati suggeriscono che il modello è in grado di catturare in modo efficace le informazioni rilevanti, producendo risposte coerenti dal punto di vista semantico.

Al contrario, il punteggio BLEU risulta più contenuto. Questo comportamento è coerente con la natura del task, trattandosi di generazione libera, risposte corrette possono differire significativamente nella formulazione lessicale rispetto alla ground truth, pur mantenendo lo stesso significato. Metriche come BLEU, basate su corrispondenze n-gram, tendono quindi a penalizzare tali variazioni, risultando meno rappresentative della reale qualità dell'output.

Un'osservazione analoga vale per la metrica di Exact Match, che assume valore nullo in tutte le epoche. Anche in questo caso, il risultato non deve essere interpretato come indice di scarsa qualità, bensì come conseguenza della natura generativa del modello. La probabilità che una risposta coincida esattamente, parola per parola, con il riferimento è estremamente bassa, anche quando il contenuto informativo è corretto.

Nel complesso, l'andamento delle metriche suggerisce che il modello apprende rapidamente le strutture tipiche delle risposte nel dominio considerato, raggiungendo già nelle prime epoche un livello di prestazione stabile. Tuttavia, la combinazione di valori di loss molto bassi e di metriche che si stabilizzano precocemente può essere indicativa di una limitata complessità del dataset o di una possibile tendenza alla memorizzazione degli esempi di training. Per questo motivo, la sola analisi quantitativa non è sufficiente a valutare pienamente le capacità del modello. Diventa quindi fondamentale affiancare a queste metriche una valutazione su dati non visti e un'analisi qualitativa delle risposte generate, al fine di verificare la reale capacità di generalizzazione e l'effettiva utilità del modello nel contesto applicativo.

4.4 Fase di inferenza

Completata la fase di adattamento del modello, il confronto tra i due approcci viene condotto in fase di inferenza, analizzando il comportamento del modello adattato tramite LoRA e della pipeline RAG nella generazione delle risposte.

A differenza della fase di training, in cui il modello viene ottimizzato rispetto a un insieme di esempi, la fase di inferenza rappresenta il momento in cui le capacità apprese vengono effettivamente messe alla prova. Diventa dunque possibile osservare come le due strategie di adattamento influenzino il processo generativo, sia in termini di qualità delle risposte sia rispetto alla gestione della conoscenza.

Per garantire un confronto metodologicamente corretto, sono state mantenute invariate tutte le condizioni operative comuni ai due approcci. In particolare, sono stati utilizzati lo stesso prompt, la stessa configurazione dei parametri di generazione e lo stesso dataset di valutazione. Questa scelta consente di isolare l'effetto della strategia di adattamento, riducendo al minimo le variabili confondenti e rendendo le differenze osservate direttamente attribuibili al modo in cui la conoscenza viene incorporata nel sistema.

In questo modo, il confronto non riguarda semplicemente le prestazioni assolute dei due approcci, ma permette di analizzare in modo più fine le loro caratteristiche distintive ed eventuali differenze nelle risposte prodotte possono essere ricondotte principalmente alla diversa strategia di adattamento adottata.

4.4.1 Prompt e parametri di generazione

Nel confronto tra LoRA e RAG, il prompt e la configurazione dei parametri di generazione svolgono un ruolo fondamentale, in quanto definiscono il contesto e le modalità con cui il modello produce le risposte.

Entrambi gli elementi influenzano direttamente il comportamento del modello, il prompt agisce come guida semantica e comportamentale, mentre i parametri di generazione regolano il grado di determinismo, variabilità e lunghezza dell'output. Una configurazione non controllata potrebbe introdurre differenze significative nelle risposte, rendendo difficile attribuire le variazioni osservate alla sola strategia di adattamento.

Per questo motivo, è stata adottata una configurazione comune ai due approcci, mantenendo invariati sia il prompt sia i parametri di generazione. Questa scelta consente di ridurre la variabilità introdotta dal processo generativo e di focalizzare l'analisi sulle reali differenze tra adattamento parametrico (LoRA) e integrazione di conoscenza esterna (RAG).

4.4.1.1 Definizione del prompt

Il comportamento del modello è stato guidato tramite un prompt progettato per vincolare la generazione al dominio applicativo e garantire risposte coerenti con il contesto aziendale considerato.

Il prompt definisce il ruolo del modello come assistente specializzato nel dominio delle macchine da caffè e dei prodotti collegati, specificando in modo esplicito le regole di comportamento da rispettare. In particolare, il modello viene istruito a: rispondere esclusivamente su temi pertinenti al dominio; fornire risposte corrette, chiare e sintetiche; evitare l'invenzione di dettagli tecnici o informazioni non supportate; dichiarare esplicitamente eventuali limiti informativi; mantenere un tono professionale, utile e diretto.

È importante sottolineare che il testo del prompt è stato mantenuto identico per entrambi gli approcci considerati (LoRA e RAG), al fine di garantire condizioni di confronto controllate. In questo modo, eventuali differenze nel comportamento del modello possono essere attribuite alla strategia di adattamento adottata, e non a variazioni nella formulazione delle istruzioni. Il testo del prompt utilizzato è riportato di seguito.

System prompt
Sei un assistente aziendale specializzato in macchine del caffè e prodotti collegati alle macchine del caffè. Regole di comportamento: 1. Rispondi solo su temi legati a macchine del caffè, accessori, ricambi, capsule, cialde, detergenti, decalcificanti, manutenzione, pulizia, compatibilità, uso corretto e informazioni di prodotto. 2. Fornisci risposte corrette, chiare e sintetiche. 3. Non inventare dettagli tecnici, compatibilità, prezzi, disponibilità, garanzie o caratteristiche non presenti nella conoscenza appresa dal modello. 4. Se non hai informazioni sufficienti, dillo esplicitamente con una formula breve, per esempio: "Non ho abbastanza informazioni per confermarlo." 5. Se la domanda è ambigua, interpreta nel modo più plausibile senza fare domande di follow-up e specifica eventuali limiti della risposta. 6. Non uscire dal dominio aziendale. 7. Mantieni un tono professionale, utile e diretto. 8. Non aggiungere informazioni superflue. 9. Quando opportuno, struttura la risposta in 1-3 frasi brevi oppure in elenco puntato breve. 10. Non citare di essere un modello linguistico o un'IA. Obiettivo: Fornire la migliore risposta possibile a una domanda di question answering nel dominio delle macchine del caffè e dei relativi prodotti.

Dal punto di vista operativo, la costruzione dell'input presenta differenze tra i due approcci considerati, pur mantenendo invariato il contenuto del system prompt e la struttura generale della richiesta.

Nel caso del modello adattato tramite LoRA, la costruzione del prompt avviene in mo-

do esplicito e centralizzato. Il system prompt e la domanda dell'utente vengono combinati direttamente all'interno di una struttura conversazionale che distingue i ruoli di sistema, utente e assistente. Per ogni richiesta, la domanda viene quindi inserita nel prompt finale passato al modello, secondo uno schema deterministico definito a priori.

La costruzione dell'input in questo caso segue lo schema riportato di seguito.

```
Costruzione dell'input
1 def build_prompt(question):
2     return f"""<s>[SYSTEM]
3 {SYSTEM_PROMPT}
4 [/SYSTEM]
5
6 [USER]
7 Domanda: {question}
8 [/USER]
9
10 [ASSISTANT]
11 """
```

In questo contesto, il prompt viene costruito in un unico punto della pipeline e rappresenta l'input diretto al modello, senza ulteriori trasformazioni intermedie.

Nel caso della pipeline RAG, invece, la costruzione dell'input avviene in modo distribuito lungo le diverse fasi del processo. Come descritto nel Capitolo 3, il sistema utilizza un contesto condiviso che include il system prompt (identico a quello utilizzato nel caso LoRA), la richiesta dell'utente, la cronologia conversazionale e le eventuali sorgenti informative esterne.

Anche in questo caso la domanda dell'utente viene integrata nel processo di generazione, tuttavia, essa non viene combinata una sola volta in un prompt finale, ma viene utilizzata in più fasi della pipeline. I diversi componenti del sistema, come il routing della richiesta, le capability specializzate e il modulo di aggregazione, costruiscono e utilizzano prompt distinti, ciascuno adattato al compito specifico.

Di conseguenza, la differenza tra i due approcci non risiede nella presenza o meno del prompt, né nell'integrazione della domanda dell'utente, ma nella modalità con cui il prompt viene costruito e utilizzato. Nel caso LoRA, il prompt è definito in modo centralizzato e utilizzato direttamente dal modello, nella pipeline RAG, invece, esso emerge da un processo di composizione dinamica distribuito tra più componenti.

La progettazione del prompt ha avuto un ruolo importante nel controllo del comportamento del modello. La presenza di istruzioni esplicite contribuisce infatti a limitare il rischio di allucinazioni, a mantenere le risposte all'interno del dominio applicativo e a garantire uno stile coerente con quello di un assistente aziendale.

4.4.1.2 Parametri di generazione

I parametri di generazione influenzano direttamente il comportamento del modello in fase di inferenza, determinando aspetti quali il grado di variabilità delle risposte, la loro lunghezza e la tendenza alla ripetizione. Anche a parità di modello e prompt, configurazioni diverse possono produrre output significativamente differenti, rendendo questa fase particolarmente rilevante in un contesto applicativo.

Nel caso in esame, l'obiettivo non è quello di massimizzare la creatività del modello, ma ottenere risposte affidabili, coerenti con il dominio e facilmente leggibili. Per questo motivo, la scelta dei parametri è stata orientata verso una configurazione controllata e stabile. Al fine di garantire un confronto metodologicamente corretto tra LoRA e RAG, la configurazione è stata mantenuta identica per entrambi gli approcci ed è riportata di seguito.

Configurazione dei parametri di generazione

```
1 GEN_CONFIG = {  
2     "temperature": 0.3,  
3     "max_tokens": 256,  
4     "max_new_tokens": 256,  
5     "top_p": 0.9,  
6     "frequency_penalty": 0.2,  
7     "presence_penalty": 0.1,  
8     "do_sample": True,  
9 }
```

La configurazione è stata definita attraverso una serie di prove preliminari, valutando qualitativamente le risposte generate in termini di chiarezza, pertinenza e stabilità.

In particolare, una temperatura pari a 0.3 si è dimostrata adeguata per mantenere il comportamento del modello sufficientemente controllato. Valori più elevati introducevano una maggiore variabilità, con risposte talvolta meno coerenti con il dominio, mentre valori troppo bassi portavano a output eccessivamente rigidi e ripetitivi.

Il parametro `top_p` è stato impostato a 0.9 per limitare la generazione ai token più probabili, mantenendo comunque una certa flessibilità espressiva. Durante le prove, valori più restrittivi risultavano troppo conservativi, mentre valori più elevati non apportavano benefici significativi.

Il limite di 256 token è stato scelto per garantire risposte sufficientemente complete senza risultare troppo lunghe o dispersive. Nel contesto considerato, infatti, risposte eccessivamente verbose riducono la leggibilità e l'utilità per l'utente finale.

L'introduzione di `frequency_penalty` e `presence_penalty` è stata motivata dall'esigenza di contenere ripetizioni indesiderate osservate in alcune configurazioni iniziali, migliorando la qualità complessiva dell'output senza comprometterne la coerenza.

Infine, l’attivazione di `do_sample` è stata mantenuta per preservare la naturalezza della generazione, pur all’interno di una configurazione complessivamente conservativa.

Nel complesso, la configurazione adottata rappresenta un buon compromesso tra controllo e flessibilità, risultando adeguata al caso d’uso considerato. Il mantenimento degli stessi parametri per entrambi gli approcci consente inoltre di isolare l’effetto della strategia di adattamento nel confronto tra LoRA e RAG.

4.4.2 Modalità di inferenza

La fase di inferenza rappresenta il momento in cui i due approcci vengono messi concretamente a confronto, analizzando il comportamento del modello adattato tramite LoRA e della pipeline RAG nella generazione delle risposte.

L’obiettivo è valutare come le due strategie di adattamento influenzino il processo generativo a parità di condizioni operative. Per questo motivo, sono stati mantenuti invariati il prompt, i parametri di generazione e l’insieme di domande utilizzate, in modo da isolare l’effetto della modalità con cui la conoscenza viene incorporata o recuperata.

La procedura sperimentale si articola quindi in due fasi principali: la costruzione del test set e la generazione delle risposte tramite i due approcci, successivamente utilizzate per la valutazione.

4.4.2.1 Costruzione del test set

Per la fase di valutazione è stato costruito un insieme di domande a partire da forum, discussioni online e sezioni di domande e risposte relative a macchine da caffè e problematiche d’uso reali. Questa scelta è stata effettuata con l’obiettivo di ottenere richieste il più possibile rappresentative di scenari applicativi concreti, caratterizzati da formulazioni naturali, talvolta ambigue o incomplete. Le domande raccolte sono state successivamente riorganizzate e associate a una risposta di riferimento, costruita sulla base delle informazioni disponibili nelle fonti consultate. Il dataset risultante è quindi composto da coppie domanda–risposta, dove la risposta rappresenta la ground truth utilizzata nella fase di valutazione.

Durante l’inferenza le risposte di riferimento non vengono fornite al modello, ma sono utilizzate esclusivamente in un secondo momento per confrontare gli output generati. Inoltre, il test set è stato mantenuto identico per entrambi gli approcci considerati, garantendo condizioni di confronto uniformi. L’utilizzo di dati derivati da fonti reali e non presenti nel dataset di training consente di valutare la capacità di generalizzazione dei modelli e la loro efficacia in condizioni più vicine a un contesto applicativo reale.

Alcuni esempi di istanze presenti nel test set sono riportati nella Tabella 4.2.

Domanda	Risposta gold
Cosa controllare se la X7.1 non eroga nulla al primo tentativo?	Controlla che il serbatoio sia inserito correttamente, che il circuito sia adescato e che non ci siano ostruzioni nel gruppo erogatore.
Perché il flusso della macchina esce a scatti invece che regolare?	Il flusso irregolare può indicare aria nel circuito o ostruzioni: è consigliabile eseguire un risciacquo e verificare la pulizia dell'erogatore.
Perché la macchina continua a gocciolare dopo aver fatto il caffè?	Un leggero gocciolamento è normale, ma se persistente può essere utile pulire l'erogatore e verificare il corretto inserimento della capsula.

Tabella 4.2: Esempi di domande e risposte gold presenti nel test set

4.4.2.2 Procedura di inferenza

La fase di inferenza è stata condotta sottoponendo le medesime domande del test set sia al modello adattato tramite LoRA sia alla pipeline RAG, mantenendo invariati il prompt e i parametri di generazione. Tale impostazione consente un confronto diretto tra i due approcci, garantendo condizioni sperimentali omogenee.

Per ciascuna istanza sono state quindi generate due risposte, una per ogni approccio, successivamente confrontate con la corrispondente risposta di riferimento presente nel dataset, utilizzata come ground truth nella fase di valutazione.

Nel caso del modello LoRA, l'inferenza è stata eseguita in un ambiente separato rispetto a quello utilizzato per il training. Al termine dell'addestramento, l'adapter è stato salvato e successivamente ricaricato insieme al modello base. Il test set è stato quindi utilizzato considerando esclusivamente le domande, escludendo le risposte di riferimento dal processo di generazione. Per ciascun input è stato costruito il prompt secondo lo schema definito, e il modello ha generato la risposta sfruttando i pesi adattati.

Nel caso della pipeline RAG, l'inferenza è stata effettuata tramite l'interfaccia della piattaforma Synapsis ML. Le stesse domande sono state inserite manualmente nel sistema, una alla volta, registrando le risposte prodotte. In questo contesto, l'output del modello è il risultato di un processo che integra la generazione con il recupero dinamico di informazioni da fonti esterne.

Al termine della fase di inferenza, è stato costruito un dataset di confronto contenente, per ciascuna istanza, la domanda, le risposte generate dai due approcci e la risposta di riferimento. Tale dataset è stato quindi utilizzato per la successiva fase di valutazione.

4.5 Valutazione

La fase di valutazione ha l'obiettivo di fornire una lettura strutturata e approfondita delle risposte generate dai due approcci considerati, analizzandone il comportamento attraverso una combinazione di analisi quantitativa e qualitativa. L'interesse non è limitato alla sola quantificazione delle prestazioni, ma si estende alla comprensione delle modalità con cui il contenuto informativo viene costruito, organizzato ed espresso dal modello.

Nel task di question answering generativo, la qualità delle risposte non è riconducibile a una singola misura numerica. Risposte semanticamente corrette possono differire in modo significativo dal punto di vista lessicale, sintattico e strutturale, rendendo necessario adottare criteri di analisi capaci di cogliere sia il contenuto informativo sia le modalità espressive. La relazione tra forma e significato rappresenta quindi un elemento centrale nella valutazione, in quanto variazioni nella formulazione non implicano necessariamente una perdita di correttezza o pertinenza.

L'analisi si basa sul confronto tra le risposte generate e le corrispondenti risposte di riferimento, utilizzate come ground truth. Questo confronto consente di valutare il grado di aderenza al contenuto atteso, ma anche di osservare fenomeni quali la parafrasi, la sintesi dell'informazione e la riorganizzazione del contenuto. Inoltre, permette di analizzare la capacità del modello di mantenere coerenza rispetto al dominio applicativo, evitando deviazioni tematiche o introduzione di informazioni non supportate.

Un ulteriore aspetto rilevante riguarda la variabilità delle risposte. Anche in presenza dello stesso input, il processo generativo può produrre formulazioni differenti, influenzate dai parametri di generazione e dalla natura probabilistica del modello. Tale variabilità rende necessario considerare non solo la correttezza puntuale delle singole risposte, ma anche la stabilità del comportamento del modello su un insieme di istanze.

La valutazione è articolata secondo due metodologie complementari. Da un lato, una valutazione automatica basata su metriche quantitative permette di misurare in modo sistematico la similarità tra le risposte generate e quelle di riferimento. Questo approccio consente di ottenere una visione aggregata delle prestazioni, evidenziando pattern ricorrenti e differenze tra i due sistemi in termini di distribuzione dei punteggi. Le metriche utilizzate permettono di osservare il comportamento del modello da diverse prospettive, includendo sia aspetti lessicali sia semantici.

Dall'altro lato, viene adottato un approccio basato su un modello linguistico come giudice (*LLM as a judge*), finalizzato a valutare le risposte secondo criteri più vicini al giudizio umano, quali correttezza, completezza, chiarezza e adeguatezza rispetto al contesto della domanda. Questo tipo di valutazione introduce un livello interpretativo che consente di cogliere relazioni semantiche, implicazioni contestuali e sfumature linguistiche

difficilmente rappresentabili attraverso metriche automatiche.

Le metriche quantitative forniscono una misura oggettiva e replicabile, ma risultano sensibili alla formulazione testuale e possono penalizzare risposte semanticamente equivalenti ma lessicalmente differenti. Al contrario, la valutazione tramite LLM consente di considerare la qualità complessiva della risposta in modo più flessibile, includendo aspetti legati alla comprensibilità e all'utilità per l'utente finale.

L'integrazione delle due metodologie consente quindi di analizzare il comportamento dei modelli su più livelli. È possibile identificare tendenze generali attraverso l'analisi aggregata delle metriche e approfondire casi specifici mediante una valutazione qualitativa delle risposte. Questa combinazione fornisce una base metodologica solida per l'interpretazione dei risultati e per la successiva discussione delle differenze tra gli approcci considerati.

4.5.1 Valutazione automatica

La valutazione automatica è stata condotta confrontando, per ciascuna domanda del test set, la risposta generata con la corrispondente risposta di riferimento.

Per ogni istanza è stato costruito un record contenente la domanda, la risposta prodotta dal modello e la risposta gold, sul quale sono state calcolate le metriche di valutazione. Questo ha permesso di ottenere sia una misura aggregata delle prestazioni, attraverso il calcolo dei valori medi, sia una base per l'analisi puntuale di singoli casi.

La procedura è stata applicata in modo identico ai due approcci considerati, consentendo un confronto diretto dei risultati in condizioni omogenee.

4.5.1.1 Metriche di valutazione

Data la natura del task, caratterizzato da generazione libera di testo, è stato necessario adottare un insieme articolato di metriche, in grado di catturare diversi aspetti della qualità delle risposte.

Le metriche basate su sovrapposizione lessicale (BLEU, ROUGE) consentono di misurare la similarità superficiale tra le sequenze, risultando però sensibili alla formulazione testuale. Al contrario, metriche come BERTScore permettono di valutare la similarità semantica tra testi, risultando più robuste in presenza di parafrasi. Sono state inoltre considerate metriche complementari (METEOR, chrF) e metriche più restrittive (Exact Match, Token F1), utili per fornire ulteriori punti di osservazione sul comportamento del modello.

4.5.1.2 Modello adattato tramite LoRA

Per il modello adattato tramite LoRA, la valutazione automatica è stata effettuata sulle 100 domande del test set, confrontando per ciascuna istanza la risposta generata dal modello con la corrispondente risposta gold. Le metriche medie ottenute sono riportate di seguito.

	metrica	valore
0	num_examples	100.000000
1	bleu	0.011615
2	rouge1	0.168344
3	rouge2	0.027097
4	rougeL	0.123994
5	rougeLsum	0.123310
6	bertscore_precision_mean	0.661597
7	bertscore_recall_mean	0.699013
8	bertscore_f1_mean	0.679637
9	meteor	0.151197
10	chrf	27.720460
11	exact_match	0.000000
12	token_f1	0.205564

Figura 4.3: Metriche medie ottenute dal modello LoRA sul test set

I valori mostrano una distinzione netta tra metriche lessicali e metriche semantiche. Le metriche basate sulla sovrapposizione esplicita dei token, come BLEU, ROUGE e Token F1, assumono valori contenuti. Questo andamento è tipico nei task di question answering generativo, dove una risposta può essere corretta anche quando non riproduce la stessa formulazione della risposta di riferimento. La presenza di sinonimi, riformulazioni o differenze nell'ordine delle informazioni tende infatti a ridurre le metriche di overlap, anche in presenza di contenuti semanticamente affini.

Il valore nullo di Exact Match conferma questa osservazione. Nessuna risposta generata coincide esattamente con la risposta gold, ma questo dato non è di per sé indicativo di un fallimento del modello. In un task generativo, soprattutto con risposte discorsive, la corrispondenza parola per parola rappresenta un criterio molto restrittivo e poco adatto a descrivere la qualità effettiva dell'output.

Più informativo è invece il comportamento di BERTScore. Il valore medio di BERTScore F1 è pari a 0.6796, con una mediana pari a 0.6774. La vicinanza tra media e mediana

suggerisce una distribuzione abbastanza equilibrata, non dominata da pochi casi estremi. Inoltre, il primo e il terzo quartile, rispettivamente pari a 0.6586 e 0.6999, indicano che una parte consistente delle risposte si concentra in un intervallo relativamente ristretto. Questo suggerisce che il modello mantiene una similarità semantica abbastanza stabile rispetto alle risposte di riferimento.

Per approfondire questa osservazione e analizzare come la similarità semantica si relazioni alla sovrapposizione lessicale, la Figura 4.4 mette in relazione i valori di BERTScore F1 e Token F1 per ciascuna risposta generata.

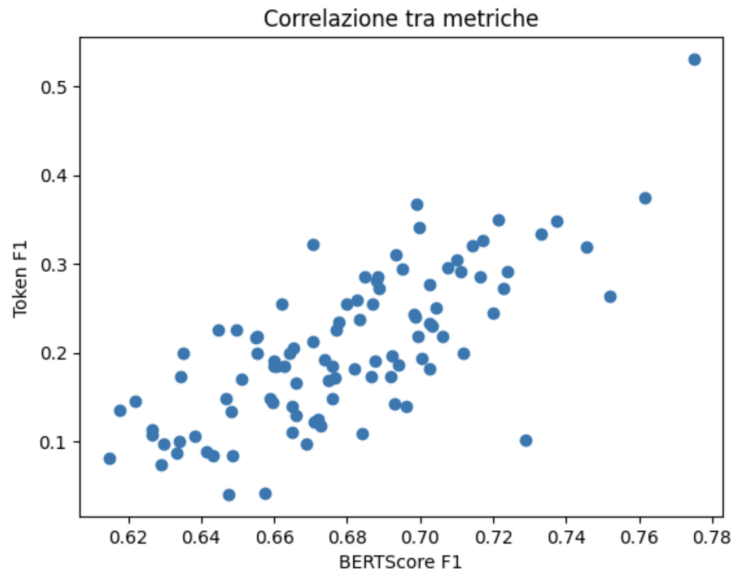


Figura 4.4: Relazione tra BERTScore F1 e Token F1 per le risposte generate dal modello LoRA

Il grafico evidenzia una correlazione positiva tra le due metriche: all'aumentare della similarità semantica tende ad aumentare anche la sovrapposizione lessicale. Tuttavia, la dispersione dei punti mostra chiaramente che le due misure non sono equivalenti. In particolare, si osservano numerosi casi in cui il BERTScore assume valori relativamente elevati mentre il Token F1 rimane contenuto. Questo indica che il modello riesce spesso a preservare il significato generale della risposta, pur utilizzando una formulazione diversa rispetto alla risposta gold.

La visualizzazione risulta quindi utile perché permette di cogliere una dinamica che non emerge immediatamente dai valori medi, ovvero la presenza di risposte semanticamente corrette ma lessicalmente diverse. Questo aspetto è centrale nel valutare modelli generativi, in quanto evidenzia come le metriche basate su overlap possano sottostimare la qualità effettiva delle risposte prodotte.

Per fornire una lettura più concreta dei risultati, la Tabella 4.3 riporta alcuni esempi rappresentativi, selezionati in modo da mostrare casi con diverso grado di aderenza alla risposta gold.

Domanda	Risposta LoRA	Risposta gold	BERTScore F1
Cosa va controllato prima di premere il tasto di erogazione sulla Y3.3?	Prima di erogare sulla Y3.3, verificare che la capsula o la cialda sia inserita correttamente e che il serbatoio acqua non sia vuoto.	Assicurati che ci sia acqua, che la tazza sia al posto giusto e che la capsula sia inserita correttamente.	0.77
Cosa controllare se la X7.1 non eroga nulla al primo tentativo?	Al primo tentativo di avvio della X7.1, controllare che la capsula o la cialda sia inserita correttamente, che l'erogatore non sia ostruito e che l'intero circuito non sia disidratato.	Controlla che il serbatoio sia inserito bene, che il circuito sia adescato e che nel gruppo non ci siano residui; poi prova un'erogazione senza capsula.	0.74
Perché dalla Jessica continuano a cadere gocce anche dopo aver finito il caffè?	Dalle gocce della Jessica potrebbero essere responsabili sia l'erogazione che il risciacquo. Controllare che non ci siano residui e eseguire un risciacquo finale.	Qualche goccia finale è normale, ma se il gocciolamento dura a lungo conviene pulire l'erogatore e controllare che la capsula sia stata perforata correttamente.	0.68
Come si esegue un risciacquo rapido tra due capsule diverse sulla X7.1?	Sulla X7.1 un risciacquo rapido tra due capsule diverse si esegue risciacquando la capsula usata con acqua pulita prima di inserire la nuova.	Tra una bevanda e l'altra fai passare per qualche secondo solo acqua, così elimini i residui del gusto precedente.	0.62
Come si pulisce la griglia appoggiata tazza della X7.1?	Per decalcificare la X9.2 usare un prodotto specifico per macchine del caffè, seguire il ciclo indicato dal sistema supporto della X9.2 e fare almeno un ciclo di risciacquo finale.	Rimuovi la griglia, lavala con acqua calda e detergente neutro, asciugala e reinseriscila solo quando è perfettamente pulita.	0.61

Tabella 4.3: Esempi rappresentativi di risposte generate dal modello LoRA

Gli esempi permettono di osservare più chiaramente il significato delle metriche. Nei casi con BERTScore più elevato, il modello recupera correttamente gli elementi centrali della risposta, anche quando la formulazione non coincide con quella della risposta gold. Ad esempio, nella domanda sulla Y3.3, la risposta generata menziona il controllo dell'acqua e dell'inserimento della capsula, elementi coerenti con il riferimento. La differenza principale riguarda alcune informazioni mancanti o riformulate, come la posizione della tazza.

Nei casi intermedi, il modello produce risposte plausibili e pertinenti al dominio, ma meno aderenti alla risposta attesa. La domanda relativa al gocciolamento della Jessica mostra che la risposta generata rimane nel campo della manutenzione e cita la presenza di residui e il risciacquo, ma non distingue chiaramente tra gocciolamento normale e gocciolamento persistente, aspetto invece presente nella risposta gold.

I casi con punteggio più basso evidenziano invece errori più marcati. Nella domanda sulla pulizia della griglia della X7.1, il modello genera una risposta relativa alla decalcificazione di un altro prodotto, la X9.2. In questo caso il testo mantiene una forma lin-

guisticamente coerente e appartiene ancora al dominio delle macchine da caffè, ma non risponde alla domanda posta. Questo tipo di errore è rilevante perché mostra un limite tipico dell'adattamento parametrico. Il modello può produrre risposte formalmente plausibili ma associate a prodotto, procedura o problema non corretti.

Questa analisi mostra che il modello LoRA tende a generare risposte coerenti con il dominio e spesso semanticamente vicine ai riferimenti, ma non sempre riesce a mantenere una corrispondenza precisa con il prodotto o con la procedura richiesta. Le metriche lessicali restituiscono valori bassi perché penalizzano fortemente le riformulazioni, mentre BERTScore offre una lettura più aderente alla similarità semantica. Al tempo stesso, l'analisi degli esempi evidenzia che un punteggio semantico medio-alto non è sufficiente, da solo, a garantire la piena correttezza applicativa della risposta. In alcuni casi il modello produce contenuti plausibili ma non perfettamente allineati alla domanda.

4.5.1.3 Pipeline RAG

Per la pipeline Retrieval-Augmented Generation (RAG), la valutazione automatica è stata condotta sul medesimo test set utilizzato per il modello LoRA, considerando lo stesso insieme di 100 domande. Le metriche medie ottenute sono riportate di seguito.

	metrica	valore
0	num_examples	100.000000
1	bleu	0.006506
2	rouge1	0.130862
3	rouge2	0.023496
4	rougeL	0.098227
5	rougeLsum	0.097799
6	meteor	0.165893
7	chrf	24.593633
8	bertscore_precision_mean	0.611170
9	bertscore_recall_mean	0.699134
10	bertscore_f1_mean	0.651275
11	exact_match	0.000000
12	token_f1	0.141418

Figura 4.5: Metriche medie ottenute dalla pipeline RAG sul test set

I risultati evidenziano anche per la pipeline RAG una chiara separazione tra metriche lessicali e metriche semantiche, ma con caratteristiche leggermente diverse rispetto al modello LoRA. Le metriche basate sulla sovrapposizione esplicita dei token, come BLEU (0.0065), ROUGE-1 (0.1309), ROUGE-L (0.0982) e Token F1 (0.1414), assumo-

no valori contenuti e complessivamente inferiori rispetto a quelli osservati nel caso LoRA. Questi valori indicano che le risposte generate dalla pipeline RAG tendono a discostarsi maggiormente dalla formulazione della risposta di riferimento, mostrando una variabilità lessicale più elevata. Tale comportamento può essere ricondotto al fatto che il contenuto della risposta non deriva dalla conoscenza appresa nei pesi del modello, ma viene costruito combinando informazioni recuperate dinamicamente, che possono essere riformulate o sintetizzate in modi diversi rispetto al target.

Il valore nullo di Exact Match si colloca in continuità con quanto osservato per LoRA e conferma che nessuna risposta coincide esattamente con la risposta gold. Come già discusso, questo risultato è atteso in un contesto di generazione libera e non costituisce un indicatore affidabile della qualità delle risposte.

Dal punto di vista semantico, il valore medio di BERTScore F1 è pari a 0.6513, inferiore rispetto a quello ottenuto dal modello LoRA (0.6796). Tale differenza suggerisce una minore aderenza complessiva tra le risposte generate e quelle di riferimento. Tuttavia, l'analisi delle componenti del BERTScore permette di interpretare meglio questo comportamento: il valore di recall (0.6991) risulta sensibilmente più alto rispetto alla precision (0.6112). Lo sbilanciamento tra recall e precision indica che il modello tende a coprire una parte significativa delle informazioni rilevanti presenti nella risposta gold, ma lo fa spesso introducendo contenuti aggiuntivi o meno focalizzati. In altre parole, la pipeline RAG appare incline a fornire risposte più ampie e meno selettive, includendo elementi corretti ma non sempre strettamente necessari rispetto alla domanda.

Questo risultato può essere spiegato considerando il funzionamento della pipeline RAG. La qualità della risposta dipende infatti non solo dalla capacità generativa del modello, ma anche dalla pertinenza e dalla granularità delle informazioni recuperate nella fase di retrieval. Quando il contesto recuperato contiene informazioni parzialmente rilevanti o eterogenee, il modello tende a integrarle producendo risposte semanticamente plausibili ma meno precise e più dispersive.

In modo analogo a quanto osservato per il modello LoRA, è utile analizzare come la similarità semantica si relazioni alla sovrapposizione lessicale a livello di singola istanza. A tal fine, la Figura 4.6 mette in relazione i valori di BERTScore F1 e Token F1 per ciascuna risposta generata dalla pipeline RAG.

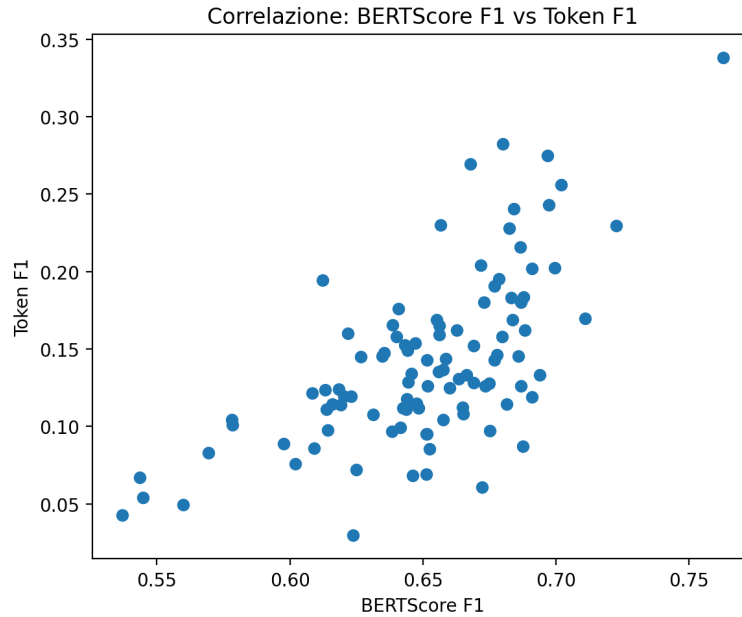


Figura 4.6: Relazione tra BERTScore F1 e Token F1 per le risposte generate dalla pipeline RAG

La distribuzione dei punti evidenzia differenze rilevanti rispetto a quanto osservato nel caso LoRA. Sebbene anche in questo caso emerga una correlazione positiva tra BERTScore F1 e Token F1, la relazione risulta meno compatta e caratterizzata da una dispersione più marcata.

In particolare, mentre nel modello LoRA i punti tendevano a concentrarsi in una fascia relativamente ristretta, nel caso della pipeline RAG si osserva una variabilità più ampia sia lungo l'asse semantico sia lungo quello lessicale. Questo indica che la qualità delle risposte non è uniforme. Accanto a istanze con elevata aderenza alla risposta gold, sono presenti casi in cui la similarità semantica risulta solo moderata.

Un aspetto particolarmente rilevante è la presenza di numerosi punti con valori medi di BERTScore (intorno a 0.6–0.65) associati a valori bassi di Token F1. Rispetto a LoRA, questo fenomeno appare più frequente e accentuato, suggerendo che la pipeline RAG tende più spesso a produrre risposte semanticamente plausibili ma formulate in modo significativamente diverso rispetto al riferimento.

Questa maggiore dispersione può essere interpretata come un effetto della componente di retrieval. La qualità della risposta dipende dalla pertinenza e dalla coerenza del contesto recuperato, che può variare sensibilmente tra le diverse istanze. Di conseguenza, il modello genera output meno stabili, alternando risposte molto aderenti a risposte più generiche o parzialmente allineate.

Per facilitare un confronto diretto tra i due approcci, gli esempi riportati nella Tabella 4.4 fanno riferimento alle stesse istanze già considerate per il modello LoRA. In questo modo è possibile osservare come, a parità di domanda, le differenze nelle risposte siano riconducibili esclusivamente alla diversa strategia di adattamento.

Domanda	Risposta RAG	Risposta gold	BERTScore F1
Cosa va controllato prima di premere il tasto di erogazione sulla Y3.3?	Verifica che il serbatoio sia pieno e che la capsula sia inserita correttamente. Assicurati inoltre che la macchina sia pronta per l'erogazione.	Assicurati che ci sia acqua, che la tazza sia al posto giusto e che la capsula sia inserita correttamente.	0.75
Cosa controllare se la X7.1 non eroga nulla al primo tentativo?	Controlla che il serbatoio sia pieno e inserito correttamente, verifica che il circuito sia stato inizializzato e che non ci siano blocchi nell'erogatore.	Controlla che il serbatoio sia inserito bene, che il circuito sia adescato e che nel gruppo non ci siano residui; poi prova un'erogazione senza capsula.	0.76
Perché dalla Jessica continuano a cadere gocce anche dopo aver finito il caffè?	Un leggero gocciolamento è normale dopo l'erogazione, ma se continua potrebbe essere necessario pulire il gruppo erogatore o controllare la capsula.	Qualche goccia finale è normale, ma se il gocciolamento dura a lungo conviene pulire l'erogatore e controllare che la capsula sia stata perforata correttamente.	0.72
Come si esegue un risciacquo rapido tra due capsule diverse sulla X7.1?	Per effettuare un risciacquo, fai scorrere acqua senza inserire una capsula per alcuni secondi prima di procedere con la nuova erogazione.	Tra una bevanda e l'altra fai passare per qualche secondo solo acqua, così elimini i residui del gusto precedente.	0.66
Come si pulisce la griglia appoggiata tazza della X7.1?	È consigliabile eseguire una pulizia generale della macchina con prodotti specifici e verificare che tutte le parti siano prive di residui.	Rimuovi la griglia, lavala con acqua calda e detergente neutro, asciugala e reinseriscila solo quando è perfettamente pulita.	0.60

Tabella 4.4: Esempi di risposte generate dalla pipeline RAG sulle stesse istanze considerate per il modello LoRA

L'analisi degli esempi evidenzia alcune caratteristiche distintive del comportamento della pipeline RAG. Nei casi con BERTScore più elevato, le risposte risultano molto vicine a quelle di riferimento sia in termini di contenuto sia di struttura, mostrando che il sistema è in grado di recuperare informazioni pertinenti e integrarle efficacemente nella generazione.

Tuttavia, rispetto al modello LoRA, emerge una maggiore tendenza a produrre risposte leggermente più generiche o riformulate. Ad esempio, nelle domande relative a problemi di erogazione o flusso irregolare, la pipeline RAG mantiene correttamente il contenuto informativo principale, ma tende a esprimerlo in forma più descrittiva e meno aderente alla formulazione della risposta gold. Tale andamento si riflette nei valori più contenuti osservati nelle metriche basate su sovrapposizione lessicale.

Nei casi intermedi, le differenze diventano più evidenti. Le risposte risultano corrette dal punto di vista semantico, ma meno precise o meno complete rispetto al riferimento. Questo suggerisce che il contesto recuperato durante la fase di retrieval non sem-

pre contiene tutte le informazioni necessarie oppure che tali informazioni non vengono completamente sfruttate dal modello in fase di generazione.

Infine, nei casi con punteggi più bassi, la pipeline RAG tende a produrre risposte plausibili ma eccessivamente generiche. A differenza del modello LoRA, che può incorrere in errori di associazione tra prodotti o procedure, gli errori della pipeline RAG sono più frequentemente legati a una perdita di specificità. Il modello rimane nel dominio corretto, ma fornisce indicazioni troppo ampie, che non rispondono in modo puntuale alla domanda.

Ciò rispecchia quanto emerge dall'analisi aggregata delle metriche. Il valore relativamente elevato di recall indica che una parte significativa delle informazioni rilevanti viene effettivamente recuperata, mentre il valore più basso di precision riflette la presenza di contenuti aggiuntivi o meno focalizzati. Di conseguenza, la pipeline RAG tende a privilegiare la copertura informativa rispetto alla selettività, producendo risposte generalmente corrette ma meno mirate rispetto al modello LoRA.

Il confronto diretto sulle stesse istanze mette in evidenza come la pipeline RAG tenda a mantenere una maggiore coerenza semantica e una minore propensione a generare risposte fuori contesto, grazie al supporto del recupero dinamico di informazioni esterne. Tuttavia, questo vantaggio si accompagna a una minore precisione nella selezione dei contenuti rilevanti. Le risposte risultano spesso più generiche o includono informazioni non strettamente necessarie rispetto alla domanda. Inoltre, la dipendenza dalla fase di retrieval introduce una maggiore variabilità tra le istanze, rendendo meno uniforme la qualità complessiva delle risposte rispetto al modello LoRA.

4.5.2 LLM as a judge

Le metriche automatiche forniscono una misura oggettiva e sistematica della similarità tra le risposte generate e quelle di riferimento, ma presentano alcuni limiti intrinseci. Le metriche basate su sovrapposizione lessicale risultano fortemente sensibili alla formulazione testuale, mentre anche le metriche semantiche, pur più robuste, non sono in grado di cogliere pienamente aspetti qualitativi come la chiarezza espositiva, la completezza della risposta o la sua utilità per l'utente finale.

In un contesto di question answering generativo, infatti, la qualità di una risposta non dipende unicamente dalla similarità rispetto a una ground truth, ma anche dalla capacità di rispondere in modo corretto, pertinente e adeguato al contesto della domanda. Due risposte possono essere semanticamente simili ma differire significativamente in termini di precisione, struttura o livello di dettaglio, elementi che le metriche automatiche faticano a rappresentare.

Per questo motivo, al fine di ottenere una valutazione più completa, è stato introdotto un approccio basato su un modello linguistico utilizzato come giudice (*LLM as a judge*).

ge). In questo schema, un LLM viene impiegato per valutare le risposte generate secondo criteri più vicini al giudizio umano, considerando aspetti quali correttezza, completezza, chiarezza e aderenza alla domanda.

L'utilizzo di un LLM come valutatore consente di superare, almeno in parte, i limiti delle metriche automatiche, introducendo un livello di analisi più interpretativo e contestuale. Questo approccio permette di cogliere differenze qualitative tra le risposte che non emergono dai soli punteggi numerici, fornendo una prospettiva complementare e più aderente all'effettiva esperienza dell'utente finale.

4.5.2.1 Definizione del sistema di valutazione

Il modello selezionato come giudice è `openai/gpt-oss-20b`, un modello linguistico di grandi dimensioni con supporto nativo al formato conversazionale, accessibile tramite la libreria `transformers` di Hugging Face. La scelta è motivata dalla necessità di disporre di un modello sufficientemente espressivo da applicare criteri di valutazione articolati, mantenendo al contempo una chiara separazione rispetto ai modelli oggetto dell'analisi, così da preservare l'indipendenza del giudizio ed evitare possibili bias dovuti alla condivisione di parametri o dati di addestramento. Il modello è stato caricato utilizzando `device_map="auto"`, al fine di sfruttare automaticamente le risorse hardware disponibili, in particolare l'accelerazione GPU nell'ambiente Google Colab, e `torch_dtype="auto"`, per ottimizzare l'utilizzo della memoria durante l'inferenza. L'interazione con il modello avviene in formato chat, mediante l'utilizzo del *chat template* del tokenizer, che consente di strutturare l'input in termini di ruoli, ad esempio *system* e *user*, coerentemente con il formato atteso dal modello. Questo approccio garantisce una maggiore stabilità nella generazione e una migliore interpretazione delle istruzioni di valutazione. La fase di generazione è stata configurata in modalità deterministica (`do_sample=False`, `temperature=None`), in linea con l'esigenza di garantire la riproducibilità delle valutazioni e ridurre la variabilità stocastica tipica dei modelli generativi.

Le istruzioni fornite al modello richiedono esplicitamente la produzione di un oggetto JSON valido. Il modello giudice è infatti istruito a restituire esclusivamente un JSON contenente i punteggi per ciascuna dimensione di valutazione e una breve motivazione testuale in italiano. Il formato di output atteso include quindi sia le metriche quantitative sia un campo testuale aggiuntivo denominato *reason*, utilizzato per fornire una giustificazione sintetica del giudizio espresso. Questo campo ha la funzione di aumentare l'interpretabilità del processo di valutazione. La *reason* non rappresenta una dimensione di scoring, ma una motivazione esplicativa del punteggio complessivo. La sua presenza è quindi parte integrante della progettazione del sistema di giudizio, in quanto consente di affiancare alla valutazione numerica una traccia qualitativa del ragionamento del modello.

Per garantire un output coerente con questo formato e direttamente utilizzabile nella fase di parsing strutturato, è stata inoltre adottata una gestione esplicita della sequenza generata dal modello. Dopo la generazione viene separata la porzione corrispondente al prompt iniziale dai token effettivamente prodotti, considerando esclusivamente questi ultimi come output finale. Questa operazione evita che il testo di input venga incluso nella risposta decodificata, un problema che può verificarsi nei modelli autoregressivi quando viene restituita l'intera sequenza (prompt + generazione). In questo modo si ottiene una stringa contenente unicamente il contenuto generato, riducendo le ambiguità e facilitando l'estrazione del JSON richiesto. Infine, per gestire le inevitabili imperfezioni dell'output di un modello generativo, è stato implementato un parser a tre livelli di fallback: un primo tentativo di parsing JSON diretto, un secondo livello basato su estrazione tramite espressioni regolari (*loose parsing*), e un ultimo livello che prevede l'assegnazione di punteggi di default in caso di fallimento completo del parsing.

Il sistema di giudizio è stato progettato per valutare ciascuna risposta secondo sette dimensioni qualitative distinte, alle quali si aggiunge un punteggio complessivo. Le dimensioni sono state selezionate con l'obiettivo di coprire aspetti complementari della qualità della risposta, includendo sia criteri di accuratezza informativa sia aspetti legati alla forma e all'utilità pratica della risposta nel contesto di un sistema di assistenza tecnica nel dominio delle macchine da caffè. La definizione delle dimensioni riflette la necessità di distinguere tra correttezza fattuale, adeguatezza rispetto alla domanda e qualità comunicativa, evitando di ridurre la valutazione a un singolo indicatore aggregato. Le dimensioni considerate sono le seguenti:

- **Correctness:** accuratezza fattuale dei contenuti rispetto alla risposta gold;
- **Completeness:** copertura dei punti informativi rilevanti presenti nella risposta di riferimento;
- **Relevance:** pertinenza della risposta rispetto alla domanda posta;
- **Faithfulness:** aderenza alle informazioni corrette, con penalizzazione esplicita di allucinazioni e dettagli inventati;
- **Clarity:** qualità espositiva e chiarezza del testo generato;
- **Conciseness:** capacità di essere sintetici senza sacrificare le informazioni essenziali;
- **Domain Compliance:** aderenza al dominio tecnico e al tipo di assistenza richiesto;
- **Overall Score:** punteggio complessivo, non meccanicamente derivato come media delle dimensioni precedenti, ma espressione di un giudizio olistico della risposta.

L'introduzione di dimensioni distinte consente di analizzare in modo più fine il comportamento dei modelli, evidenziando eventuali trade-off tra accuratezza, completezza e qualità espositiva. In particolare, dimensioni come *faithfulness* e *domain compliance* risultano cruciali nel contesto considerato, in quanto permettono di intercettare errori tipici dei modelli generativi, quali allucinazioni o risposte formalmente corrette ma non aderenti al dominio tecnico. Ogni dimensione viene valutata su una scala discreta intera da 0 a 7, dove 0 indica una risposta completamente errata o fuori dominio e 7 una risposta eccellente e completa. L'utilizzo di una scala a granularità relativamente elevata consente al giudice di esprimere valutazioni più sfumate rispetto a scale più ristrette, pur mantenendo un livello di discrezionalità controllato.

La rubrica fornita al modello giudice è stata progettata con due obiettivi principali: definire in modo esplicito e non ambiguo i criteri di valutazione per ciascuna dimensione e guidare il modello verso un utilizzo efficace dell'intera scala di punteggio, contrastando la tendenza dei modelli linguistici a concentrare le valutazioni su valori intermedi. A tal fine, il prompt include istruzioni esplicite sulla distribuzione attesa dei punteggi e introduce penalizzazioni mirate per allucinazioni e omissioni rilevanti, con l'obiettivo di rendere il processo di valutazione più discriminativo e coerente.

Rubrica di valutazione e descrizione della scala (RUBRIC_IT e SCALE_DESC_IT)

```

1 SCALE_DESC_IT = """
2 Usa SOLO punteggi interi da 0 a 7.
3 0 = completamente errata o fuori dominio
4 1 = quasi totalmente errata
5 2 = in gran parte errata
6 3 = parzialmente corretta ma debole o vaga
7 4 = discreta ma con lacune evidenti
8 5 = buona e corretta ma non completa
9 6 = molto buona e precisa
10 7 = eccellente e completa
11
12 ISTRUZIONI CRITICHE:
13 - Devi differenziare chiaramente le risposte.
14 - NON assegnare sempre lo stesso punteggio.
15 - Usa tutta la scala disponibile.
16 - Forza una distribuzione realistica dei punteggi.
17 """.strip()
18
19 RUBRIC_IT = """
20 Valuta la risposta candidata rispetto alla risposta gold
21 nel dominio delle macchine da caffè'.
22
23 Dimensioni da valutare:
24 - correctness, completeness, relevance, faithfulness,
25   clarity, conciseness, domain_compliance, overall_score
26
27 Regole importanti:
28 - Penalizza fortemente hallucinations e dettagli inventati.
29 - Penalizza omissioni rilevanti.
30 - Premia risposte corrette, utili e specifiche.
31 - Restituisci SOLO un oggetto JSON valido.
32 - reason deve essere breve, in italiano, massimo 20 parole.
33 """.strip()

```

Con l'obiettivo di analizzare l'effetto del contesto fornito al giudice sulla stabilità e qualità delle valutazioni, il sistema è stato eseguito in tre distinte modalità di prompting:

- **Zero-shot:** il modello riceve unicamente la rubrica e la descrizione della scala, senza alcun esempio di riferimento;
- **One-shot:** viene fornito un singolo esempio di calibrazione annotato, rappresentativo di una valutazione di bassa qualità;
- **Few-shot:** vengono forniti quattro esempi di calibrazione, coperti su diversi livelli qualitativi della scala (punteggi 1, 7, 5 e 3), con l'obiettivo di guidare il modello verso un uso più ampio e distribuito della scala di valutazione.

Gli esempi di calibrazione sono stati costruiti manualmente su domande del dominio applicativo e coprono scenari rappresentativi: risposta completamente fuori tema, risposta eccellente, risposta corretta ma incompleta, e risposta troppo sintetica. Il codice di costruzione del prompt per ciascuna modalità è riportato di seguito.

Costruzione del prompt per le tre modalità di prompting

```
1 CALIBRATION_EXAMPLES = [  
2     {  
3         "question": "Perche' il caffe' esce freddo?",  
4         "gold": "Puo' dipendere da mancato preriscaldamento, calcare o problema al sistema di  
5             riscaldamento.",  
6         "prediction": "Devi cambiare capsula e controllare il Wi-Fi della macchina.",  
7         "scores": {"correctness": 1, "completeness": 1,  
8             "relevance": 1, "faithfulness": 1,  
9             "clarity": 3, "conciseness": 5,  
10            "domain_compliance": 1, "overall_score": 1},  
11         "reason": "Fuori tema e fattualmente scorretta."  
12     },  
13     # ... (altri 3 esempi coprono score 7, 5, 3)  
14 ]  
15 def _render_single_examples(mode: str) -> str:  
16     if mode == "zero_shot":  
17         return ""  
18     exs = CALIBRATION_EXAMPLES[:1] if mode == "one_shot" \  
19         else CALIBRATION_EXAMPLES  
20     blocks = []  
21     for i, ex in enumerate(exs, 1):  
22         blocks.append(  
23             f"Esempio {i}\n"  
24             f"Domanda: {ex['question']}\n"  
25             f"Risposta gold: {ex['gold']}\n"  
26             f"Risposta candidata: {ex['prediction']}\n"  
27             f"JSON corretto:\n"  
28             f"{json.dumps({**ex['scores'],  
29                 'reason': ex['reason']}, ensure_ascii=False)}"  
30         )  
31     return "\n\n".join(blocks)
```

Per ciascuna delle 100 domande del test set, il giudice ha valutato separatamente la risposta prodotta dalla pipeline RAG e quella prodotta dal modello LoRA, ripetendo la

procedura per ciascuna delle tre modalità di prompting. In questo modo, ogni istanza del dataset è stata valutata in modo indipendente per entrambe le soluzioni e sotto tutte le configurazioni di contesto del giudice. Il processo complessivo ha pertanto prodotto $100 \times 2 \times 3 = 600$ valutazioni individuali. Questa struttura consente di analizzare sia le differenze tra i due sistemi sia la sensibilità delle valutazioni rispetto alla modalità di prompting adottata. Al termine di ciascuna coppia di valutazioni (RAG vs LoRA), il vincitore è stato determinato in modo pairwise confrontando i punteggi `overall_score` assegnati ai due sistemi. In caso di parità, la preferenza è stata considerata neutra. Questo approccio consente di ridurre la dipendenza dai valori assoluti dei punteggi e di focalizzarsi sulla comparazione relativa tra le due architetture.

4.5.2.2 Analisi dei punteggi

Il processo di valutazione ha prodotto un totale di 600 giudizi individuali, organizzati in un dataset strutturato che costituisce la base dell'analisi presentata in questa sezione. Per ciascuna delle 100 domande del test set, il giudice LLM ha valutato separatamente la risposta generata dalla pipeline RAG e quella prodotta dal modello LoRA, ripetendo la procedura per ognuna delle tre modalità di prompting (zero-shot, one-shot, few-shot).

Ogni record del dataset è identificato da due campi di contesto: `shot_mode`, che indica la modalità di prompting adottata dal giudice, e `row_idx`, che identifica univocamente la domanda all'interno del test set. A questi si affiancano i campi testuali `domanda`, `risposta_gold`, `rag_answer` e `lora_answer`. Per ciascuna risposta valutata, il giudice ha prodotto un punteggio numerico intero su scala 0–7 per ognuna delle sette dimensioni qualitative definite nella rubrica, registrati con il prefisso `rag_` o `lora_` a seconda del sistema valutato. Il campo `rag_overall_score` (o `lora_overall_score`) riporta il punteggio complessivo come valutazione olistica, non derivato meccanicamente dalla media delle dimensioni ma espresso come giudizio sintetico autonomo. Accanto ai punteggi numerici, i campi `rag_reason` e `lora_reason` registrano una motivazione testuale sintetica del giudizio, utile per interpretare i punteggi e identificare le principali criticità o i punti di forza di ciascuna risposta. Infine, il dataset include due campi derivati: `winner_from_scores`, che indica il sistema vincitore per ogni istanza (con valore `tie` in caso di parità), e `overall_delta_lora_minus_rag`, che esprime la differenza $\Delta = \text{LoRA}_{\text{overall}} - \text{RAG}_{\text{overall}}$, con valori negativi a indicare preferenza per RAG e valori positivi per LoRA.

A partire da questo dataset è stata condotta un'analisi aggregata per modalità di prompting, i cui risultati sono riportati nella Tabella 4.5.

La pipeline RAG ottiene punteggi medi superiori in tutte e tre le modalità, con un divario che varia da +0.196 punti nella modalità few-shot a +0.420 nella modalità one-shot.

Modalità	n	RAG μ	RAG σ	LoRA μ	LoRA σ	Δ medio	RAG vince	LoRA vince	Pareggio
Zero-shot	100	3.497	1.758	3.257	1.396	−0.240	56	38	6
One-shot	100	3.663	1.864	3.243	1.465	−0.420	56	42	2
Few-shot	100	3.618	1.896	3.422	1.553	−0.196	54	39	7
Totale	300	3.593	–	3.307	–	−0.285	166	119	15

Tabella 4.5: Statistiche aggregate per modalità di prompting. μ e σ indicano media e deviazione standard dell’`overall_score`. Δ è il delta medio LoRA–RAG: valori negativi indicano superiorità della pipeline RAG.

Aggregando le tre modalità, RAG risulta preferita nel 55.3% delle valutazioni pairwise (166 su 300), contro il 39.7% di LoRA (119 su 300), con un 5% di pareggi. La modalità one-shot produce il delta più ampio, suggerendo che un singolo esempio calibrante fortemente negativo orienti il giudice verso valutazioni più discriminanti. La modalità few-shot riduce invece il delta a -0.196 , probabilmente perché una calibrazione più articolata rende il giudice più cauto nell’assegnare punteggi estremi. In tutti e tre i casi la direzione del vantaggio rimane invariata, indicando che la superiorità numerica della pipeline RAG non è un artefatto della configurazione del giudice ma un pattern replicabile attraverso le diverse condizioni sperimentali. I valori di deviazione standard sono tuttavia considerevolmente elevati — nell’ordine di 1.4–1.9 punti su una scala a otto valori — riflettendo l’eterogeneità del test set e anticipando le implicazioni sull’analisi statistica discussa più avanti.

Al fine di verificare se il vantaggio numerico osservato sia statisticamente significativo, è stato applicato il test di Wilcoxon signed-rank sui punteggi `overall_score` delle coppie RAG–LoRA per ciascuna modalità. Il test di Wilcoxon è stato preferito al t -test parametrico in quanto non assume la normalità della distribuzione dei punteggi, che hanno natura ordinale e discreta. I risultati sono riportati nella Tabella 4.6.

Modalità	W	p-value	Significativo ($\alpha = 0.05$)
Zero-shot	2147.0	0.7470	No
One-shot	2243.0	0.5176	No
Few-shot	2107.0	0.7634	No

Tabella 4.6: Risultati del test di Wilcoxon signed-rank applicato sull’`overall_score` per ciascuna modalità di prompting. H_0 : le distribuzioni dei punteggi RAG e LoRA sono identiche. Soglia di significatività $\alpha = 0.05$.

I p-value ottenuti sono ampiamente superiori alla soglia convenzionale in tutte e tre le modalità (compresi tra 0.518 e 0.763), rendendo impossibile rifiutare l'ipotesi nulla. Il delta medio ($\Delta \approx -0.28$) risulta modesto rispetto alla deviazione standard intra-condizione, la differenza di prestazioni è numericamente consistente e replicabile, ma la varianza delle valutazioni a livello di singola istanza è sufficiente da non permettere di escludere che le differenze osservate siano imputabili alla variabilità del giudice. Questo non invalida il pattern di superiorità numerica della pipeline RAG, ma impone cautela nell'attribuirgli una valenza assoluta.

I grafici che seguono forniscono una lettura visiva complementare, permettendo di articolare l'analisi su più livelli: il profilo qualitativo complessivo, le differenze per dimensione, la distribuzione dei punteggi e quella dei delta pairwise.

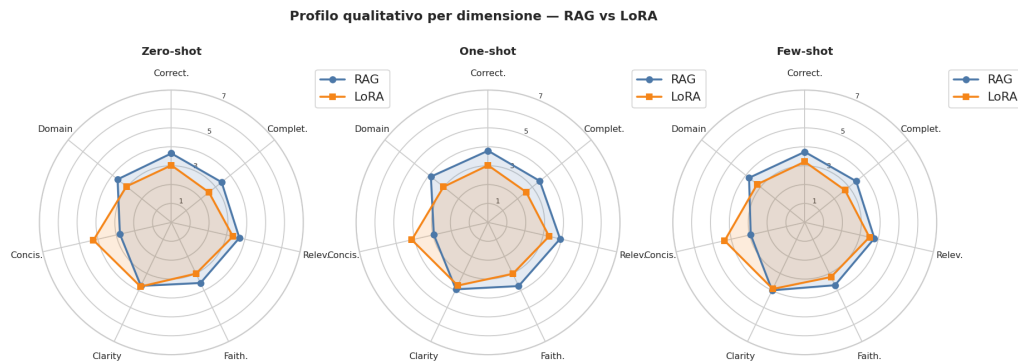


Figura 4.7: Profilo qualitativo medio per dimensione nelle tre modalità di prompting. Il poligono blu (RAG) risulta più esteso di quello arancione (LoRA) in quasi tutte le direzioni, con l'unica eccezione della dimensione *conciseness*.

Il grafico radar della Figura 4.7 offre una visione d'insieme del profilo qualitativo dei due sistemi su tutte le sette dimensioni contemporaneamente. In ciascuno dei tre pannelli il poligono della pipeline RAG risulta più esteso rispetto a quello del modello LoRA, con un vantaggio particolarmente marcato su *correctness*, *completeness*, *faithfulness* e *domain compliance*. L'unica eccezione rilevante è *conciseness*, dimensione in cui il poligono LoRA sporge verso l'esterno: il modello tende a produrre risposte più sintetiche, mentre la pipeline RAG, recuperando informazioni da fonti esterne, produce testi più articolati e talvolta più dispersivi. Il profilo complessivo rimane stabile nelle tre modalità, confermando la robustezza del vantaggio RAG al variare della configurazione del giudice.

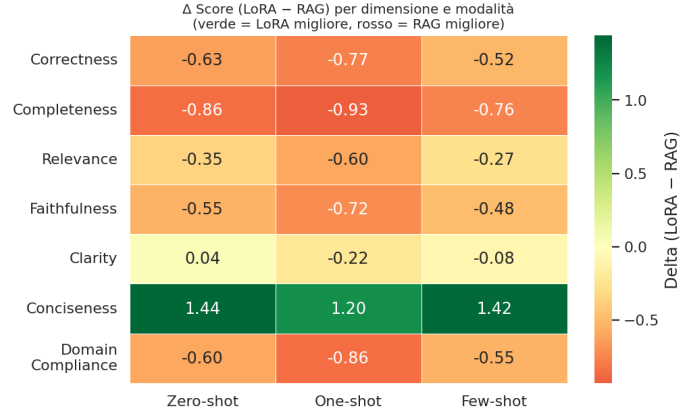


Figura 4.8: Δ Score (LoRA–RAG) per dimensione e modalità di prompting. Valori negativi (in rosso) indicano superiorità della pipeline RAG; l’unico valore positivo sistematico (in verde) riguarda la dimensione *conciseness*.

La Figura 4.8 quantifica il delta per ciascuna combinazione di dimensione e modalità. Il pattern è netto e coerente, il vantaggio RAG è diffuso e stabile su *correctness* (Δ medio ≈ -0.64), *completeness* (≈ -0.85), *faithfulness* (≈ -0.58) e *domain compliance* (≈ -0.67), con valori che si intensificano nella modalità one-shot. La dimensione *clarity* risulta sostanzialmente in pareggio, con delta prossimi allo zero. La sola *conciseness* mostra un vantaggio sistematico e marcato del modello LoRA ($\Delta \approx +1.35$), dato che il giudice valuta positivamente la maggiore sintesi delle risposte LoRA, caratteristica che non è tuttavia sufficiente a compensare le lacune su accuratezza, completezza e aderenza al dominio.

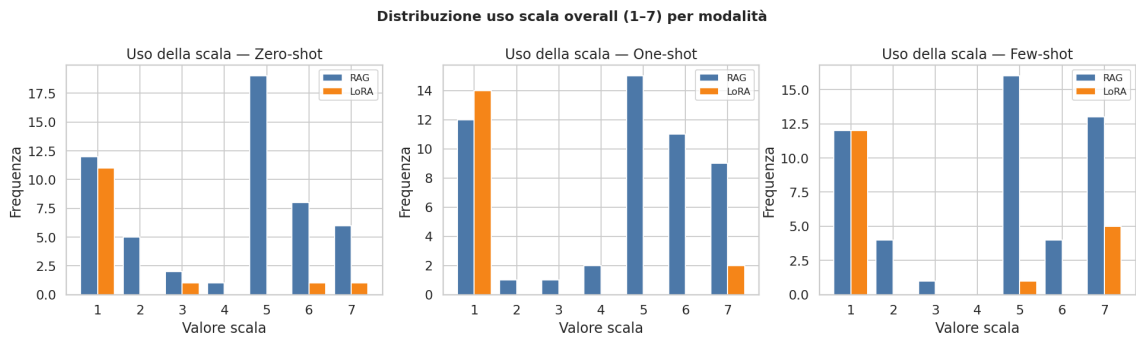


Figura 4.9: Distribuzione dei punteggi `overall_score` assegnati a RAG e LoRA per ciascuna modalità di prompting. La pipeline RAG mostra una distribuzione ampia e spostata verso i valori alti della scala; il modello LoRA presenta una concentrazione marcata sul valore 1.

La Figura 4.9 rivela una differenza strutturale tra le distribuzioni dei due sistemi. La

pipeline RAG presenta masse significative sui valori 5, 6 e 7, indicando che una parte consistente delle risposte viene giudicata di buona o ottima qualità. Il modello LoRA mostra invece una distribuzione fortemente concentrata sul valore 1, con pochissime istanze che superano il valore 2. Questo contrasto asimmetrico spiega l'elevata deviazione standard osservata nelle statistiche aggregate e, soprattutto, chiarisce perché il test di Wilcoxon non raggiunga la significatività. La variabilità non è distribuita simmetricamente tra i due sistemi, ma riflette una polarizzazione netta nelle valutazioni LoRA che contribuisce ad allargare la dispersione complessiva dei delta.

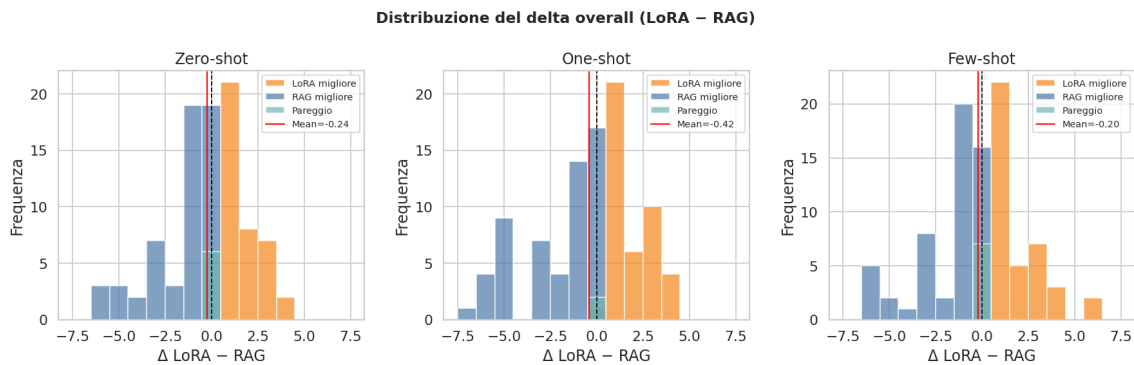


Figura 4.10: Distribuzione del delta `overall_score` (LoRA–RAG) per modalità di prompting. Le barre blu indicano istanze in cui RAG è risultato migliore, le barre arancioni istanze in cui LoRA è risultato migliore. La linea rossa indica il valore medio del delta; la linea tratteggiata nera indica lo zero.

La Figura 4.10 mostra la distribuzione del delta pairwise per ciascuna modalità. In tutti e tre i pannelli la distribuzione è centrata in prossimità dello zero, con la media (linea rossa) leggermente spostata verso sinistra. La coda sinistra — casi in cui RAG vince nettamente — è più pronunciata rispetto alla coda destra, e la modalità one-shot mostra il delta medio più negativo (-0.42) con una distribuzione più spostata a sinistra, confermando che in questa configurazione il giudice risulta più discriminante. La presenza di una quota significativa di vittorie LoRA attorno allo zero spiega e dà riconferma del perché il test statistico non raggiunga la soglia di significatività. La direzione del vantaggio è consistente, ma la variabilità a livello di singola istanza è sufficiente a rendere il pattern non statisticamente distinguibile dal rumore.

Per rendere più concreta l'analisi quantitativa, vengono infine riportati due esempi rappresentativi tratti dal dataset, selezionati in modo da illustrare casi opposti: un'istanza in cui il giudice ha preferito il modello LoRA e una in cui ha preferito la pipeline RAG.

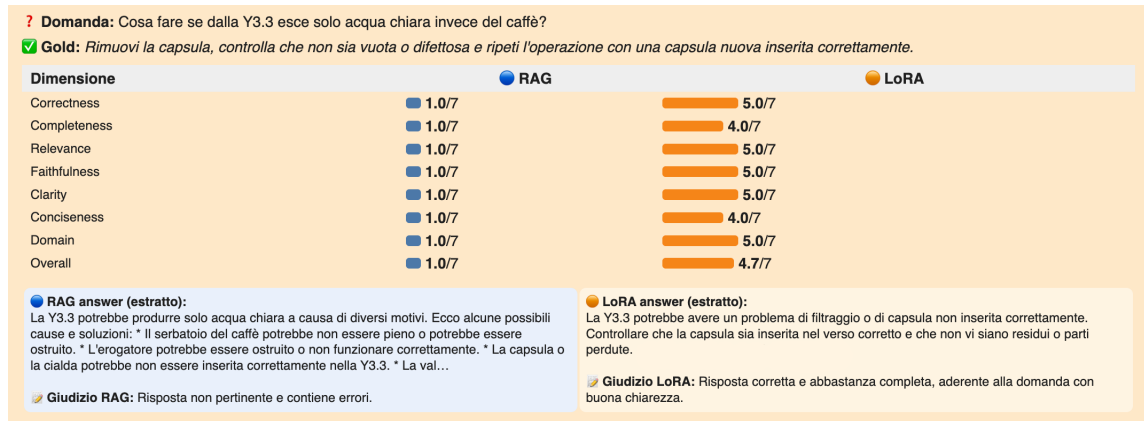


Figura 4.11: Esempio di istanza in cui il modello LoRA ottiene punteggi superiori alla pipeline RAG.

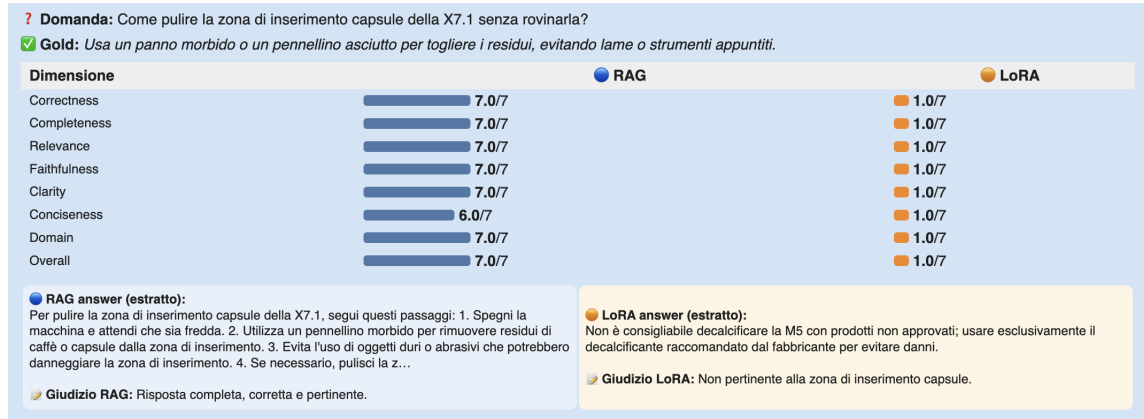


Figura 4.12: Esempio di istanza in cui il modello RAG ottiene punteggi superiori alla pipeline LoRA.

La Figura 4.11 mostra uno dei casi in cui LoRA si è dimostrato superiore. Alla domanda sulla macchina Y3.3, la pipeline RAG ha prodotto una risposta articolata ma fuori tema, elencando cause generiche non pertinenti alla richiesta. Il giudice ha correttamente assegnato il punteggio minimo su tutte le dimensioni, motivando con *"risposta non pertinente e contiene errori"*. Il modello LoRA ha invece fornito una risposta concisa e corretta, identificando il problema della capsula e suggerendo le azioni appropriate. L'errore RAG è riconducibile a un retrieval che ha recuperato documenti solo parzialmente pertinenti, portando il modello a generare una risposta plausibile nella forma ma non allineata alla richiesta specifica.

La Figura 4.12 illustra il caso opposto. Alla domanda su come pulire la zona di inserimento capsule della X7.1, RAG fornisce istruzioni precise, strutturate e complete, ricevendo 7/7 su sei dimensioni e 6/7 su *conciseness*. Il modello LoRA genera invece una

risposta sulla decalcificazione di un prodotto completamente diverso, la M5, commettendo lo stesso errore di associazione tra prodotti già osservato nell'analisi della valutazione automatica. Il giudice ha assegnato 1/7 su tutte le dimensioni, motivando con *"non pertinente alla zona di inserimento capsule"*. Questo esempio evidenzia uno dei limiti strutturali dell'adattamento parametrico tramite LoRA. La conoscenza appresa durante il fine-tuning può risultare parzialmente confusa tra prodotti simili, portando il modello a generare risposte formalmente plausibili ma associate al contesto sbagliato.

I due esempi confermano che il framework di valutazione basato su LLM-as-a-Judge è in grado di discriminare efficacemente tra risposte di qualità molto diversa, assegnando punteggi coerenti con la qualità effettiva e motivando il giudizio in modo interpretabile. Essi illustrano inoltre i pattern di errore caratteristici dei due sistemi, che saranno ripresi e approfonditi nel capitolo dedicato alla discussione dei risultati. La pipeline RAG tende a fallire quando il retrieval recupera contesto non sufficientemente pertinente. Il modello LoRA tende invece a commettere errori di associazione tra prodotti, generando risposte plausibili ma relative a un contesto scorretto.

I risultati ottenuti nei casi analizzati permettono di osservare con chiarezza le differenze comportamentali tra le due strategie considerate, sia in termini di accuratezza delle risposte sia rispetto alla tipologia di errore commesso. Tuttavia, l'analisi condotta in questo capitolo è volutamente focalizzata sulla descrizione puntuale degli esempi e sull'evidenza empirica fornita dal framework di valutazione, senza entrare nel merito di un confronto interpretativo più ampio tra i due approcci. Una lettura comparativa più strutturata, che mette in relazione i risultati quantitativi e qualitativi emersi e ne discute le implicazioni rispetto all'efficacia delle due soluzioni nel dominio considerato, è presentata nella Sezione 5.2, dedicato all'analisi e alla discussione complessiva dei risultati.

5. Discussione dei risultati

Il presente capitolo ha l'obiettivo di interpretare criticamente i risultati emersi dalla fase sperimentale, andando oltre la semplice descrizione delle metriche per evidenziare il significato delle evidenze ottenute nel contesto applicativo considerato. L'analisi si concentra sul confronto tra i due approcci adottati — pipeline RAG nella piattaforma Synapsis ML e adattamento parametrico tramite LoRA — con l'intento di comprendere non solo quale soluzione performi meglio, ma soprattutto perché emergano determinate differenze e quali implicazioni esse abbiano in uno scenario reale.

A differenza del capitolo precedente, in cui i risultati sono stati presentati in modo quantitativo e descrittivo, in questa sede l'attenzione è rivolta alla lettura trasversale delle evidenze, alla coerenza tra le diverse modalità di valutazione e al valore complessivo dell'approccio proposto.

5.1 Sintesi dei risultati

I risultati della valutazione automatica mostrano, per entrambi i sistemi, un pattern atteso nel contesto della generazione libera di testo. Le metriche basate su sovrapposizione lessicale (come BLEU e ROUGE) assumono valori contenuti, mentre le metriche semantiche, in particolare BERTScore, offrono una lettura più informativa della qualità delle risposte.

Per il modello LoRA emerge un evidente disallineamento tra metriche lessicali e semantiche, fenomeno tipico dei sistemi generativi orientati alla parafrasi. Il modello riesce a trasmettere contenuti informativi corretti pur adottando formulazioni diverse rispetto alle risposte di riferimento. Questo comportamento è confermato dalla presenza di numerosi casi in cui la similarità semantica risulta elevata a fronte di una bassa sovrapposizione lessicale. Tuttavia, l'analisi qualitativa degli esempi evidenzia un limite rilevante dell'adattamento parametrico tramite LoRA, la tendenza a commettere errori di associazione tra prodotti. Questo comportamento è riconducibile al meccanismo di memorizzazione implicita del fine-tuning, in cui la conoscenza viene codificata nei pesi del modello in modo non sempre sufficientemente selettivo. In presenza di entità simili, il modello può quindi generare risposte plausibili ma riferite a prodotti o procedure non corretti. La dimensione ridotta del dataset di addestramento e i segnali di convergenza precoce suggeriscono che il modello abbia appreso le strutture linguistiche del dominio, ma non con una granularità tale da garantire una distinzione affidabile tra entità simili.

Per la pipeline RAG, i risultati mostrano valori leggermente inferiori nelle metriche semantiche rispetto a LoRA. Questo dato, tuttavia, non si traduce automaticamente in una

minore qualità complessiva delle risposte. L'analisi delle componenti del BERTScore evidenzia infatti una dinamica diversa, il sistema tende a coprire una porzione ampia delle informazioni rilevanti, ma introducendo talvolta contenuti aggiuntivi o meno focalizzati. In altre parole, la pipeline RAG produce risposte più estese e meno selettive, comportamento direttamente riconducibile alla fase di retrieval, in cui il contesto recuperato può includere informazioni eterogenee o solo parzialmente pertinenti.

Il confronto diretto tra i due sistemi sulle stesse istanze evidenzia quindi due profili distinti. La pipeline RAG tende a mantenere una maggiore coerenza con il dominio e una minore propensione a generare risposte fuori contesto, mentre il modello LoRA produce risposte più concise e strutturalmente stabili, ma più esposte a errori di contenuto. È importante sottolineare che, sia nelle metriche automatiche sia nella valutazione tramite LLM-as-a-judge, le differenze tra i due sistemi non risultano nette o fortemente significative. Piuttosto, emerge una sostanziale comparabilità delle prestazioni, con variazioni contenute tra i due approcci.

Un elemento particolarmente rilevante emerge tuttavia dalla valutazione qualitativa tramite LLM-as-a-judge. La pipeline RAG risulta complessivamente competitiva e, in diversi casi, leggermente superiore al modello LoRA, nonostante sia stata realizzata utilizzando risorse computazionali limitate e strumenti a basso costo. Questo risultato assume rilievo se si considera la natura dei due approcci. Il modello LoRA richiede una fase esplicita di addestramento, con costi computazionali e complessità legati alla costruzione del dataset e all'utilizzo di risorse hardware dedicate. La pipeline RAG, al contrario, non necessita di fine-tuning del modello e consente un utilizzo più flessibile delle risorse, pur mantenendo prestazioni comparabili.

Nel complesso, le metriche automatiche e la valutazione qualitativa convergono nel delineare due profili distinti: da un lato un sistema (RAG) più robusto e ancorato al contesto, dall'altro un sistema (LoRA) più compatto e coerente dal punto di vista stilistico, ma più fragile in presenza di ambiguità del dominio.

5.2 Interpretazione delle differenze tra RAG e LoRA

Le differenze osservate tra i due sistemi possono essere ricondotte direttamente alle rispettive caratteristiche architetturali.

La pipeline RAG basa il proprio funzionamento su un meccanismo di recupero dinamico dell'informazione, la risposta viene generata a partire da contenuti esterni selezionati in fase di inferenza. Questo approccio consente di mantenere un'elevata aderenza al dominio, ma introduce una dipendenza dalla qualità del retrieval, che può influenzare la precisione e la focalizzazione della risposta. Il modello LoRA, al contrario, incorpora

la conoscenza all'interno dei propri parametri. Questo porta a risposte generalmente più concise e coerenti, ma introduce un limite nella gestione di informazioni simili o ambigue, con il rischio di generare associazioni errate tra entità.

In questo senso, emerge una differenza nei pattern di errore: la pipeline RAG tende a sbagliare per eccesso, introducendo informazioni non perfettamente rilevanti, mentre il modello LoRA tende a sbagliare per confusione, mescolando conoscenze simili. Nel dominio considerato, caratterizzato da numerose entità tecniche tra loro affini, il secondo tipo di errore risulta più critico, poiché compromette direttamente la correttezza fattuale della risposta.

Un ulteriore aspetto riguarda la stabilità delle prestazioni. La pipeline RAG mostra un comportamento più uniforme, mentre il modello LoRA presenta una maggiore variabilità, alternando risposte adeguate a casi di errore marcato. Questo suggerisce una maggiore robustezza operativa dell'approccio basato su retrieval.

Dal punto di vista applicativo, i risultati suggeriscono che la pipeline RAG rappresenta una soluzione particolarmente adatta per sistemi di supporto clienti in domini tecnici. Il principale vantaggio risiede nella capacità di aggiornamento, la conoscenza del sistema può essere estesa intervenendo sulla base documentale, senza necessità di riaddestrare il modello. Aspetto fondamentale in contesti in cui le informazioni evolvono nel tempo.

Un ulteriore elemento riguarda il rapporto tra prestazioni e costo computazionale. La pipeline RAG consente di ottenere risultati competitivi senza una fase di addestramento, riducendo la complessità operativa e i requisiti infrastrutturali. Il modello LoRA, pur mostrando vantaggi in termini di concisione e coerenza stilistica, risulta più vincolato alla qualità e alla copertura del dataset di addestramento. La necessità di aggiornamenti tramite nuove fasi di training ne limita la flessibilità in contesti dinamici.

Al fine di rendere più concreta e verificabile l'interpretazione proposta, vengono di seguito analizzati quattro casi rappresentativi tratti dal dataset di valutazione, selezionati in modo da illustrare i pattern di errore caratteristici di ciascun sistema. Per ciascun caso viene riportato il confronto diretto tra le risposte generate dai due sistemi, i punteggi assegnati dal giudice LLM per ogni dimensione qualitativa e la motivazione sintetica del giudizio. I casi sono stati selezionati dalla modalità few-shot.

I primi due esempi mostrano casi in cui la pipeline RAG risulta nettamente superiore, i successivi due illustrano casi in cui il modello LoRA si dimostra più efficace.

Risposta RAG	Risposta LoRA
Rimuovi il coperchio del serbatoio della Jessica. Puliscilo con acqua tiepida e un detergente delicato. Risciacqua accuratamente per rimuovere eventuali residui. Asciuga con un panno morbido per evitare residui d'acqua. Una volta pulito e asciugato, rimontalo sul serbatoio.	La durata della garanzia sulla Jessica è di 2 anni dalla data di acquisto. Per avviare le procedure di assistenza, contattare il servizio clienti della Jessica.

Tabella 5.1: Caso 1 — RAG superiore.

Domanda: *Come si pulisce correttamente il coperchio del serbatoio della Jessica?*

Gold: *Puliscilo con un panno morbido o sotto acqua corrente tiepida, evitando detergenti aggressivi che potrebbero rovinare le superfici.*

Sistema	Corr.	Comp.	Rel.	Faith.	Clar.	Conc.	Dom.	Overall
RAG	7	7	7	7	6	6	7	7.0
LoRA	1	1	1	1	1	1	1	1.0

Reason RAG: Risposta dettagliata, corretta e pertinente, con istruzioni chiare e completa.

Reason LoRA: Risposta non pertinente e errata.

Tabella 5.2: Punteggi giudice LLM — Caso 1 (scala 0–7).

Il confronto tra i punteggi assegnati dal giudice evidenzia uno scarto netto tra i due sistemi ($\Delta = -6.0$). La pipeline RAG ottiene il punteggio massimo su cinque dimensioni su sette, con una lieve penalizzazione solo su *clarity* e *conciseness* dovuta alla struttura leggermente articolata della risposta. Il modello LoRA riceve invece il punteggio minimo su tutte le dimensioni. Non si tratta di una risposta parzialmente corretta o incompleta, ma di una risposta completamente fuori tema. Il giudice ha azzerato non solo la correttezza fattuale, ma anche la rilevanza e l'aderenza al dominio, dimensioni che normalmente mostrano valori almeno parziali anche in presenza di errori.

Questo indica che il fallimento del modello LoRA in questo caso non è graduato ma binario. Il meccanismo di associazione ha prodotto una risposta appartenente a una categoria semantica completamente diversa da quella richiesta, garanzia del prodotto invece di procedura di pulizia, rendendo il contenuto inutilizzabile indipendentemente dalla sua qualità formale.

Risposta RAG	Risposta LoRA
Spegni la macchina e attendi che sia fredda. Utilizza un pennellino morbido per rimuovere residui di caffè o capsule dalla zona di inserimento. Evita l'uso di oggetti duri o abrasivi che potrebbero danneggiare la zona. Se necessario, pulisci con un panno morbido umido e asciuga completamente per evitare la formazione di muffa.	Non è consigliabile decalcificare la M5 con prodotti non approvati; usare esclusivamente il decalcificante raccomandato dal fabbricante per evitare danni.

Tabella 5.3: Caso 2 — RAG superiore.

Domanda: *Come pulire la zona di inserimento capsule della X7.1 senza rovinarla?*

Gold: *Usa un panno morbido o un pennellino asciutto per togliere i residui, evitando lame o strumenti appuntiti.*

Sistema	Corr.	Comp.	Rel.	Faith.	Clar.	Conc.	Dom.	Overall
RAG	7	7	7	7	7	6	7	7.0
LoRA	1	1	1	1	1	1	1	1.0

Reason RAG: Risposta completa, corretta e pertinente.

Reason LoRA: Non pertinente alla zona di inserimento capsule.

Tabella 5.4: Punteggi giudice LLM — Caso 2 (scala 0–7).

Anche in questo caso il delta è massimo ($\Delta = -6.0$), con la pipeline RAG che ottiene punteggio pieno su sei dimensioni su sette e il modello LoRA che riceve il minimo su tutte. Rispetto al caso precedente, tuttavia, l'errore di LoRA è ancora più marcato, non solo il modello sbaglia prodotto, ma sbaglia anche la categoria di operazione, passando dalla pulizia alla decalcificazione. Si tratta quindi di un doppio errore di associazione, che il giudice ha sanzionato in modo uniforme su tutte le dimensioni, incluse *relevance* e *domain_compliance*, normalmente più tolleranti verso risposte che restano nel dominio corretto. Il fatto che anche *domain_compliance* sia a 1 indica che il giudice non ha ritenuto sufficiente la generica appartenenza al dominio delle macchine da caffè, la pertinenza al contesto specifico della domanda è stata valutata come condizione necessaria, non la sola coerenza con il dominio in senso ampio. La pipeline RAG ottiene 6 su 7 solo in *conciseness*, penalizzazione lieve dovuta alla struttura leggermente più articolata della risposta rispetto alla gold, che era più diretta. Su tutte le altre dimensioni il punteggio è massimo, confermando che il recupero documentale ha fornito un contesto sufficientemente preciso da guidare una generazione corretta e focalizzata.

Risposta RAG	Risposta LoRA
La Baby Frog è una macchina del caffè del marchio Didiessè. Se la Baby Frog vibra troppo sul piano di lavoro, potrebbe essere dovuto a diverse cause: superficie non piana o instabile; vaschetta raccogliogocce piena o mal posizionata; componenti interni allentati o danneggiati. Verificare che la macchina sia posizionata su una superficie piana e stabile, svuotare la vaschetta raccogliogocce e verificare che tutti i componenti siano correttamente posizionati.	La Baby Frog potrebbe stare sul piano instabile o avere una carica diseguale. Spostala su un piano più stabile e verifica che sia correttamente posizionata.

Tabella 5.5: Caso 3 — LoRA superiore.

Domanda: *Cosa fare se la Baby Frog vibra troppo sul piano di lavoro?*

Gold: *Appoggia la macchina su una superficie perfettamente piana, svuota la vaschetta se è piena e controlla che tutti i componenti siano correttamente posizionati.*

Sistema	Corr.	Comp.	Rel.	Faith.	Clar.	Conc.	Dom.	Overall
RAG	4	4	4	4	4	2	4	3.7
LoRA	5	4	5	5	5	4	5	4.7

Reason RAG: Risposta corretta nei contenuti principali ma eccessivamente verbosa e poco concisa.

Reason LoRA: Risposta corretta e abbastanza completa, aderente alla domanda con buona chiarezza.

Tabella 5.6: Punteggi giudice LLM — Caso 3 (scala 0–7).

A differenza dei casi precedenti, questo esempio mostra uno scenario in cui entrambi i sistemi producono risposte corrette e pertinenti, e la differenza nei punteggi è graduata anziché netta ($\Delta = -1.0$). Il confronto dimensione per dimensione è particolarmente istruttivo. Su *correctness*, *completeness*, *relevance*, *faithfulness* e *domain_compliance* i due sistemi si distanziano di un solo punto (4 per RAG, 5 per LoRA), a indicare che entrambi hanno identificato correttamente le cause della vibrazione e le azioni da intraprendere. Il contenuto informativo è sostanzialmente equivalente, entrambi citano la superficie instabile e il riposizionamento della macchina. La differenza determinante si manifesta su *conciseness*, dove il divario è massimo (2 per RAG, 4 per LoRA). La risposta RAG introduce una presentazione del brand Didiessè non richiesta dalla domanda, struttura le cause in un elenco puntato più lungo del necessario e ripete alcune informazioni già implicite nella gold. Il modello LoRA produce invece due frasi dirette che coprono l'essenziale senza introdurre contenuto superfluo. Il giudice ha propagato questa differenza anche su *clarity* (4 contro 5): una risposta più articolata non è necessariamente più chiara, e in questo caso la struttura verbose della risposta RAG ne riduce la leggibilità rispetto alla formulazione compatta di LoRA. Questo caso mostra che il vantaggio di LoRA su *conciseness*, già emerso nell'analisi aggregata, non è un artefatto statistico ma riflette una differenza siste-

matica. Il modello parametrico, avendo interiorizzato le strutture tipiche delle risposte nel dominio, tende a produrre formulazioni più mirate, mentre la pipeline RAG, costruendo la risposta a partire da contesto recuperato, tende a includere informazioni contestuali non sempre strettamente necessarie.

Risposta RAG	Risposta LoRA
La Frog segnala mancanza d’acqua a causa di bolle d’aria nel circuito. Assicuratevi che il serbatoio sia inserito correttamente e che il circuito sia libero da bolle d’aria. Eseguire l’adescamento può essere utile. <i>[La risposta originale contiene numerosi artefatti di generazione del tipo [/s] che ne compromettono la leggibilità.]</i>	La Frog potrebbe segnalare mancanza d’acqua anche con il serbatoio pieno a causa di aria nel circuito o di un verso chiuso. Controllare le valvole e fare un risciacquo del circuito.

Tabella 5.7: Caso 4 — LoRA superiore.
Domanda: *Perché la Frog segnala mancanza d’acqua anche se il serbatoio è quasi pieno?*
Gold: *Verifica che il serbatoio sia posizionato fino in fondo e che il galleggiante o la valvola non siano bloccati da calcare o sporco.*

Sistema	Corr.	Comp.	Rel.	Faith.	Clar.	Conc.	Dom.	Overall
RAG	2	2	2	1	2	1	2	1.7
LoRA	5	4	5	5	5	4	5	4.7

Reason RAG: Risposta con artefatti di generazione che ne compromettono la qualità e la chiarezza.
Reason LoRA: Risposta corretta e abbastanza completa, aderente alla domanda con buona chiarezza.

Tabella 5.8: Punteggi giudice LLM — Caso 4 (scala 0–7).

Il caso 4 è il più interessante dal punto di vista comparativo. La pipeline RAG ottiene punteggi compresi tra 1 e 2, il modello LoRA tra 4 e 5, con un delta di -3.0 . Questa differenza più contenuta rispetto ai casi 1 e 2 riflette una situazione in cui entrambi i sistemi hanno identificato correttamente la causa del problema — aria nel circuito — ma con qualità espressiva molto diversa. Il confronto su *faithfulness* è notevolmente significativo, RAG ottiene 1, LoRA ottiene 5. Il giudice ha penalizzato duramente la risposta RAG non per un errore fattuale, ma per la presenza di artefatti di generazione che ne compromettono l’affidabilità percepita. In altre parole, un contenuto potenzialmente corretto viene valutato come non fedele perché la sua forma è corrotta. Su *conciseness* il confronto è 1 contro 4, la risposta RAG, pur includendo informazioni rilevanti, è resa prolissa e frammentata dagli artefatti, mentre LoRA produce due frasi pulite e dirette. Tuttavia, anche LoRA non raggiunge il punteggio massimo, su *completeness* e *conciseness* si ferma a 4, perché la risposta gold richiedeva un’indicazione più specifica (verificare il galleggiante o la valvola bloccata da calcare) che nessuno dei due sistemi ha fornito. Questo caso evidenzia

quindi un limite comune ai due approcci nella gestione di domande che richiedono una conoscenza tecnica granulare, ma mostra al contempo come un problema di forma nella risposta RAG possa trasformare un errore parziale in una penalizzazione totale da parte del giudice.

L'analisi comparativa dei quattro casi permette di identificare con maggiore precisione i pattern di errore che caratterizzano i due sistemi e che si riflettono nelle distribuzioni aggregate discusse in precedenza. Il modello LoRA commette errori prevalentemente di contenuto, la risposta è ben formata ma riferita a un prodotto, una procedura o un tema diversi da quelli richiesti. Questi errori derivano dalla natura stessa dell'adattamento parametrico, in cui la conoscenza codificata nei pesi può confondersi tra entità simili del dominio. La pipeline RAG commette invece errori prevalentemente di forma o di focalizzazione, quando il retrieval non recupera documenti sufficientemente pertinenti, la risposta diventa generica, dispersiva o, come nel quarto caso, affetta da artefatti di generazione. Nel dominio considerato, caratterizzato da numerosi prodotti tecnici con nomi e funzionalità simili, gli errori di contenuto del modello LoRA risultano più critici dal punto di vista applicativo, poiché fornire informazioni relative al prodotto sbagliato può generare confusione o decisioni errate nell'utente finale.

5.3 Limiti del sistema

Nonostante i risultati positivi complessivamente ottenuti dalla pipeline RAG, è necessario riconoscere i limiti che caratterizzano il sistema analizzato, sia a livello architetturale che implementativo. Una valutazione scientificamente rigorosa non può prescindere da un'analisi critica delle condizioni in cui i risultati sono stati prodotti e delle aree in cui il sistema presenta margini di miglioramento.

Il primo e più rilevante limite della piattaforma Synapsis ML riguarda la dipendenza da provider esterni per l'esecuzione del modello. Nel contesto sperimentale, il modello è stato eseguito tramite il servizio Groq, che offre accesso alle API di LLaMA con alte prestazioni ma introduce vincoli sul numero di richieste effettuabili, sul throughput e sulla lunghezza massima delle sequenze gestibili. Tali limitazioni hanno reso necessaria una progettazione attenta delle chiamate al modello e hanno impedito l'esecuzione di sperimentazioni su scala più ampia o con varianti della configurazione che avrebbero richiesto un numero elevato di query. In un contesto di produzione, la dipendenza da un provider esterno introduce inoltre rischi legati alla disponibilità del servizio, alla latenza delle risposte e alla gestione dei costi, aspetti che devono essere considerati nella valutazione della fattibilità operativa del sistema.

Un secondo limite riguarda la qualità e la granularità della fase di retrieval. Come emerso dall'analisi dei risultati, la pipeline RAG tende a produrre risposte più generiche o meno precise nei casi in cui il contesto recuperato non è sufficientemente pertinente alla domanda specifica. La configurazione attuale del retriever utilizza una strategia di retrieval semantico basata su embedding vettoriali, con parametri di chunking e di similarità fissati a priori. Non è stato implementato alcun meccanismo di re-ranking, filtraggio adattivo o retrieval ibrido che combini ricerca semantica e ricerca lessicale. L'assenza di queste componenti, presenti nelle architetture Advanced RAG e Modular RAG descritte nel capitolo teorico, costituisce un limite esplicito rispetto allo stato dell'arte e rappresenta una delle principali aree di miglioramento del sistema.

Un terzo limite riguarda la gestione della cronologia conversazionale. La memoria della pipeline è limitata alla sessione corrente e viene ricostruita a ogni richiesta a partire dai messaggi forniti dal frontend. Non è prevista alcuna forma di persistenza o compressione della memoria a lungo termine, il che può penalizzare le prestazioni in conversazioni estese o in scenari multi-turno complessi, nei quali il contesto rilevante si accumula nel corso di più interazioni.

Un quarto limite riguarda i vincoli di compatibilità tra le versioni delle librerie. Come descritto nel capitolo dedicato all'architettura, lo sviluppo della piattaforma è avvenuto su una base software esistente, con la necessità di adattarsi a versioni specifiche di framework come FastAPI e LangChain. Questo ha impedito l'adozione di alcune soluzioni più recenti disponibili nelle versioni aggiornate di LangChain, limitando le possibilità di ottimizzazione della pipeline e richiedendo in alcuni casi l'implementazione di soluzioni alternative.

Infine, un limite trasversale riguarda l'osservabilità del sistema. La pipeline orchestrata, con i suoi componenti di routing, esecuzione parallela e aggregazione, introduce una complessità che rende più difficile il debugging e l'analisi del comportamento del sistema rispetto a una pipeline lineare. Comprendere le cause di un errore richiede l'ispezione di più componenti intermedi — il routing, i risultati parziali delle capability e la fase di aggregazione — il che aumenta il costo di manutenzione e ottimizzazione del sistema.

6. Conclusioni

Il presente elaborato ha illustrato il percorso di analisi, progettazione e sperimentazione condotto nell'ambito dello sviluppo della componente di intelligenza artificiale della piattaforma Synapsis ML, sistema appartenente all'ecosistema Mative. Il lavoro ha avuto come obiettivo principale la comprensione e la valutazione di due strategie di adattamento dei Large Language Models a un dominio applicativo specifico: da un lato, l'implementazione e l'analisi di una pipeline Retrieval-Augmented Generation integrata all'interno di Synapsis ML, elemento centrale attorno a cui ruota l'intera componente conversazionale della piattaforma; dall'altro, la sperimentazione di un approccio di adattamento parametrico tramite Low-Rank Adaptation, sviluppato in parallelo come termine di confronto per valutare le scelte architetturali adottate.

I risultati ottenuti hanno permesso di validare la pipeline RAG come soluzione efficace e concretamente adottabile in un contesto aziendale reale. La valutazione condotta attraverso metriche automatiche e un sistema di giudizio qualitativo basato su LLM-as-a-Judge ha mostrato come il sistema sia in grado di produrre risposte accurate, pertinenti e aderenti al dominio tecnico, con un vantaggio consistente rispetto al modello adattato tramite LoRA sulle dimensioni di correttezza fattuale, completezza e aderenza al dominio. Questi risultati assumono un significato che va oltre il confronto tra due tecniche specifiche, dimostrano che un'architettura RAG ben progettata rappresenta oggi un'alternativa concreta e vantaggiosa rispetto all'adattamento parametrico dei modelli, ottenendo prestazioni comparabili o superiori a fronte di costi di sviluppo e manutenzione significativamente inferiori. Non è necessario addestrare un modello su dati proprietari, gestire pipeline di fine-tuning o dipendere da infrastrutture computazionali dedicate, è sufficiente progettare con cura la base documentale e l'architettura di recupero per ottenere un sistema conversazionale di qualità adeguata a contesti applicativi reali.

In conclusione, questo lavoro ha messo in luce come l'integrazione di architetture basate su Large Language Models in sistemi software industriali già operativi ponga sfide che vanno ben oltre la scelta del modello o della strategia di adattamento. La compatibilità tra librerie, la gestione delle dipendenze, la necessità di operare su basi di codice esistenti e i vincoli imposti dalle risorse computazionali disponibili rappresentano dimensioni progettuali che influenzano in modo determinante le scelte tecniche e che richiedono un approccio metodico e orientato alla stabilità del sistema nel suo complesso. L'esperienza ha confermato che la qualità di una soluzione basata su intelligenza artificiale non dipende esclusivamente dagli algoritmi adottati, ma dalla capacità di integrarli in modo coerente in un contesto tecnologico e organizzativo reale, in cui teoria e vincoli pratici devono trovare

continuamente un punto di equilibrio.

6.1 Sviluppi futuri

I risultati ottenuti e i limiti identificati delineano con chiarezza le direzioni lungo cui il sistema potrà evolvere. Una prima direzione, la più direttamente connessa alle scelte sperimentali di questo lavoro, riguarda la combinazione delle due strategie di adattamento analizzate. Nel contesto sperimentale presentato, LoRA e RAG sono stati valutati separatamente per ragioni metodologiche e di risorse disponibili, ma in una fase di produzione più matura, in cui si disponga di risorse computazionali adeguate, la scelta più naturale sarebbe integrarli in un sistema ibrido. Il modello verrebbe prima specializzato sul dominio tramite fine-tuning con LoRA, acquisendo una conoscenza parametrica delle strutture linguistiche e delle entità specifiche del dominio, e successivamente affiancato da una pipeline RAG per il recupero dinamico di informazioni aggiornate. Questo approccio potrebbe ridurre sia gli errori di associazione tipici del solo LoRA sia la tendenza alla dispersività osservata nel solo RAG, beneficiando al contempo dei punti di forza di entrambe le strategie. Un ulteriore elemento che rende questa direzione particolarmente interessante nel contesto di Synapsis ML è la possibilità di integrare l'intero ciclo direttamente nella piattaforma. Il processo di fine-tuning con LoRA potrebbe essere eseguito e tracciato tramite MLflow, che consentirebbe di registrare l'adapter addestrato come artefatto versionato nel registry dei modelli. Il modello adattato sarebbe così disponibile come provider on-premise all'interno di Synapsis ML, utilizzabile direttamente dalla pipeline RAG in fase di inferenza senza dipendere da servizi esterni. Questo chiuderebbe il ciclo tra adattamento parametrico e recupero dinamico all'interno di un'unica infrastruttura coerente, sfruttando le capacità di tracciabilità e gestione dei modelli già presenti nella piattaforma.

L'evoluzione più naturale dell'architettura attuale riguarda inoltre l'adozione di un paradigma multi-agente, realizzabile attraverso framework come CrewAI o AutoGen, in cui agenti specializzati coopererebbero in modo autonomo per risolvere richieste complesse che richiedono pianificazione, ragionamento multi-step e verifica incrociata delle informazioni, aprendo scenari applicativi oggi non gestibili con un singolo livello di orchestrazione.

Sul piano infrastrutturale, portare l'esecuzione del modello all'interno di Synapsis ML tramite framework di inferenza ad alte prestazioni come vLLM o TGI su hardware dedicato eliminerebbe la dipendenza da provider esterni, garantirebbe piena riservatezza dei dati e consentirebbe di sperimentare con modelli di dimensioni superiori, trasformando il sistema da client di un servizio esterno a infrastruttura autonoma e controllata. Que-

sto passaggio avrebbe implicazioni rilevanti non solo sulle prestazioni, ma anche sulla compliance in contesti industriali con requisiti di sicurezza stringenti.

Un'ulteriore direzione riguarda l'introduzione di meccanismi di apprendimento continuo dal feedback degli utenti. Un sistema conversazionale in produzione genera naturalmente segnali di qualità — valutazioni esplicite, domande di chiarimento, correzioni — che potrebbero essere integrati in un ciclo di ottimizzazione basato su tecniche come *Direct Preference Optimization* (DPO), consentendo al sistema di adattarsi progressivamente ai pattern di errore osservati e alle preferenze degli utenti reali senza richiedere cicli di riaddestramento espliciti.

In una prospettiva più ampia, lo sviluppo della piattaforma si sta orientando sempre più verso l'integrazione di soluzioni basate su dati provenienti da contesti IoT, in cui l'acquisizione continua di informazioni dal campo abilita applicazioni di analisi avanzata e modelli predittivi. Tra queste rientrano, ad esempio, sistemi per la classificazione automatica di eventi e incidenti, nonché strumenti per il monitoraggio e l'interpretazione di fenomeni complessi. In tale scenario, la componente conversazionale sviluppata in questo lavoro non rappresenta un elemento isolato, ma si configura come un'interfaccia trasversale in grado di facilitare l'accesso a queste funzionalità, rendendo interrogabili e comprensibili anche sistemi analitici articolati.

L'obiettivo a lungo termine è quello di costruire una piattaforma capace di unire modelli di analisi dei dati, sistemi IoT e interfacce conversazionali in un unico ecosistema coerente, in cui funzionalità avanzate siano accompagnate da modalità di interazione semplici e immediate. L'intento è quello di ridurre la complessità percepita dall'utente finale, offrendo strumenti che siano al tempo stesso potenti dal punto di vista tecnico e facilmente utilizzabili in contesti operativi reali.

In questa direzione, il valore della soluzione non risiede unicamente nelle singole componenti tecnologiche, ma nella loro integrazione efficace, che consente di trasformare dati grezzi in informazioni utili e accessibili. Il lavoro svolto si inserisce quindi in un percorso più ampio volto alla realizzazione di sistemi intelligenti completi, progettati per adattarsi alle esigenze applicative e per supportare in modo concreto i processi decisionali all'interno delle aziende.

Bibliografia

- Abdallah, Abdelrahman et al. (2025). «From Retrieval to Generation: Comparing Different Approaches». In: *arXiv preprint arXiv:2502.20245*.
- Bengio, Yoshua et al. (2003). «A Neural Probabilistic Language Model». In: *Journal of Machine Learning Research* 3.Feb, pp. 1137–1155.
- Brill, Eric et al. (ago. 1998). «Beyond n-grams: Can linguistic sophistication improve language modeling?» In: *Proceedings of the 36th Annual Meeting of the Association for Computational Linguistics and the 17th International Conference on Computational Linguistics, Volume 1*, pp. 186–190.
- Brown, Peter F. et al. (1990). «A Statistical Approach to Machine Translation». In: *Computational Linguistics* 16.2, pp. 79–85.
- Brown, Tom et al. (2020). «Language models are few-shot learners». In: *Advances in neural information processing systems*. Vol. 33, pp. 1877–1901.
- Chang, Yupeng et al. (2024). «A survey on evaluation of large language models». In: *ACM transactions on intelligent systems and technology* 15.3, pp. 1–45.
- Chen, Wenhui et al. (2022). «Murag: Multimodal retrieval-augmented generator for open question answering over images and text». In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 5558–5570.
- Church, Kenneth e Robert L. Mercer (1993). «Introduction to the special issue on computational linguistics using large corpora». In: *Computational Linguistics* 19.1, pp. 1–24.
- Devlin, Jacob et al. (2019). «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186.
- Dong, Qingxiu et al. (2024). «A survey on in-context learning». In: *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 1107–1128.
- Douze, Matthijs et al. (2025). «The faiss library». In: *IEEE Transactions on Big Data*.
- Es, Shahul et al. (2024). «Ragas: Automated evaluation of retrieval augmented generation». In: *Proceedings of the 18th conference of the european chapter of the association for computational linguistics: system demonstrations*, pp. 150–158.
- FlowiseAI Contributors (2024). *FlowiseAI Documentation*. Accesso: 2026-03-09. URL: <https://docs.flowiseai.com>.

- Gao, Yunfan et al. (2023). «Retrieval-augmented generation for large language models: A survey». In: *arXiv preprint arXiv:2312.10997* 2.1, p. 32.
- Hochreiter, Sepp e Jürgen Schmidhuber (1997). «Long Short-Term Memory». In: *Neural Computation* 9.8, pp. 1735–1780.
- Holtzman, Ari et al. (2019). «The curious case of neural text degeneration». In: *arXiv preprint arXiv:1904.09751*.
- Houlsby, Neil et al. (2019). «Parameter-efficient transfer learning for NLP». In: *International Conference on Machine Learning (ICML)*. PMLR, pp. 2790–2799.
- Hu, Edward J. et al. (2022). «LoRA: Low-Rank Adaptation of Large Language Models». In: *International Conference on Learning Representations (ICLR)*.
- IBM (2024). *IBM Watson Assistant Documentation*. Accesso: 2026-03-09. URL: <https://cloud.ibm.com/docs/watson-assistant>.
- Jegou, Herve, Matthijs Douze e Cordelia Schmid (2010). «Product quantization for nearest neighbor search». In: *IEEE transactions on pattern analysis and machine intelligence* 33.1, pp. 117–128.
- Jurafsky, Daniel e James H. Martin (2023). *Speech and Language Processing*. 3^a ed. Pearson. Cap. 3, 6, 9, 10.
- Karpukhin, Vladimir et al. (2020). «Dense passage retrieval for open-domain question answering». In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pp. 6769–6781.
- Kimothi, Abhinav (2025). *A simple guide to retrieval augmented generation*. Manning Publications.
- LangChain Development Team (2024). *LangChain Documentation*. Accesso: 2026-03-04. URL: <https://www.langchain.com/docs/>.
- Lenci, Alessandro (2023). *Understanding Natural Language Understanding Systems*. Vol. 35. 2. Sistemi intelligenti, pp. 277–302.
- Lewis, Patrick et al. (2020). «Retrieval-augmented generation for knowledge-intensive nlp tasks». In: *Advances in neural information processing systems* 33, pp. 9459–9474.
- Malkov, Yu A e Dmitry A Yashunin (2018). «Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs». In: *IEEE transactions on pattern analysis and machine intelligence* 42.4, pp. 824–836.
- Marvin, Ggaliwango et al. (2023). «Prompt engineering in large language models». In: *International conference on data intelligence and cognitive informatics*. Springer, pp. 387–402.
- Mavroudis, Vasilios (2024). «LangChain v0.3». In: *Preprints*. Preprint, non peer-reviewed. DOI: 10.20944/preprints202411.0566.v1. URL: <https://doi.org/10.20944/preprints202411.0566.v1>.

- Mikolov, Tomas et al. (2013). «Efficient Estimation of Word Representations in Vector Space». In: *arXiv preprint arXiv:1301.3781*.
- Minaee, Shervin et al. (2024). *Large Language Models: A Survey*. arXiv preprint arXiv:2402.06196.
- MLflow Contributors (2026). *MLflow Documentation*. Accesso: 2026-03-05. URL: <https://mlflow.org/docs/latest/>.
- Naveed, Humza et al. (2023). «A Comprehensive Overview of Large Language Models». In: *arXiv preprint arXiv:2307.06435*.
- Radford, Alec et al. (2018). *Improving Language Understanding by Generative Pre-Training*. Rapp. tecn. OpenAI.
- Schulhoff, Sander et al. (2024). «The Prompt Report: A Systematic Survey of Prompt Engineering Techniques». In: *arXiv preprint arXiv:2406.06608*.
- Sutskever, Ilya, Oriol Vinyals e Quoc V. Le (2014). «Sequence to Sequence Learning with Neural Networks». In: *Advances in Neural Information Processing Systems* 27.
- Tarwani, Kanchan M. e Swathi Edem (2017). «Survey on Recurrent Neural Network in Natural Language Processing». In: *International Journal of Engineering Trends and Technology* 48.6, pp. 301–304.
- Vaswani, Ashish et al. (2017). «Attention is All You Need». In: *Advances in Neural Information Processing Systems*. Vol. 30.
- Vijayakumar, Ashwin K. et al. (2018). «Diverse Beam Search: Decoding Diverse Solutions from Neural Sequence Models». In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. URL: <https://arxiv.org/abs/1610.02424>.
- Zaharia, Matei et al. (2018). «Accelerating the machine learning lifecycle with MLflow.» In: *IEEE Data Eng. Bull.* 41.4, pp. 39–45.
- Zhang, Tianyi et al. (2019). «Bertscore: Evaluating text generation with bert». In: *arXiv preprint arXiv:1904.09675*.